

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	12
Planteamiento y justificación del problema	13
Objetivos	14
1. Marco de referencia	16
1.1. Marco Conceptual	16
1.1.1. Termodinámica del silicio y potencia CMOS	17
1.1.2. Modelo Roofline y regímenes de limitación del rendimiento	17
1.1.3. Telemetría microarquitectónica	20
1.1.4. Espacios de ejecución: <i>user-space</i> y <i>kernel-space</i>	24
1.1.5. Interfaces estándar de observabilidad e instrumentación	25
1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y <i>governors</i>	28
1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks	33
1.1.8. Aprendizaje automático supervisado ligero	33
1.1.9. Evaluación del desempeño de un clasificador	38
1.1.10. Hiperparámetros y selección multiobjetivo	40
1.1.11. Validación anidada y fuga de información	41
1.1.12. Comparaciones estadísticas pareadas	44
1.1.13. Producto Energía–Retardo (EDP)	44
1.1.14. Síntesis: alcance y límites del marco adoptado	45
1.2. Estado del Arte	46

2. Metodología	51
2.1. Enfoque metodológico	51
2.2. Diseño metodológico	51
2.2.1. Población y muestra	51
2.2.2. Plataforma experimental	52
2.2.3. Instrumentos de recolección	53
2.2.4. Variables observadas	56
2.3. Fase 1: Recolección de información y caracterización de cargas	59
2.3.1. Verificación previa de la plataforma	59
2.3.2. Catálogo de cargas de trabajo	59
2.3.3. Selección preliminar y definición del catálogo definitivo	61
2.3.4. Calibración y criterio de etiquetado	62
2.3.5. Caracterización de intensidad operacional y calibración Roofline en GPU	62
2.3.6. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional	63
2.3.7. Observabilidad efectiva y latencia de conmutación de reloj en GPU	64
2.3.8. Matriz experimental y control de sesgos	64
2.3.9. Post-procesamiento y control de calidad de los datos	66
2.3.10. Unidad de observación del conjunto de entrenamiento	67
2.3.11. Medición directa de FLOPs por ventana y su validación empírica	68
2.3.12. Reproducibilidad	69
2.3.13. Selección y registro de características	69
2.4. Fase 2: Desarrollo del modelo de aprendizaje automático	70
2.5. Fase 3: Implementación del agente de control DVFS	71
2.5.1. Arquitectura de dos bucles desacoplados	71
2.5.2. Detección de fase en GPU	71
2.5.3. Diseño del bucle de CPU e integración con el clasificador	71
2.5.4. Derivación <i>offline</i> de la tabla de política	71
2.5.5. Actuación y validación en modo simulado	72
2.6. Fase 4: Validación experimental y análisis de resultados	72
2.6.1. Escenarios de comparación	72
2.6.2. Conmutación de gobernadores y protocolo de medición	72
2.6.3. Cálculo del EDP y prueba de significancia	72
2.6.4. Generación del reporte comparativo	73
2.7. Aspectos éticos	73

3. Resultados	74
3.1. Descripción del conjunto de datos experimental	74
3.1.1. Validación de la medición directa de FLOPs	75
3.2. Validación de la actuación DVFS en CPU	75
3.3. Caracterización de precisión aritmética en el catálogo de GPU	76
3.4. Estado del control de frecuencia en GPU	77
3.5. Medición directa de bytes movidos en memoria (<i>uncore</i>)	78
3.5.1. Validación de esfuerzo y aislamiento de la campaña	78
3.6. Validación cruzada de los techos Roofline con una herramienta independiente	79
3.7. Evolución del catálogo GPU: del DGEMM de referencia a un reemplazo verificable	80
3.8. Barrido de memoria real y calibración de warmup: composición final del dataset	81
3.8.1. Techos Roofline de referencia para GPU	81
3.8.2. Barrido de memoria a escala real de VRAM	81
3.8.3. Calibración de warmup y su efecto sobre el conjunto de entrenamiento	83
3.9. Análisis exploratorio y clasificador diagnóstico del dataset GPU	88
3.9.1. Contrato de <i>features</i> y granularidad de entrenamiento	88
3.9.2. Balanceo de clases y validación	90
3.9.3. Selección de modelo y búsqueda de hiperparámetros	91
3.9.4. Matriz de confusión y limitaciones del resultado	93
3.10. Ampliación del catálogo con kernels de patrón de acceso irregular	97
4. Discusión	99
5. Conclusiones	103
5.1. Limitaciones	104
5.2. Trabajo Futuro	104
Repositorio y recursos complementarios	112

Índice de tablas

2.1. Características de la plataforma experimental.	53
2.2. Señales de telemetría de CPU activas en el instrumento de producción. . . .	58
2.3. Catálogo de cargas de trabajo.	60
2.4. Rejilla fina de frecuencia de la campaña definitiva.	64
2.5. Taxonomía de clasificación de frecuencia por ventana de muestreo.	65
2.6. Taxonomía de anotaciones de calidad por ventana de muestreo.	67
2.7. FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.	69
3.1. Resultado del barrido de memoria a escala real de VRAM para las cinco fa- milias <i>dual_*</i> de GPU.	82
3.2. Warmup calibrado con evidencia real para los 18 kernels de la campaña final GPU.	84
3.3. Los 18 kernels de la campaña final GPU: precisión, intensidad operacional (OI) declarada, ridge de referencia de su propia precisión, $\log_2(\text{OI}/\text{ridge})$ y zona Roofline resultante.	86
3.4. Resultado del clasificador diagnóstico GPU final (<i>leave-one-familia-out</i> , 334 corridas, 11 familias).	91
3.5. Métricas de clasificación estándar del modelo final, calculadas sobre la matriz de confusión acumulada (334 corridas).	93
3.6. Errores del clasificador final por familia algorítmica (<i>leave-one-familia-out</i> , 334 corridas). Familias con 0 errores omitidas.	94
3.7. Parámetros finales calibrados de los 4 kernels CPU de patrón de acceso irregular.	97

Índice de figuras

1.1. Modelo Roofline conceptual	18
1.2. Unidad de Monitorización de Rendimiento (PMU)	20
1.3. Contadores de núcleo y contadores de <i>uncore</i>	22
1.4. Espacios de usuario y de kernel	24
1.5. Lectura de eventos mediante <code>perf_event</code>	26
1.6. Dominios energéticos de Intel RAPL	27
1.7. P-states y C-states del procesador	29
1.8. Jerarquía de los mecanismos DVFS	30
1.9. Solicitudes de frecuencia entre hermanos SMT	31
1.10. Dominios de procesamiento y memoria en GPU	32
1.11. Bosque aleatorio (<i>Random Forest</i>)	35
1.12. Árboles extremadamente aleatorizados (Extra-Trees)	36
1.13. Construcción aditiva de un modelo XGBoost	37
1.14. Representación conceptual del frente de Pareto para dos objetivos en conflicto: maximizar la calidad predictiva y minimizar la latencia de inferencia. Cada punto representa una configuración de hiperparámetros. Las configuraciones A–D no son dominadas, mientras que E es dominada por B al presentar simultáneamente mayor latencia y menor calidad predictiva. Fuente: elaboración propia con base en Deb et al. [17].	41
1.15. Validación cruzada anidada	43
2.1. Ruta de muestreo con estructura circular de un productor y un consumidor .	55
2.2. Recorrido de una campaña a través de las dos capas del instrumento	56
3.1. Barrido de memoria real de GPU	83
3.2. Balance de clases del conjunto de entrenamiento GPU	85
3.3. Posición Roofline de los 18 kernels finales de GPU	87
3.4. Composición de clase por kernel del conjunto de entrenamiento GPU	88

3.5. Matriz de correlación de las señales NVML del contrato de <i>features</i>	90
3.6. F1 macro de los siete modelos comparados	91
3.7. Frente de calidad frente a latencia de inferencia	92
3.8. Matriz de confusión del clasificador diagnóstico GPU	93

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [51] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [15].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 10].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [1], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [51, 15, 3, 10, 1]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. Sobre esa base física, este trabajo asume una cadena de dependencias que articula el resto del capítulo: el régimen que limita el rendimiento de una fase de ejecución, entendido en los términos del modelo Roofline, deja una traza observable en los contadores del procesador y en las interfaces de telemetría de la plataforma; esa traza puede resumirse en un vector de características y, si es suficientemente discriminante, permite inferir el régimen mediante un modelo de clasificación cuyo costo de inferencia sea compatible con un lazo de control; y esa inferencia habilita, a su vez, una política discreta de frecuencia cuyo beneficio neto debe juzgarse con una métrica que integre energía y tiempo. Cada eslabón de esa cadena es una suposición que puede fallar, y las secciones siguientes desarrollan tanto el concepto que lo sustenta como las condiciones bajo las cuales deja de sostenerse.

El capítulo sigue ese mismo orden. Las secciones 1.1.1 a 1.1.2 establecen el fundamento físico y el criterio de clasificación de regímenes; las secciones 1.1.3 a 1.1.5 describen qué puede observarse realmente de ese comportamiento y con qué limitaciones de alcance, privilegio y granularidad; la sección 1.1.6 presenta el mecanismo de actuación y sus restricciones temporales; la sección 1.1.7 delimita las cargas sobre las que se mide; las secciones 1.1.8 a 1.1.11 desarrollan el modelo de inferencia, su evaluación y las condiciones de validez de esa evaluación; y las secciones 1.1.12 y 1.1.13 fijan los criterios con los que se decidirá si el sistema propuesto aporta una mejora defendible.

1.1.1. Termodinámica del silicio y potencia CMOS

La disipación de potencia en un circuito CMOS moderno se descompone en potencia estática (corrientes de fuga) y potencia dinámica, originada por la conmutación de cargas capacitivas. Como DVFS actúa sobre frecuencia y voltaje, su efecto principal recae sobre la potencia dinámica, modelable para una compuerta como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α es el factor de actividad, C la capacitancia efectiva conmutada y V , f el voltaje y la frecuencia de operación [21]. Dado que operar a mayor frecuencia suele exigir mayor voltaje por integridad temporal, reducir frecuencia habitualmente permite también reducir voltaje, con una caída de potencia más que proporcional por la dependencia cuadrática en V . Este acoplamiento es la base física del compromiso energía–rendimiento en DVFS: cuánto se gana en potencia al bajar el punto operativo depende del comportamiento computacional predominante de la aplicación, no solo de la reducción de frecuencia en sí.

1.1.2. Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline analiza el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} es el rendimiento aritmético pico, I la intensidad operacional (operaciones por byte transferido desde memoria) y BW_{pico} el ancho de banda pico [59]. Si el término activo es $I \cdot BW_{\text{pico}}$ el kernel es *memory-bound*; si es P_{pico} , es *compute-bound*.

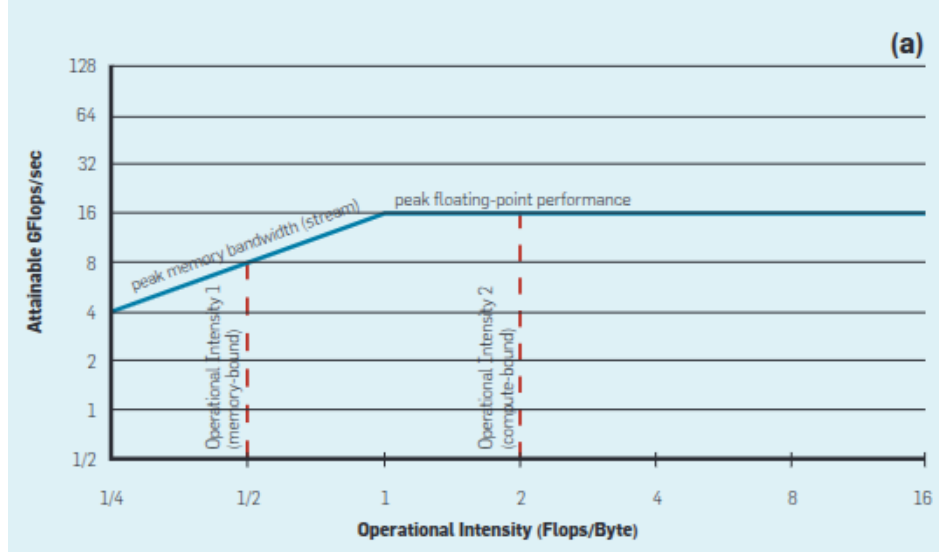


Figura 1.1: Modelo Roofline conceptual: relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Fuente: tomado de Williams et al. [59, Fig. 1(a)].

La intensidad operacional I relaciona el trabajo aritmético con los bytes transferidos desde el nivel de memoria modelado, no con el tamaño nominal del conjunto de trabajo [59, p. 66]. Por ello, que los datos excedan la caché de último nivel solo indica que parte de ellos deberá atravesar niveles inferiores de la jerarquía, pero no determina cuánto tráfico se genera ni cuánta reutilización ocurre. La multiplicación de matrices densas ilustra esta diferencia: mediante bloqueo o mosaico (*tiling*), los bloques pueden reutilizarse y su movimiento entre niveles de memoria se amortiza, sin requerir que las matrices completas residan simultáneamente en caché [22, Secs. 3–4]. Para una cantidad fija de operaciones, esa reducción del tráfico aumenta la intensidad operacional. El tamaño del conjunto de trabajo frente a la caché es, por tanto, un indicio útil, pero no un criterio suficiente para clasificar una carga como *memory-bound* o *compute-bound*.

Punto de inflexión y criterio de clasificación

La frontera entre regímenes es el punto de inflexión o *ridge point*, donde ambas ramas de (1.2) se igualan:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

I_{ridge} se convierte así en el umbral operativo de etiquetado: una intensidad observada inferior se clasifica como *memory-bound* y una superior como *compute-bound*, de acuerdo con la separación de regímenes que plantea el modelo Roofline [59]. La caracterización Roofline

puede determinar techos de cómputo por separado para distintos tipos de dato y precisiones aritméticas [2]. En consecuencia, cuando esos techos difieren, el valor de P_{pico} empleado en (1.3) debe corresponder a la precisión de las operaciones medidas; cada precisión requiere entonces su propio I_{ridge} . Una carga de precisión mixta no puede compararse directamente con un único techo sin definir previamente cómo se tratará esa mezcla.

Por la misma razón, I_{ridge} tampoco es una constante del dispositivo cuando el punto operativo varía. Ambos términos de (1.3) dependen de la frecuencia: el rendimiento aritmético pico escala con el reloj de las unidades de cómputo, mientras que el ancho de banda alcanzable hacia memoria principal responde a un dominio de reloj distinto y no necesariamente en la misma proporción [24, 1]. En consecuencia, un barrido de frecuencia desplaza el punto de inflexión, y comparar intensidades medidas en niveles distintos contra un único I_{ridge} atribuiría al régimen de la carga un efecto que en realidad procede del punto operativo. La calibración debe entonces resolverse por precisión y por nivel de frecuencia, no una sola vez por dispositivo.

Los valores de fábrica sirven como referencia inicial, pero no representan necesariamente los techos alcanzables bajo la configuración experimental. La caracterización Roofline recomienda estimarlos mediante microkernels y condiciones de ejecución representativas [47].

Cálculo de la intensidad operacional observada

Aplicar el criterio anterior exige calcular la intensidad sobre un mismo intervalo de observación:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa el trabajo aritmético realizado y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante el intervalo i [59]. Ambos términos deben corresponder al mismo intervalo temporal; dividir magnitudes obtenidas con granularidades distintas produciría una intensidad sin consistencia física (sección 1.1.3).

Los contadores de rendimiento no miden FLOPs como una magnitud abstracta, sino eventos definidos por una microarquitectura concreta. Incluso eventos aparentemente directos, como las instrucciones retiradas, pueden presentar diferencias de implementación y sensibilidad a las condiciones de medición [58]. Por ello, cualquier conversión desde eventos PMU hacia FLOPs requiere verificar la disponibilidad, la codificación y la semántica del evento utilizado.

El tráfico hacia memoria principal se observa desde un dominio distinto, mediante los contadores *uncore* del controlador de memoria integrado (IMC), descritos en la sección 1.1.3. Cuando su granularidad temporal es más gruesa que la de los contadores de núcleo, los FLOPs deben agregarse sobre el mismo intervalo antes de calcular la intensidad operacional.

1.1.3. Telemetría microarquitectónica

Los procesadores modernos incorporan Unidades de Monitorización de Rendimiento (*Performance Monitoring Units*, PMU), encargadas de registrar mediante hardware eventos asociados al comportamiento interno del procesador. Para ello, disponen de Contadores de Monitorización de Rendimiento (*Performance Monitoring Counters*, PMC), que acumulan ocurrencias de eventos como instrucciones retiradas, ciclos de ejecución, fallos de caché o accesos a determinados recursos microarquitectónicos. Los PMC pueden ser fijos o programables. Los primeros están asociados a eventos predefinidos por la arquitectura y, en la documentación de Intel, se identifican como registros de función fija independientes de los programables ($FC_0, FC_1, \dots$); los segundos permiten seleccionar el evento que se desea contabilizar mediante registros de configuración y se identifican como $PMC_0, PMC_1, \dots, PMC_n$ [32]. En arquitecturas Intel, esta selección se expresa principalmente mediante un código de evento (*Event Select*) y una máscara de unidad (*Unit Mask*), junto con calificadores adicionales cuando el evento lo requiere [32] (Figura 1.2).

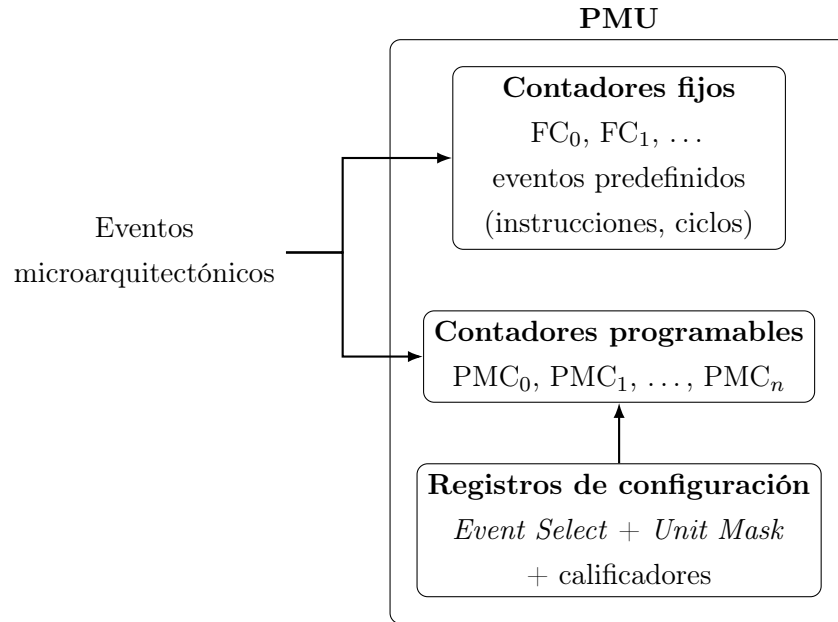


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Fuente: elaboración propia.

La cantidad de contadores disponibles para realizar mediciones simultáneas es limitada y depende de la microarquitectura. En consecuencia, aunque el procesador pueda exponer un catálogo amplio de eventos, solo un subconjunto puede contabilizarse al mismo tiempo. Cuando la cantidad de eventos solicitados supera los contadores físicos disponibles, es necesario compartirlos mediante mecanismos de multiplexación [42], cuyo efecto sobre la medición se discute a continuación.

Multiplexación de contadores y escalado de medidas

Cuando los eventos solicitados exceden los contadores físicos disponibles, el kernel puede multiplexarlos, alternando periódicamente su asignación sobre los contadores de hardware. En este caso, el conteo observado se escala a partir de la relación entre el tiempo durante el cual el evento permaneció habilitado (*time enabled*) y el tiempo durante el cual contó efectivamente (*time running*). Este procedimiento constituye una extrapolación y supone que la tasa de ocurrencia del evento se mantiene aproximadamente estable durante los intervalos no observados, supuesto que puede perder validez cuando el comportamiento de la carga cambia dentro de la ventana [42].

La multiplexación introduce, por tanto, una fuente adicional de incertidumbre en la medición. Para reducirla, conviene limitar el conjunto de eventos a la capacidad física de la PMU y verificar la relación entre *time running* y *time enabled*, que permite identificar si una lectura estuvo sujeta a multiplexación [42, 40].

Eventos genéricos y eventos crudos

Los eventos genéricos son una abstracción portable que el kernel traduce a la codificación concreta de la microarquitectura; los eventos crudos se especifican mediante la codificación nativa del procesador (selector de evento, máscara de subevento y calificadores adicionales) [57, 32]. Cuando el kernel carece de una traducción para un evento disponible en el procesador, puede ser necesario usar su codificación cruda, lo que traslada al instrumento la responsabilidad de verificar que esa codificación corresponde a la microarquitectura evaluada. Una codificación incorrecta puede producir valores sin significado físico aun cuando la apertura del contador no genere un error explícito.

Contadores de núcleo y contadores de *uncore*

Los contadores de núcleo observan eventos atribuibles a un núcleo concreto y pueden asociarse a un proceso. Los contadores de *uncore* observan recursos compartidos del zócalo, como el controlador de memoria, la caché de último nivel distribuida y las interconexiones,

y por tanto son de alcance de sistema, no atribuibles a un proceso específico. Los contadores del controlador de memoria contabilizan el tráfico que alcanza la memoria principal, tanto por accesos de demanda como por precarga [29], por lo que constituyen la fuente estándar para estimar el denominador de la intensidad operacional (ecuación (1.4)). Su acceso, sin embargo, está sujeto a una categoría de privilegio distinta a la de los contadores de núcleo (sección 1.1.5).

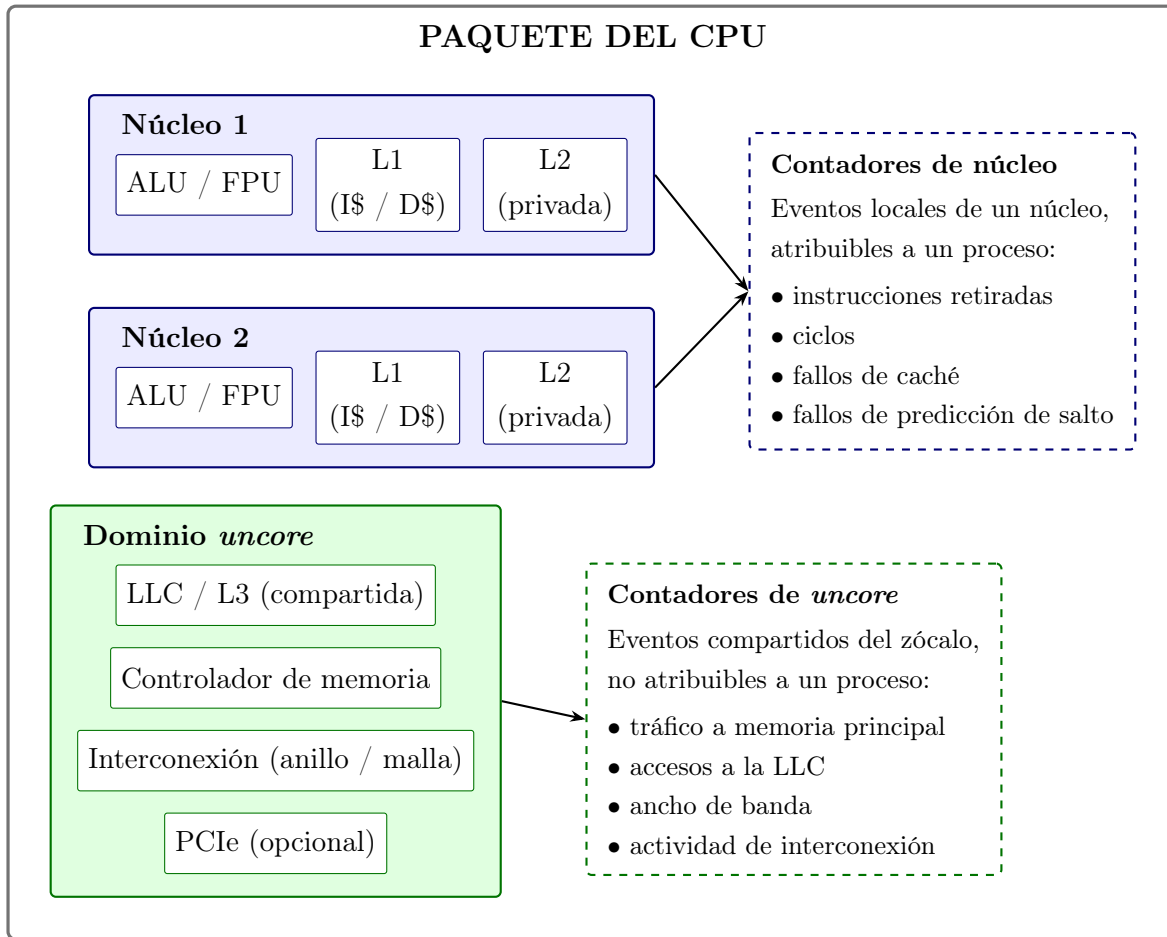


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Los contadores de núcleo observan eventos locales de un núcleo concreto y pueden asociarse a un proceso; los de *uncore* observan recursos compartidos del zócalo y son de alcance de sistema. Fuente: elaboración propia.

Métricas microarquitectónicas de eficiencia

Las instrucciones por ciclo (IPC), las métricas de fallos de caché y los ciclos de estancamiento describen aspectos complementarios del comportamiento de la ejecución. El IPC relaciona las instrucciones retiradas con los ciclos transcurridos y constituye una medida del aprovechamiento efectivo del procesador [23, Ch. 6]. Por su parte, los eventos genéricos de

Linux `cache-references` y `cache-misses` permiten derivar métricas como la tasa de fallos de caché y los fallos por cada mil instrucciones (MPKI). La primera normaliza los fallos respecto al número de referencias, mientras que el MPKI los normaliza respecto al trabajo retirado, por lo que ambas métricas describen aspectos distintos de la presión ejercida sobre la jerarquía de memoria. No obstante, la abstracción genérica de estos eventos no garantiza un nivel específico de caché: aunque habitualmente se asocian al último nivel, su correspondencia puede variar entre procesadores e incluir accesos de precarga o tráfico de coherencia [40].

Los ciclos de estancamiento representan intervalos durante los cuales la ejecución no progresa al ritmo esperado. Sin embargo, un contador agregado de estancamiento del *backend* no permite identificar por sí solo la causa del bloqueo. Este puede originarse tanto en esperas relacionadas con memoria como en limitaciones internas del núcleo, entre ellas dependencias entre instrucciones o saturación de recursos de ejecución [28]. Para obtener una señal más directamente relacionada con presión de memoria pueden emplearse eventos que contabilizan ciclos con cargas pendientes hacia el subsistema de memoria, disponibles de forma específica para determinadas microarquitecturas [31]. Como ocurre con otros eventos crudos de la PMU (sección 1.1.3), su disponibilidad, codificación y semántica deben verificarse para la plataforma evaluada antes de interpretarlos [58, 32].

En conjunto, IPC, fallos de caché y ciclos de estancamiento constituyen señales microarquitectónicas complementarias para caracterizar el comportamiento de una carga y pueden utilizarse como características de entrada para un clasificador. Sin embargo, ninguna de estas métricas determina por sí sola que una ejecución sea *compute-bound* o *memory-bound*. En el modelo Roofline, esta distinción se establece a partir de la intensidad operacional de la carga y de su posición respecto al punto de inflexión de la plataforma [59] (sección 1.1.2).

Ventanas de muestreo y naturaleza temporal de la observación

En modo de conteo, las lecturas de los eventos de rendimiento representan valores acumulados desde su habilitación o último reinicio [40]. Para analizar el comportamiento dentro de una ventana temporal, interesa por tanto el incremento entre dos lecturas consecutivas y no únicamente el valor acumulado. En contraste, existen modos de reporte periódico que entregan directamente los conteos correspondientes a cada intervalo de observación, en lugar del valor acumulado [41].

El período de muestreo determina la resolución temporal de la observación. Intervalos más cortos permiten distinguir cambios de comportamiento con mayor detalle, pero requieren lecturas más frecuentes y pueden aumentar la sobrecarga de medición [41]. Intervalos más

largos reducen la frecuencia de observación, aunque pueden agrupar dentro de una misma ventana comportamientos distintos. Por esta razón, las tasas derivadas deben calcularse utilizando el tiempo realmente transcurrido entre lecturas, medido con una fuente temporal que no se vea afectada por ajustes del reloj del sistema [39].

1.1.4. Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos emplean distintos niveles de privilegio para regular el acceso a los recursos del sistema. Los procesos en modo usuario operan con acceso restringido, mientras que el modo kernel permite acceder también a las regiones de memoria y recursos reservados al núcleo. Operaciones sensibles, como la gestión de memoria, la entrada/salida o la interacción directa con el hardware, requieren estos privilegios para proteger las estructuras críticas del sistema frente a procesos ordinarios [35, ch. 2]. En consecuencia, las aplicaciones acceden a dichos recursos mediante interfaces controladas por el sistema operativo, principalmente llamadas al sistema en entornos Unix y Linux [45]. Del mismo modo, cualquier mecanismo de monitoreo o control implementado en espacio de usuario debe apoyarse en interfaces que el kernel o la propia plataforma expongan explícitamente.

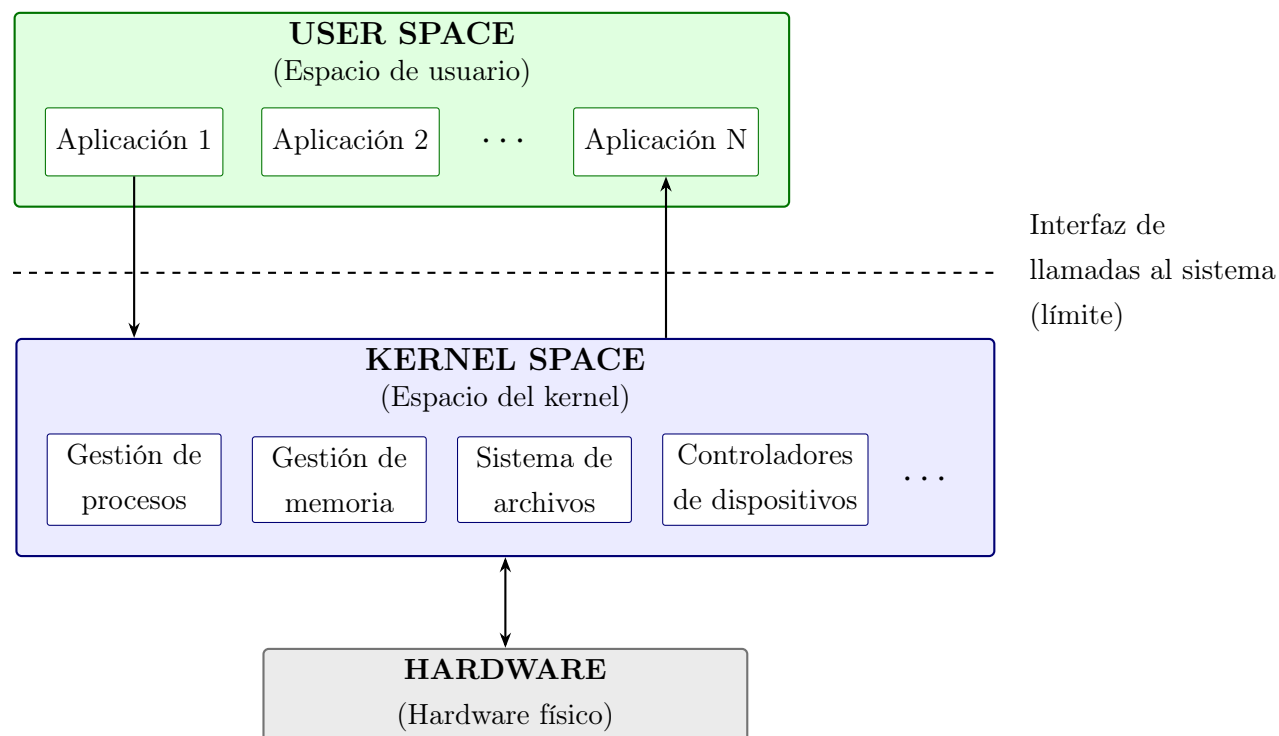


Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico, mediada por la interfaz de llamadas al sistema. Fuente: elaboración propia.

1.1.5. Interfaces estándar de observabilidad e instrumentación

La observabilidad de sistemas heterogéneos CPU–GPU depende de interfaces que expongan, desde *user-space*, métricas de rendimiento, utilización y consumo sin modificar el kernel ni los controladores. Las plataformas modernas exponen `perf_event` para contadores de CPU, Intel RAPL para energía de CPU, y NVIDIA Management Library (NVML) para telemetría de GPU [10, 57, 16, 48].

La interfaz `perf_event`

Linux proporciona la infraestructura `perf_event` para acceder desde espacio de usuario a los mecanismos de monitorización de rendimiento del procesador. A través de esta interfaz, el kernel configura los eventos de la PMU, administra su asignación sobre los contadores físicos disponibles y expone los conteos obtenidos a las herramientas de instrumentación [40]. De esta forma, herramientas como `perf` pueden observar eventos asociados a instrucciones, ciclos de ejecución, comportamiento de caché y otros aspectos microarquitectónicos sin acceder directamente a los registros de la PMU. En particular, el modo de reporte periódico introducido en la sección 1.1.3 corresponde al subcomando `perf stat`, que entrega los conteos de cada intervalo de observación en lugar del valor acumulado [41].

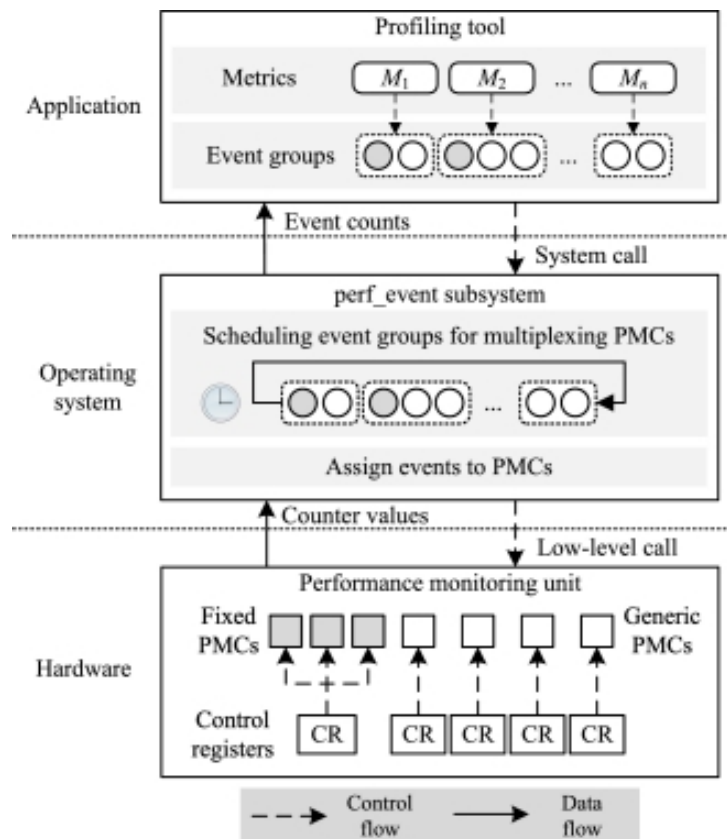


Figura 1.5: Flujo de configuración y lectura de eventos de rendimiento mediante el subsistema `perf_event` de Linux. Fuente: tomado de Liu et al. [42].

La interfaz permite además definir el alcance de una medición. Un evento puede asociarse a una tarea, siguiendo su ejecución aunque sea planificada sobre distintas CPU, o a una CPU determinada, contabilizando la actividad que ocurre en ese procesador durante el intervalo observado [40]. Esta distinción determina qué actividad contribuye al conteo y, por tanto, condiciona la interpretación de las métricas obtenidas.

Niveles de privilegio en el acceso a contadores

El acceso a los contadores de rendimiento desde espacio de usuario está regulado en Linux por el parámetro `perf_event_paranoid`, que establece restricciones para los procesos que no disponen de privilegios específicos de monitorización [40]. A medida que su valor aumenta, se restringe progresivamente el alcance de las mediciones: los niveles más restrictivos impiden la monitorización de alcance de CPU o sistema y pueden limitar la observación a la ejecución del propio proceso en espacio de usuario.

Esta restricción es especialmente relevante para las PMU de *uncore*, cuyos eventos representan actividad de componentes compartidos a nivel de paquete o socket y, por tanto, no

se atribuyen exclusivamente a un proceso individual (sección 1.1.3) [40]. En consecuencia, la disponibilidad efectiva de una métrica depende tanto de las capacidades de monitorización del hardware como de las políticas de acceso establecidas por el sistema operativo.

Intel RAPL

RAPL (*Running Average Power Limit*) es un mecanismo de gestión y monitorización energética incorporado en procesadores Intel que permite observar el consumo de energía mediante dominios asociados a distintos componentes del sistema. Entre los dominios disponibles pueden encontrarse el paquete completo del procesador (*Package*), los núcleos (*PP0/Core*) y la memoria DRAM, aunque su disponibilidad depende de la microarquitectura [16, 53].

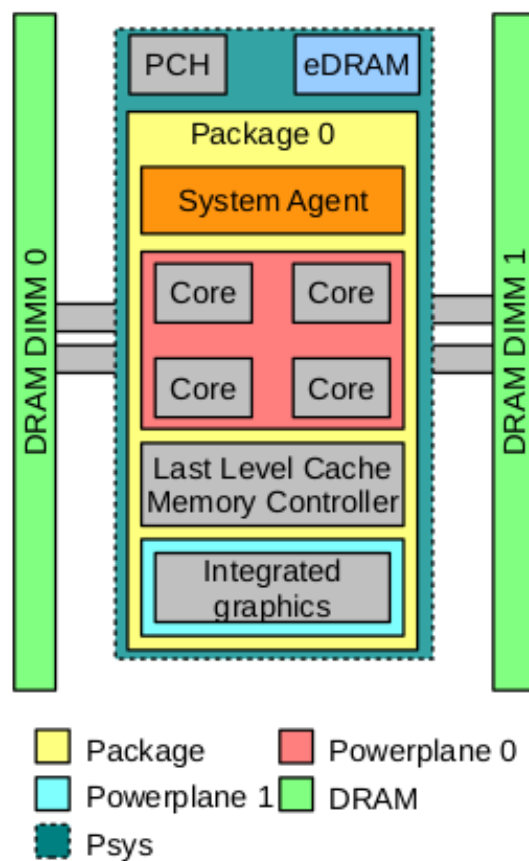


Figura 1.6: Dominios energéticos expuestos mediante Intel RAPL. Fuente: tomado de Thamm y Leser [53].

La información energética se mantiene mediante contadores acumulativos expuestos por registros específicos del modelo (MSR), cuyos valores deben interpretarse utilizando las unidades definidas por la plataforma [16, 53]. Debido a que estos contadores poseen un rango

finito, pueden presentar desbordamiento durante ejecuciones prolongadas, por lo que este comportamiento debe considerarse al calcular diferencias entre lecturas consecutivas. Asimismo, RAPL no representa necesariamente el consumo energético completo del nodo, sino únicamente los dominios que el procesador expone mediante esta infraestructura [53].

NVIDIA Management Library y Nsight Compute

NVIDIA Management Library (NVML) proporciona una interfaz para consultar el estado operativo de GPUs NVIDIA, incluyendo métricas de utilización, memoria, potencia y frecuencia [48]. Su información se obtiene a nivel del dispositivo y mediante períodos de muestreo definidos por el controlador. Por ejemplo, la utilización de GPU representa la fracción del intervalo durante la cual uno o más kernels estuvieron en ejecución, mientras que la utilización de memoria indica la fracción del intervalo durante la cual la memoria global fue leída o escrita [48]. Estas métricas describen actividad y estado operativo, pero no proporcionan directamente el número de operaciones aritméticas ni el volumen de datos transferidos necesarios para calcular la intensidad operacional del modelo Roofline.

Nsight Compute, en cambio, es un perfilador orientado al análisis detallado de kernels CUDA. La herramienta permite recolectar métricas de rendimiento asociadas al uso de unidades de cómputo, jerarquía de memoria, instrucciones y tráfico de datos mediante contadores de hardware e instrumentación del kernel [49]. Debido a que no todas las métricas pueden obtenerse simultáneamente, algunos perfiles requieren múltiples pasadas o repeticiones del lanzamiento analizado, lo que incrementa significativamente la sobrecarga de la medición [49].

NVML y Nsight Compute responden, por tanto, a niveles de observación distintos: la primera proporciona telemetría agregada del estado del dispositivo, mientras que el segundo permite caracterizar con mayor detalle el comportamiento de los kernels ejecutados. Esta diferencia determina el tipo de información que puede obtenerse de cada herramienta y su adecuación a distintos tipos de análisis.

1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y *governors*

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) permite modificar el punto operativo del procesador para adaptar su capacidad de cómputo y consumo energético a las necesidades de la carga. Estas transiciones se apoyan en mecanismos hardware de gestión de potencia encargados de materializar los cambios solicitados por el sistema operativo [25]. Dentro del modelo ACPI, los estados de rendimiento

o P-states representan distintos niveles de capacidad mientras el procesador permanece activo en C0. P0 corresponde al mayor nivel de rendimiento, mientras que estados sucesivos reducen progresivamente dicha capacidad. Los C-states, en cambio, representan estados de inactividad con distintos compromisos entre ahorro energético y latencia de reactivación.

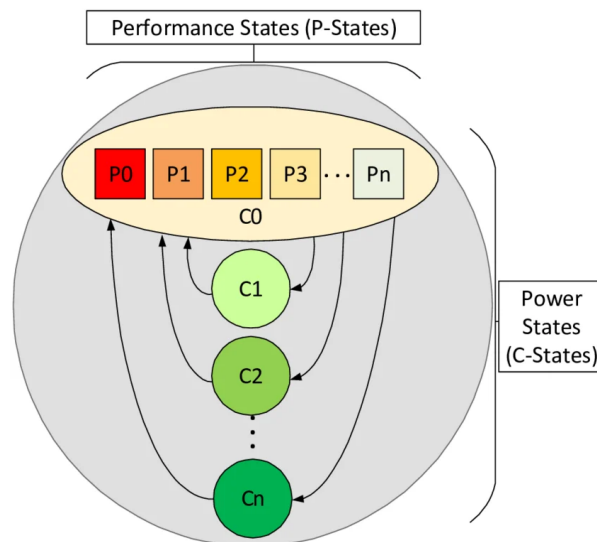


Figura 1.7: Relación conceptual entre estados de rendimiento (P-states) y estados de inactividad (C-states) del procesador. Fuente: tomado de Hebbar y Milenković [25].

Sobre estos mecanismos operan los *governors*, políticas software que determinan el nivel de rendimiento solicitado en función del estado del sistema. En el subsistema CPUFreq de Linux, los gobernadores genéricos **performance** y **powersave** solicitan respectivamente los límites superior e inferior de frecuencia disponibles, mientras que **ondemand**, **conservative** y **schedutil** modifican el nivel solicitado dinámicamente. El gobernador **userspace**, por su parte, permite que una aplicación en espacio de usuario solicite explícitamente la frecuencia de una política [25, 37].

La política de selección y el mecanismo encargado de actuar sobre el hardware son componentes distintos. Controladores como **acpi-cpufreq** operan junto con los gobernadores genéricos de CPUFreq, mientras que **intel_pstate** puede funcionar en modos activo y pasivo. En modo activo, **intel_pstate** utiliza sus propios algoritmos **performance** y **powersave**; este último realiza selección dinámica del nivel de rendimiento y no equivale al gobernador genérico **powersave**. En modo pasivo, el controlador se integra con los gobernadores genéricos de CPUFreq y se identifica como **intel_cpufreq** [38].

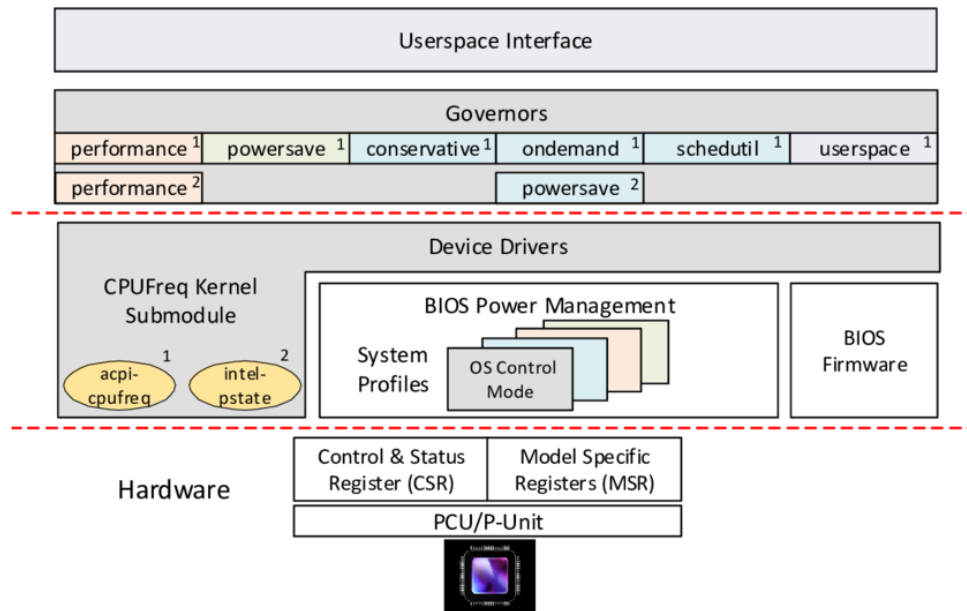


Figura 1.8: Jerarquía de los mecanismos de gestión de rendimiento desde la interfaz de espacio de usuario hasta el hardware. Fuente: tomado de Hebbar y Milenković [25].

Las transiciones entre niveles de rendimiento no son instantáneas y su tiempo de respuesta depende del procesador, del controlador utilizado y del grado de autonomía concedido al hardware. Mecanismos como los *Hardware-Managed P-states* (HWP) trasladan parte de la selección del estado operativo al propio procesador, reduciendo la intervención directa del sistema operativo [25]. En consecuencia, la granularidad temporal de una política DVFS debe considerar la capacidad efectiva de la plataforma para materializar sus decisiones.

Esa autonomía del hardware alcanza también al límite superior del rango. Los procesadores modernos incorporan mecanismos de incremento oportunista de frecuencia, conocidos en las plataformas Intel como Turbo Boost, que permiten superar la frecuencia base cuando las condiciones térmicas, de corriente y de número de núcleos activos lo admiten. Su activación depende del estado del procesador y no de una solicitud explícita del sistema operativo, por lo que el reloj efectivamente alcanzado puede no coincidir con el nivel solicitado a través de la interfaz de escalado [38, 25]. Esta distinción es determinante para cualquier experimento que compare niveles de frecuencia fijos: si el mecanismo permanece habilitado, distintos niveles solicitados pueden materializarse en relojes efectivos similares, y la diferencia de potencia atribuida al nivel quedaría contaminada. Por ello, el estado de este mecanismo debe fijarse explícitamente y la frecuencia efectivamente alcanzada debe verificarse por relectura durante la ejecución, en lugar de asumirse a partir del valor solicitado.

Dominios de frecuencia y topología SMT

El alcance de una solicitud de frecuencia no coincide necesariamente con la afinidad de ejecución de una carga. En Linux, el subsistema CPUFreq organiza el control mediante políticas asociadas a una o varias CPU lógicas, cuya composición depende del hardware y del controlador activo. Los atributos `affected_cpus` y `related_cpus` permiten identificar las CPU asociadas a cada política [37].

Esta distinción es especialmente relevante cuando está habilitado *simultaneous multithreading* (SMT). Dos procesadores lógicos hermanos comparten un mismo núcleo físico y, aunque puedan exponer límites de rendimiento independientes a nivel de software, su frecuencia efectiva no es necesariamente independiente. En particular, la documentación de `intel_pstate` establece que, con SMT habilitado, una solicitud de mayor rendimiento realizada por uno de los hermanos puede elevar el nivel efectivo alcanzado por el otro, incluso por encima del límite establecido en su política individual [38].

Por tanto, la colocación de la carga y el control de frecuencia deben tratarse como decisiones distintas. Restringir la ejecución a un solo procesador lógico por núcleo evita la contención entre hermanos SMT, pero no garantiza por sí solo el aislamiento del estado de frecuencia. Cuando se requiere controlar el punto operativo de un núcleo físico, deben considerarse también las solicitudes de rendimiento de sus procesadores lógicos hermanos y verificarse la frecuencia efectivamente alcanzada en la plataforma.

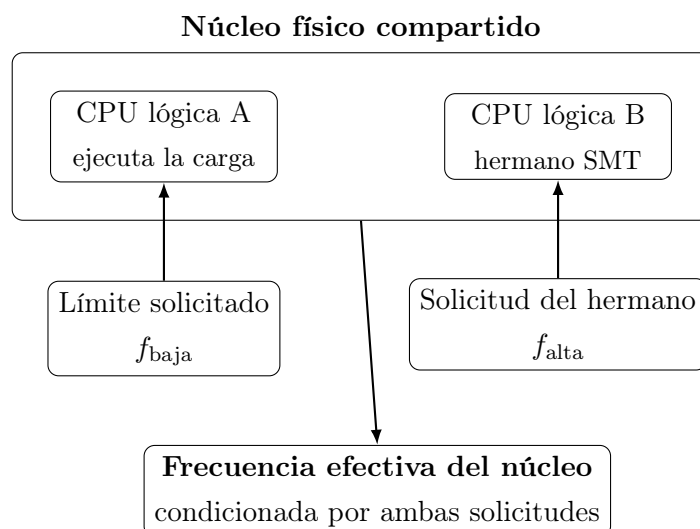


Figura 1.9: Relación conceptual entre las solicitudes de rendimiento de dos procesadores lógicos hermanos y la frecuencia efectiva de un núcleo físico con SMT habilitado. Fuente: elaboración propia con base en la documentación de `intel_pstate` [38].

Particularidades de DVFS en GPU

En las GPU, el control de frecuencia involucra dominios funcionales con sensibilidades distintas frente a la carga. De forma general, puede distinguirse un dominio de procesamiento, asociado principalmente a los multiprocesadores de *streaming* (SM), y un dominio de memoria, asociado a la memoria principal del dispositivo. La frecuencia de ambos subsistemas puede influir de manera diferente sobre el rendimiento y el consumo energético de una aplicación [24, 1].

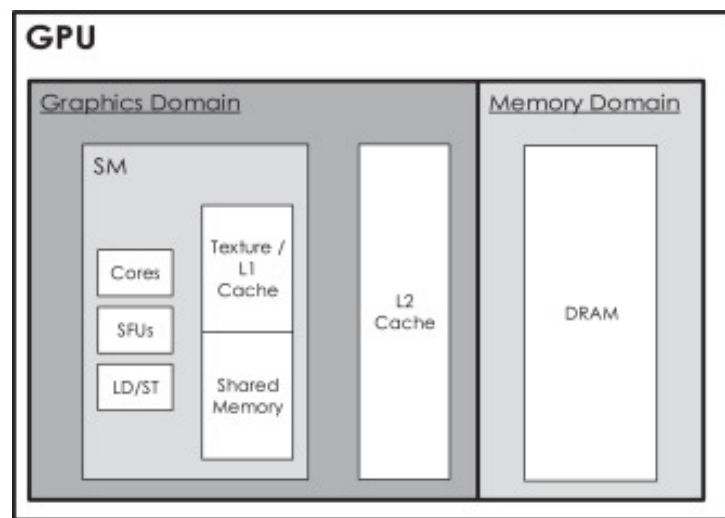


Figura 1.10: Representación conceptual de los dominios de procesamiento y memoria en una GPU. Fuente: tomado de Guerreiro et al. [24].

La sensibilidad a estos dominios depende del régimen de ejecución. En cargas limitadas por cómputo, cambios en la frecuencia del dominio de procesamiento tienden a afectar de forma más directa el tiempo de ejecución. En cargas limitadas por memoria, la influencia de esta frecuencia disminuye y adquiere mayor relevancia el comportamiento del subsistema de memoria [24, 1]. Esta relación también condiciona el consumo energético: una reducción de frecuencia puede disminuir la potencia instantánea, pero aumentar la energía total si el incremento del tiempo de ejecución compensa dicho ahorro [1].

A estas diferencias entre dominios se suma el costo temporal de modificar la frecuencia. En GPU, una transición requiere que la solicitud emitida desde el host sea recibida y aplicada por el acelerador, por lo que el cambio no es instantáneo. La latencia resultante depende de la arquitectura y del par de frecuencias involucrado, y puede alcanzar escalas de varios milisegundos [55]. Este costo limita la granularidad con la que puede aplicarse DVFS durante la ejecución, ya que cambios demasiado frecuentes pueden introducir una sobrecarga comparable al tiempo de ejecución de fases cortas.

1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks

Las aplicaciones de cómputo de alto rendimiento pueden combinar fases con demandas distintas sobre las unidades de cómputo, la jerarquía de memoria y otros recursos del sistema. Por esta razón, su comportamiento no siempre se describe adecuadamente mediante una única medida agregada de ejecución. El análisis puede realizarse sobre unidades más acotadas, denominadas kernels, que representan operaciones computacionales bien delimitadas, como una multiplicación de matrices, una transformada de Fourier o una actualización de *stencil*. Al concentrar un patrón definido de cómputo y acceso a datos, los kernels permiten estudiar con mayor claridad la relación entre intensidad operacional, ancho de banda de memoria y capacidad de cómputo [59].

Un benchmark es una carga utilizada para medir y comparar el comportamiento de un sistema bajo condiciones reproducibles. Dependiendo de su propósito, puede representar una aplicación completa, una parte significativa de ella o un conjunto de operaciones representativas. Su utilidad radica en proporcionar una base común para comparar arquitecturas, configuraciones o mecanismos de gestión del sistema [23, Ch. 12].

Un microbenchmark persigue un objetivo distinto: aislar una propiedad concreta del hardware o del software mediante una carga deliberadamente simple y controlada. Entre sus usos habituales se encuentran la medición del ancho de banda de memoria, la latencia de acceso o la capacidad de ejecución aritmética [23, Ch. 12]. En consecuencia, los benchmarks buscan representar comportamientos relevantes de una carga, mientras que los microbenchmarks permiten caracterizar de forma aislada mecanismos específicos de la plataforma.

1.1.8. Aprendizaje automático supervisado ligero

El aprendizaje supervisado busca estimar una función que relacione un conjunto de variables de entrada con una salida a partir de observaciones previamente etiquetadas. El modelo se ajusta utilizando datos de entrenamiento y su capacidad de generalización se evalúa sobre observaciones que no participaron en dicho ajuste [33, Ch. 2].

En el contexto de este trabajo, el término *ligero* se utiliza para describir modelos cuya inferencia puede ejecutarse con baja latencia y un costo computacional acotado, de modo que su incorporación a un bucle de control no introduzca una sobrecarga comparable al beneficio que se pretende obtener. Esta propiedad no depende únicamente de la familia del modelo, sino también de su complejidad efectiva, determinada por factores como la profundidad de los árboles y el número de estimadores del ensamble.

Familias de modelos candidatas

Regresión logística. La regresión logística modela la probabilidad de pertenencia a una clase binaria a partir de una combinación lineal de las características. Para un vector de entrada x , se define

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n, \quad P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}. \quad (1.5)$$

La transformación sigmoide restringe la salida al intervalo $[0, 1]$. Su estructura proporciona un modelo de baja complejidad e interpretación directa, aunque produce una frontera de decisión lineal en el espacio original de características, lo que limita su capacidad para representar separaciones fuertemente no lineales [33, Ch. 4].

Árbol de decisión. Un árbol de decisión particiona recursivamente el espacio de características mediante reglas asociadas a variables individuales. Cada observación recorre una secuencia de divisiones desde la raíz hasta una hoja, donde se determina la clase predicha. Esta estructura permite representar fronteras de decisión no lineales y facilita la inspección de las reglas aprendidas. Sin embargo, árboles de elevada profundidad pueden presentar alta varianza y ajustarse en exceso a los datos de entrenamiento [33, Ch. 8].

Bosques aleatorios (*Random Forest*). Random Forest combina múltiples árboles de decisión mediante un esquema de ensamble. Cada árbol se entrena sobre una muestra *bootstrap* del conjunto de entrenamiento y, durante la construcción de sus divisiones, solo se considera un subconjunto aleatorio de características. Estos mecanismos introducen diversidad entre los árboles y reducen su correlación. En clasificación, la predicción del bosque se obtiene agregando las decisiones individuales de los árboles, habitualmente mediante votación mayoritaria [8].

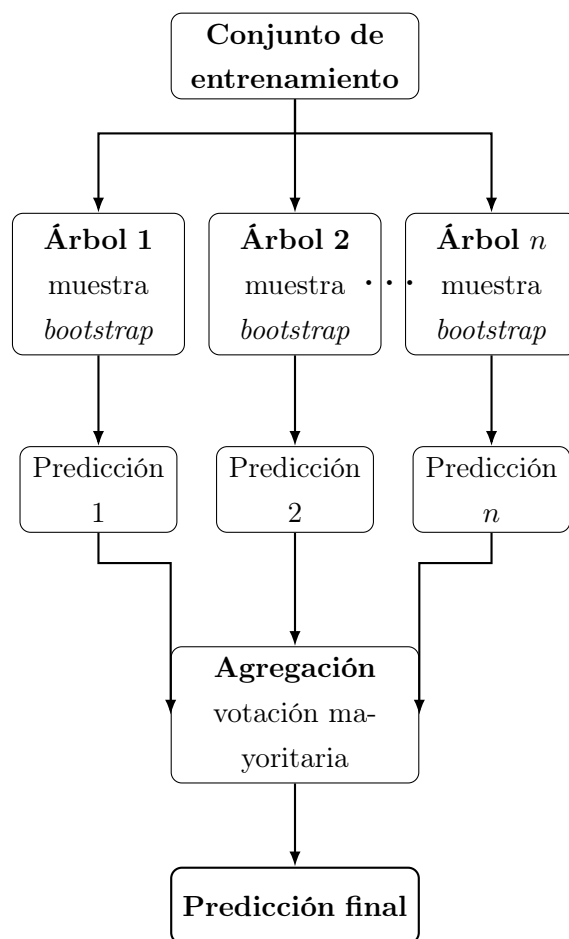


Figura 1.11: Esquema conceptual del funcionamiento de un bosque aleatorio (*Random Forest*). Los árboles se construyen a partir de muestras *bootstrap*, incorporando además selección aleatoria de características durante las divisiones, y sus predicciones se agregan para obtener la decisión final. Fuente: Adaptado de IBM [27].

Árboles extremadamente aleatorizados (*Extra-Trees*). Extra-Trees es un método de ensamble de árboles que introduce un grado adicional de aleatorización durante la construcción de las divisiones. En cada nodo se selecciona aleatoriamente un subconjunto de características y se genera un punto de corte aleatorio para cada característica candidata; posteriormente se elige la mejor división entre esos candidatos aleatorios. En su formulación original, los árboles se construyen utilizando el conjunto de entrenamiento completo, en lugar de muestras *bootstrap* independientes [20].

La aleatorización de los puntos de corte modifica el compromiso entre sesgo y varianza respecto a métodos que optimizan cada división y reduce el costo de construcción de los árboles. El efecto concreto depende del grado de aleatorización, de la configuración del ensamble y de las características de los datos [20].

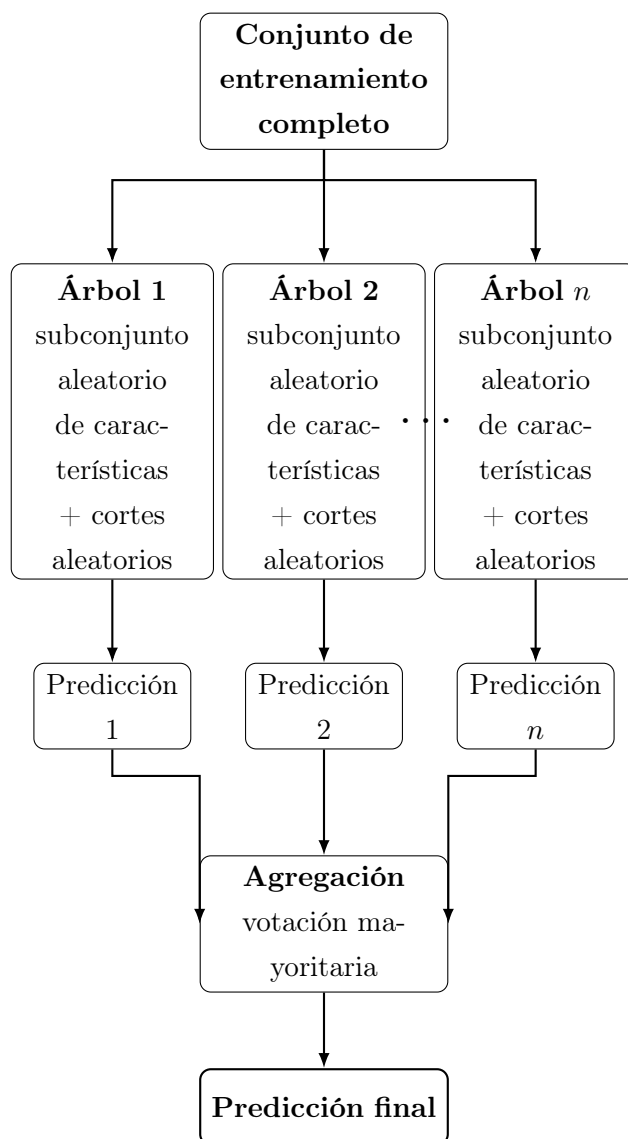


Figura 1.12: Esquema conceptual de un ensemble de árboles extremadamente aleatorizados (*Extra-Trees*). En su formulación original, los árboles utilizan el conjunto de entrenamiento completo e introducen aleatoriedad tanto en la selección de características como en los puntos de corte considerados durante las divisiones. Fuente: elaboración propia con base en Geurts et al. [20].

XGBoost. XGBoost implementa un ensemble aditivo de árboles construido de forma secuencial. En cada iteración se incorpora un nuevo árbol con el propósito de reducir la función objetivo del modelo acumulado, utilizando información de primer y segundo orden de la función de pérdida. La formulación incorpora además términos de regularización destinados a controlar la complejidad de los árboles [13]. Esta estructura permite representar relaciones no lineales e interacciones entre características, aunque su costo de inferencia depende del número y la profundidad de los árboles que conforman el ensemble.

Las cinco familias consideradas presentan compromisos diferentes entre capacidad de representación, interpretabilidad y costo de inferencia. La regresión logística proporciona una referencia lineal; el árbol de decisión introduce una frontera no lineal directamente inspeccionable; Random Forest y Extra-Trees combinan múltiples árboles mediante estrategias distintas de aleatorización y agregación; y XGBoost construye el ensamble de forma secuencial mediante potenciación. La complejidad efectiva de cada modelo depende además de sus hiperparámetros, por lo que su comparación debe considerar tanto el desempeño predictivo como el costo del modelo resultante.

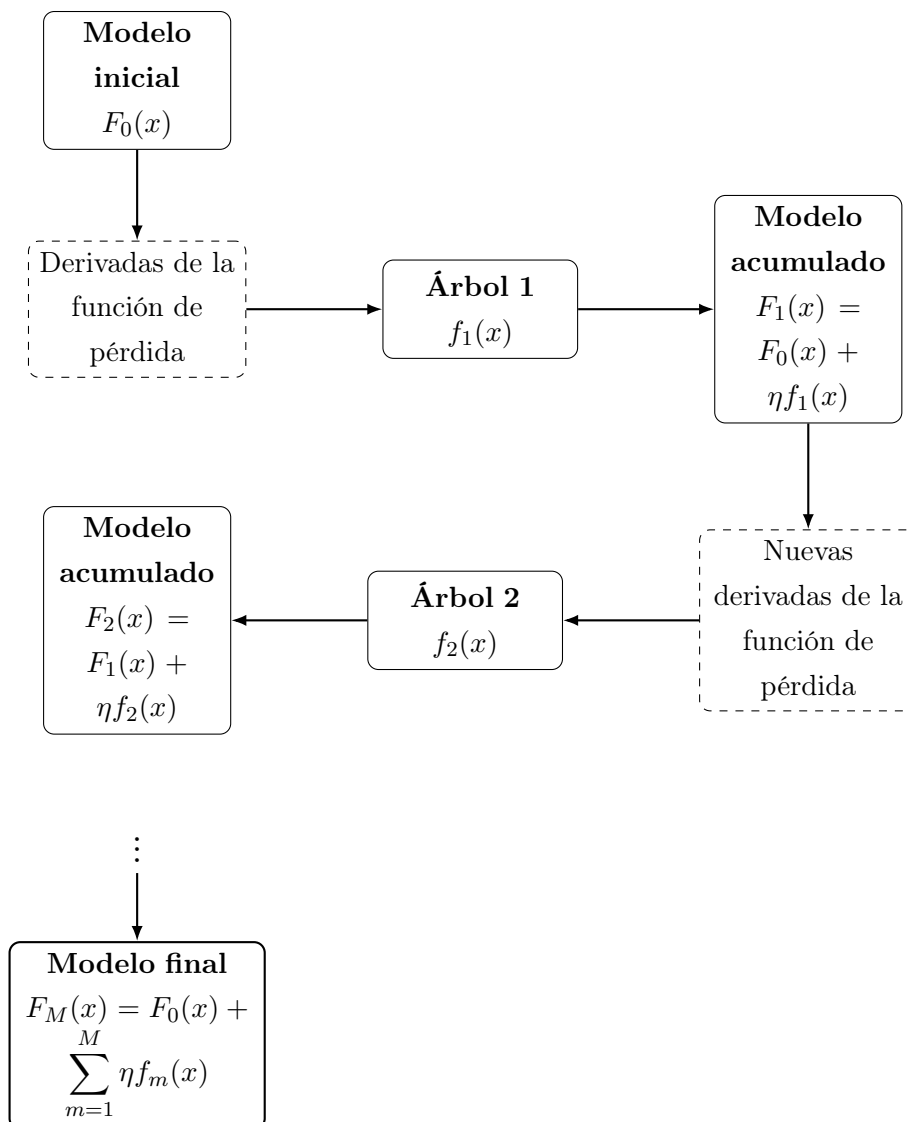


Figura 1.13: Esquema conceptual de la construcción secuencial y aditiva de un modelo XGBoost. Cada nuevo árbol se obtiene a partir de la información de la función de pérdida del modelo acumulado y su contribución se incorpora a la predicción existente. Fuente: elaboración propia con base en Chen y Guestrin [13].

1.1.9. Evaluación del desempeño de un clasificador

La matriz de confusión relaciona las clases reales con las clases predichas y permite distinguir, en clasificación binaria, verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN). A partir de estos valores pueden derivarse distintas métricas, cada una sensible a aspectos diferentes del comportamiento del clasificador [52].

La exactitud (*accuracy*) expresa la proporción total de predicciones correctas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1.6)$$

Aunque proporciona una medida directa de la tasa global de aciertos, puede resultar poco informativa cuando existe un desequilibrio importante entre las clases, ya que una elevada proporción de ejemplos de la clase mayoritaria puede dominar el resultado [9]. Una alternativa es la exactitud balanceada (*balanced accuracy*), que en clasificación binaria corresponde al promedio entre la sensibilidad y la especificidad:

$$\text{BalancedAccuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right). \quad (1.7)$$

La precisión y la sensibilidad se definen, respectivamente, como

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (1.8)$$

A partir de ellas, el puntaje F_1 se obtiene mediante su media armónica:

$$F_1 = 2 \frac{\text{Precision} \text{ Recall}}{\text{Precision} + \text{Recall}}. \quad (1.9)$$

Esta métrica permite considerar conjuntamente ambos aspectos del desempeño sobre una clase de interés, sin sustituir por ello a las demás métricas de evaluación [52].

Precisión, sensibilidad y F_1 se definen respecto a una clase tomada como positiva, de modo que su valor cambia según cuál de las dos clases se elija. Reportar un único valor global oculta esa asimetría y puede enmascarar un desempeño pobre sobre la clase menos representada. Por ello conviene reportar las tres métricas por separado para cada clase y, cuando se requiera un resumen único, emplear el promedio macro, que promedia el valor obtenido en cada clase otorgándoles el mismo peso con independencia de su frecuencia [52].

Cuando las observaciones se agrupan en unidades experimentales heterogéneas, el mismo argumento se extiende a esas unidades: un promedio sobre todas las observaciones puede quedar dominado por los grupos más numerosos y ocultar un comportamiento distinto en el resto.

Otra medida derivada de los cuatro componentes de la matriz de confusión es el coeficiente de correlación de Matthews (MCC):

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (1.10)$$

El MCC toma valores entre -1 y 1 , donde 1 representa concordancia perfecta entre las clases reales y predichas, 0 indica ausencia de correlación y -1 corresponde a una discordancia perfecta. Al incorporar simultáneamente los cuatro componentes de la matriz de confusión, constituye una medida complementaria para evaluar clasificadores binarios, especialmente cuando la distribución de las clases no es uniforme [14].

Líneas base de referencia

Ninguna de las métricas anteriores es interpretable en términos absolutos sin una referencia con la cual compararla. Un clasificador trivial que predice siempre la clase mayoritaria alcanza una exactitud igual a la frecuencia de esa clase y puede incluso obtener un F_1 elevado sobre ella, sin haber aprendido ninguna relación entre las características y la etiqueta [14]. Por esta razón, el desempeño de un modelo se contrasta frente a líneas base sin capacidad predictiva, habitualmente la predicción constante de la clase mayoritaria y la asignación aleatoria que respeta las proporciones observadas.

La comparación debe realizarse bajo el mismo esquema de partición que el modelo evaluado, de modo que ambas estimaciones estén sujetas a las mismas condiciones. Solo una mejora apreciable y sostenida sobre esas referencias permite atribuir el resultado a información contenida en las características. Cuando un modelo no supera su línea base, la conclusión pertinente es que las señales disponibles no bastan para separar las clases en ese contexto de observación, lo que constituye un resultado informativo por sí mismo y no un fallo del procedimiento de evaluación.

Costo de inferencia como dimensión de evaluación

Cuando un clasificador se destina a operar dentro de un lazo de decisión sujeto a un presupuesto temporal, su costo de inferencia deja de ser un detalle de implementación y pasa

a ser una dimensión de evaluación equiparable a la calidad predictiva. La magnitud relevante es la latencia por predicción individual, ya que el lazo decide sobre una observación a la vez y no sobre lotes.

Esta latencia depende de la microarquitectura, de la implementación empleada y de la complejidad efectiva del modelo entrenado, entre otros factores el número y la profundidad de los árboles de un ensamble. En consecuencia, no puede estimarse analíticamente a partir de la familia del modelo ni extrapolarse desde otra plataforma: debe medirse sobre el mismo hardware en el que operará.

Reportar únicamente la latencia media es insuficiente para un componente sujeto a un plazo. Lo que condiciona la viabilidad es el peor caso acotado, porque una predicción ocasionalmente lenta puede exceder la ventana de decisión aunque el promedio se mantenga holgado. Por ello la caracterización se expresa mediante percentiles altos de la distribución, por ejemplo los percentiles 95 y 99, y no solo mediante su tendencia central, siguiendo el principio de evaluación estadísticamente rigurosa del desempeño de un sistema [19]. Un modelo cuya latencia en la cola de la distribución compromete el plazo del lazo de control no resulta admisible aunque su exactitud sea superior, de modo que la elección final se plantea como un compromiso entre dos criterios en conflicto (sección 1.1.10).

1.1.10. Hiperparámetros y selección multiobjetivo

Los parámetros internos de un modelo se estiman durante el ajuste a partir de los datos de entrenamiento. Los hiperparámetros, en cambio, controlan la estructura del modelo o el propio procedimiento de ajuste y deben fijarse antes de este, por lo que no se obtienen del mismo proceso de estimación. Son ejemplos la fuerza de regularización de la regresión logística, la profundidad máxima de un árbol, el número de estimadores de un ensamble o la tasa de aprendizaje de un modelo potenciado. Su elección modifica simultáneamente la capacidad predictiva y la complejidad efectiva del modelo resultante [7].

La configuración se selecciona evaluando distintas combinaciones sobre datos reservados para ese fin. Entre las estrategias habituales se encuentran la búsqueda exhaustiva sobre una rejilla de valores predefinidos y el muestreo aleatorio del espacio de configuración; esta última resulta preferible cuando solo una fracción de los hiperparámetros influye de forma apreciable sobre el resultado, ya que dedica el mismo presupuesto de evaluaciones a explorar más valores distintos de las dimensiones relevantes [7]. En todos los casos, la comparación exige que la estimación final del desempeño se realice sobre datos que no intervinieron en esta selección, condición que motiva el procedimiento descrito en la sección 1.1.11.

Cuando la elección debe atender simultáneamente a varios criterios en conflicto, como la calidad predictiva y el costo de inferencia de la sección 1.1.9, ninguna métrica escalar basta por sí sola para ordenar las alternativas. La comparación se formaliza entonces mediante la relación de dominancia: una configuración domina a otra si no es peor en ninguno de los criterios considerados y es estrictamente mejor en al menos uno. Las configuraciones que no son dominadas por ninguna otra conforman el frente de Pareto, que representa el conjunto de compromisos alcanzables entre los objetivos [17].

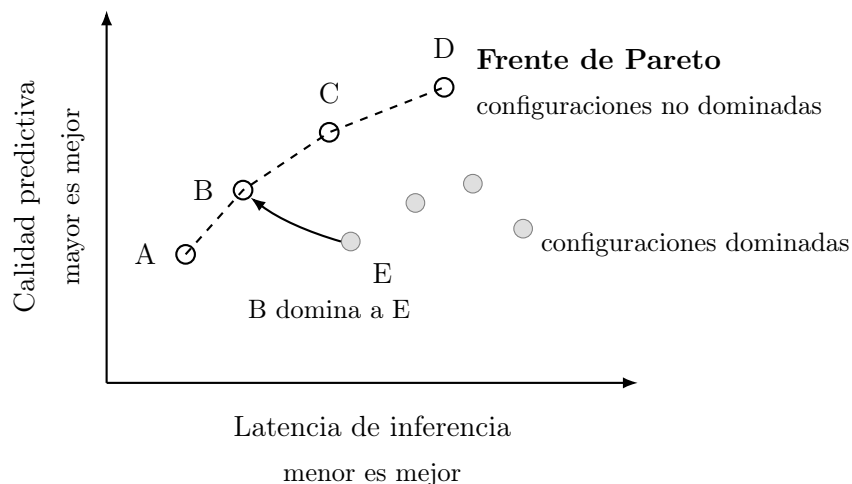


Figura 1.14: Representación conceptual del frente de Pareto para dos objetivos en conflicto: maximizar la calidad predictiva y minimizar la latencia de inferencia. Cada punto representa una configuración de hiperparámetros. Las configuraciones A–D no son dominadas, mientras que E es dominada por B al presentar simultáneamente mayor latencia y menor calidad predictiva. Fuente: elaboración propia con base en Deb et al. [17].

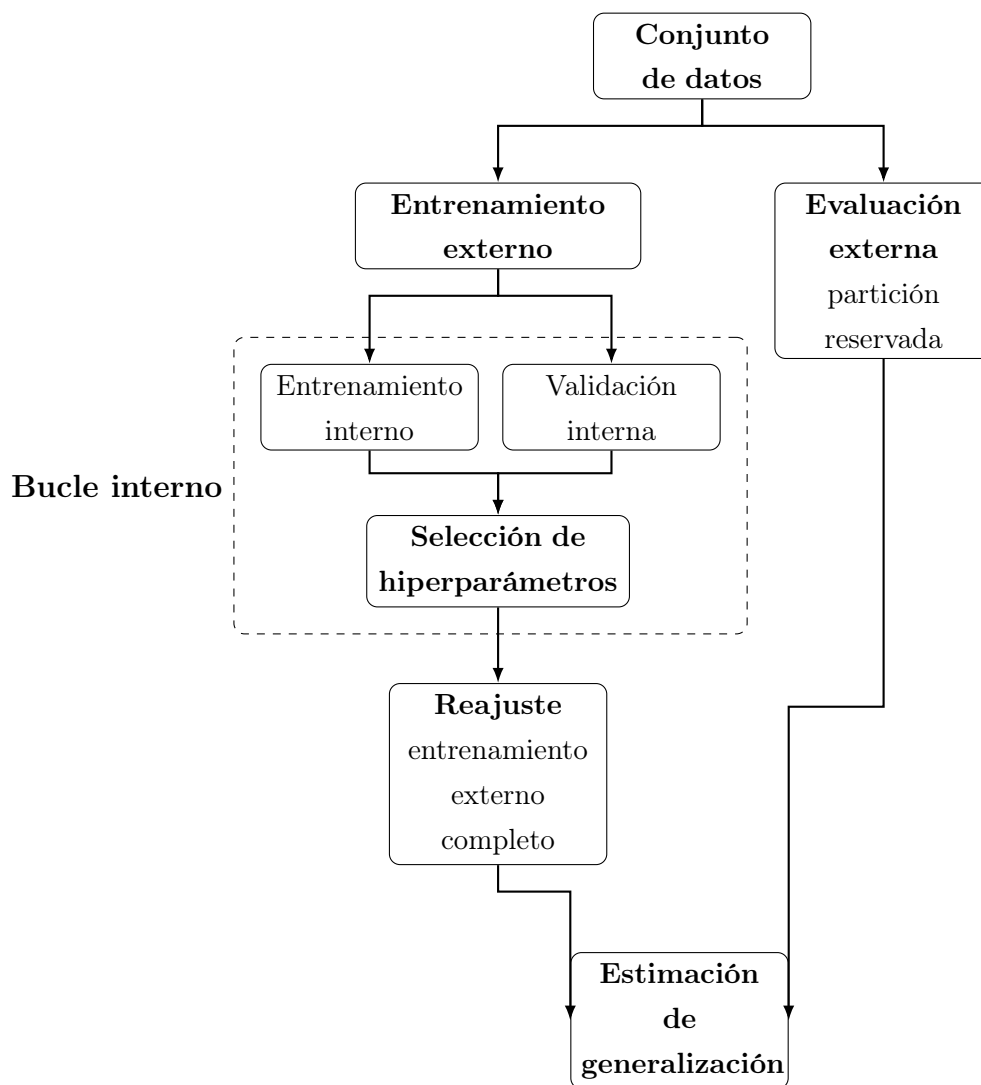
El frente delimita las opciones defendibles, pero no designa una entre ellas: seleccionar un único modelo requiere un criterio adicional, ya sea la combinación de los objetivos en una función escalar con pesos declarados explícitamente, o la imposición de una restricción sobre uno de ellos, por ejemplo un límite superior de latencia admisible por el lazo de control, para optimizar el restante dentro de esa frontera. El mismo razonamiento no se limita a las configuraciones de un modelo: permite también comparar familias distintas entre sí, al decidir si la ganancia predictiva de un ensamble más costoso justifica su mayor latencia frente a una alternativa más simple.

1.1.11. Validación anidada y fuga de información

La optimización de hiperparámetros introduce una etapa de selección adicional al entrenamiento del modelo. Si las configuraciones se seleccionan utilizando la misma información

sobre la que posteriormente se estima su desempeño, la evaluación puede presentar un sesgo optimista, ya que el proceso de selección se adapta indirectamente a esos datos [11].

La validación cruzada anidada separa ambos procesos mediante dos niveles. El bucle externo reserva observaciones para estimar la capacidad de generalización, mientras que el bucle interno utiliza únicamente los datos de entrenamiento correspondientes para seleccionar los hiperparámetros. Una vez elegida la configuración, el modelo se reajusta con los datos disponibles en el entrenamiento externo y se evalúa sobre la partición externa que no participó en su selección [11]. Esta separación se representa conceptualmente en la Figura 1.15.



El procedimiento se repite para cada partición externa

Figura 1.15: Esquema conceptual de la validación cruzada anidada. La selección de hiperparámetros se realiza únicamente dentro del conjunto de entrenamiento externo, mientras que la partición externa reservada se utiliza exclusivamente para estimar la capacidad de generalización. Fuente: elaboración propia con base en Cawley y Talbot [11].

La fuga de información constituye un problema diferente. Ocurre cuando las características utilizadas por el modelo contienen información sobre la variable objetivo que no estaría legítimamente disponible en el momento de realizar una predicción [34]. Por ejemplo, si una etiqueta se construye a partir de una determinada magnitud, emplear esa misma magnitud o una variable que la revele directamente como característica de entrada permitiría al clasificador acceder de forma indirecta al criterio que pretende inferir.

La independencia entre las particiones también debe considerar la estructura de los da-

tos. Cuando varias observaciones proceden de una misma unidad o de grupos estrechamente relacionados, distribuirlos entre entrenamiento y evaluación puede producir estimaciones excesivamente optimistas. En estos casos, la partición debe realizarse a nivel de grupo, de modo que las observaciones relacionadas permanezcan juntas y la evaluación refleje la generalización hacia grupos no observados [50]. Esta restricción debe preservarse también dentro del proceso de selección de hiperparámetros cuando se emplea validación anidada.

1.1.12. Comparaciones estadísticas pareadas

Cuando dos condiciones se evalúan sobre las mismas unidades experimentales, las observaciones correspondientes forman pares y no deben tratarse como muestras independientes. El análisis se realiza entonces sobre las diferencias observadas para cada unidad, permitiendo evaluar si existe un efecto sistemático asociado a la condición comparada [46, Ch. 2].

La prueba t pareada contrasta si la media de estas diferencias es estadísticamente distinguible de cero y supone una distribución aproximadamente normal de las diferencias. Cuando este supuesto no resulta adecuado, la prueba de rangos con signo de Wilcoxon constituye una alternativa no paramétrica que utiliza el signo y el orden de las diferencias en lugar de depender directamente de sus magnitudes [26, Ch. 3]. El valor p obtenido mediante una prueba de hipótesis permite cuantificar la evidencia contra la ausencia de una diferencia sistemática, pero no expresa por sí mismo la magnitud del efecto observado.

El remuestreo *bootstrap* permite complementar este análisis mediante la estimación de intervalos de confianza para el estadístico de interés. A partir de remuestreos con reemplazo de las observaciones disponibles se obtiene una distribución empírica que permite caracterizar la incertidumbre asociada al efecto estimado sin imponer una distribución paramétrica específica [18, Chs. 12–14].

La capacidad para detectar una diferencia depende del número de observaciones y de su variabilidad. Por esta razón, un resultado no significativo indica falta de evidencia suficiente para rechazar la hipótesis nula, pero no demuestra que las condiciones comparadas sean equivalentes.

1.1.13. Producto Energía–Retardo (EDP)

La evaluación de una estrategia de gestión energética debe considerar conjuntamente el consumo de energía y el tiempo de ejecución, ya que una reducción de potencia puede acompañarse de un incremento temporal suficiente para anular el beneficio energético. El Producto Energía–Retardo (*Energy–Delay Product*, EDP) combina ambas magnitudes en

una única métrica [36]:

$$EDP = E \cdot T, \quad (1.11)$$

donde E corresponde a la energía consumida y T al tiempo de ejecución. Un menor valor de EDP representa un compromiso más favorable entre ambas magnitudes bajo este criterio.

La formulación admite una generalización que pondera explícitamente el peso relativo del tiempo frente a la energía:

$$ED^nP = E \cdot T^n, \quad (1.12)$$

donde n expresa cuánto se penaliza la degradación temporal. Con $n = 1$ se recupera el EDP, mientras que $n = 2$ define el ED^2P , empleado cuando el rendimiento debe preservarse con mayor prioridad que el consumo [36, 1]. La elección de n es una decisión del evaluador y no una propiedad del sistema medido: dos valores distintos pueden ordenar de forma diferente las mismas configuraciones, por lo que debe declararse explícitamente antes de comparar resultados.

El EDP resulta especialmente útil en la evaluación de técnicas DVFS, donde la reducción de frecuencia puede disminuir la potencia instantánea pero aumentar el tiempo de ejecución. Por ello, evaluar únicamente potencia, energía o rendimiento de forma aislada puede conducir a conclusiones distintas de las obtenidas al considerar conjuntamente el costo energético y temporal [1, 36].

1.1.14. Síntesis: alcance y límites del marco adoptado

Los conceptos anteriores convergen en un compromiso metodológico concreto. La etiqueta de régimen procede del modelo Roofline y exige una medida directa de trabajo aritmético y de tráfico hacia memoria principal, disponible fuera de línea pero costosa o inaccesible durante la ejecución. Las señales que sí pueden leerse con baja intrusión describen actividad y no intensidad operacional, de modo que el clasificador se define precisamente como el sustituto que traduce las segundas en la primera. Esta separación entre la fuente de la verdad y la fuente de la predicción es la decisión de diseño central del trabajo, y de ella se desprenden las salvaguardas desarrolladas en las secciones previas: la exclusión de la magnitud que origina la etiqueta del vector de entrada, la partición por unidad experimental y no por observación individual, y la comparación contra líneas base sin capacidad predictiva.

Este marco delimita también lo que el trabajo no puede afirmar. La clasificación es binaria y estática respecto a un umbral calibrado, por lo que no describe fases intermedias ni transiciones continuas entre regímenes. La observabilidad disponible es asimétrica entre dispositivos, de manera que la misma pregunta no admite el mismo grado de evidencia en CPU y en GPU. Y el mecanismo de actuación tiene un costo temporal propio que acota la granularidad útil de cualquier política, con independencia de la calidad del clasificador. En consecuencia, un resultado que no muestre mejora sobre las líneas base o sobre los gobernadores nativos no invalidaría el marco: sería la respuesta que este marco permite dar sobre los límites de la telemetría y del mecanismo de control disponibles en la plataforma evaluada.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. Dentro de este proceso, el escalado dinámico de voltaje y frecuencia (DVFS) se ha consolidado como uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación, sin embargo, no conduce necesariamente al mismo resultado para todas las cargas de trabajo. La reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético esperado. Por esta razón, una parte importante de la investigación en este campo se ha desplazado desde la aplicación uniforme de DVFS hacia mecanismos capaces de reconocer el comportamiento de la carga y seleccionar configuraciones acordes con sus características.

Una de las primeras evidencias de esta dependencia fue presentada por Calore et al. [10], quienes estudiaron el efecto de DVFS sobre una CPU Intel Haswell y una GPU NVIDIA K80 mediante distintos kernels y una aplicación HPC completa. Los autores analizaron conjuntamente el tiempo y la energía de ejecución, y mostraron que una misma frecuencia no resulta igualmente conveniente para todas las regiones de una aplicación. En lugar de mantener un único estado operativo durante toda la ejecución, evaluaron cambios de frecuencia a nivel de función, lo que permitió observar que las oportunidades de ahorro dependen tanto de la naturaleza de la región ejecutada como de la arquitectura sobre la cual se realiza el cálculo. Este resultado introduce una idea central para los mecanismos de gestión adaptativa: la decisión de frecuencia debe responder al comportamiento de la carga y no únicamente a una política global fijada de antemano.

Aunque esta aproximación permite aprovechar diferencias entre regiones de ejecución,

requiere conocimiento previo sobre la estructura de la aplicación y puntos específicos en los cuales realizar la actuación. El problema se desplaza entonces hacia la posibilidad de caracterizar automáticamente una carga a partir de información observable durante su ejecución. En este sentido, Guerreiro et al. [24] estudiaron la relación entre el comportamiento de aplicaciones GPGPU y las variaciones de frecuencia de los dominios de cómputo y memoria. Su propuesta parte de métricas recolectadas en una configuración inicial y utiliza esa caracterización para anticipar la respuesta de la aplicación frente a otras configuraciones DVFS. De esta manera, los autores reducen la necesidad de recorrer exhaustivamente el espacio de frecuencias y muestran que es posible utilizar información de rendimiento para agrupar aplicaciones según su sensibilidad a DVFS y seleccionar configuraciones energéticamente favorables.

La propuesta de Guerreiro et al. [24] resulta particularmente significativa porque introduce la clasificación como un mecanismo intermedio entre la observación de la carga y la decisión energética. No obstante, las clases definidas en su trabajo responden principalmente a la sensibilidad de las aplicaciones frente a modificaciones en las frecuencias de cómputo y memoria, y no a una detección continua de fases *compute-bound* y *memory-bound* durante la ejecución. Su objetivo consiste en utilizar una caracterización limitada para inferir el comportamiento esperado bajo distintas configuraciones, manteniendo además el análisis concentrado en GPU. Aun así, el estudio demuestra que las métricas obtenidas del hardware contienen información suficiente para reducir el espacio de búsqueda de una configuración energética adecuada.

La misma problemática de evitar una exploración exhaustiva fue abordada posteriormente por Ali et al. [1]. Los autores propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas obtenidas inicialmente en la configuración predeterminada del dispositivo. Sobre esta caracterización construyen modelos de potencia y tiempo de ejecución, con los cuales estiman la respuesta de la aplicación en otros estados DVFS sin necesidad de ejecutarla en cada uno de ellos. La selección final se realiza considerando simultáneamente consumo energético y degradación del desempeño, de modo que una reducción de potencia no sea interpretada automáticamente como una mejora si viene acompañada de un incremento excesivo en el tiempo de ejecución. En las plataformas evaluadas, la propuesta alcanzó reducciones promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 % sobre NVIDIA GA100 y de 22.6 % con una pérdida de 4.7 % sobre NVIDIA GV100 [1].

En conjunto, los trabajos de Guerreiro et al. [24] y Ali et al. [1] muestran que el espacio DVFS puede reducirse mediante modelos contruidos a partir del comportamiento de la aplicación. Sin embargo, ambos enfoques parten principalmente de una caracterización utilizada

para predecir configuraciones posteriores. Esta estrategia es adecuada cuando la aplicación puede tratarse como una carga relativamente estable, pero resulta más limitada cuando su comportamiento cambia durante la ejecución. Las aplicaciones científicas suelen combinar regiones con demandas diferentes sobre las unidades de cómputo y el subsistema de memoria; por tanto, una política verdaderamente adaptativa requiere no solo estimar qué frecuencia puede resultar adecuada, sino también reconocer cuándo cambia el régimen de ejecución.

La clasificación de este comportamiento constituye precisamente el problema abordado por Antici et al. [3] mediante MCBound. Los autores plantean la distinción entre cargas *memory-bound* y *compute-bound* como un problema de clasificación basado en datos, utilizando información de ejecuciones HPC para caracterizar automáticamente nuevos trabajos. Su evaluación sobre datos del supercomputador Fugaku mostró que esta separación puede realizarse con una puntuación F1-macro de al menos 0.89 y con un costo reducido para el funcionamiento del sistema [3]. Este resultado aporta evidencia directa de que la naturaleza computacional de una carga puede inferirse automáticamente en lugar de depender exclusivamente de una clasificación manual basada en el conocimiento previo de la aplicación.

MCBound representa un antecedente especialmente cercano al componente de clasificación, aunque su propósito no es gobernar directamente los estados DVFS de CPU o GPU. La clasificación se utiliza para caracterizar trabajos HPC, pero no se integra con un mecanismo que traduzca de manera continua la clase inferida en una decisión de frecuencia durante las distintas fases de la ejecución. Esta separación resulta importante: demostrar que una carga puede clasificarse como limitada por memoria o por cómputo resuelve el problema de identificación, pero aún queda por determinar cómo utilizar esa información dentro de una política energética y si el costo de realizar dicha inferencia es suficientemente bajo para justificar su uso en tiempo de ejecución.

La actuación dinámica sobre la frecuencia dentro de una aplicación científica ha sido explorada por Simsek et al. [51] en el contexto de simulaciones astrofísicas ejecutadas sobre GPU. Los autores instrumentaron SPH-EXA para medir el comportamiento energético de diferentes regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente a una política de frecuencia uniforme, esta adaptación permitió reducir el consumo energético por GPU hasta en 7.82 %, con una degradación del rendimiento de aproximadamente 2.95 %. El estudio demuestra que existe un beneficio potencial al cambiar la frecuencia durante la ejecución y no únicamente antes de iniciar la aplicación. No obstante, la decisión depende de la instrumentación del código y del conocimiento de las regiones sobre las cuales se debe actuar, por lo que la política permanece vinculada a la estructura particular de la aplicación [51].

Un antecedente adicional, más cercano a un control *online* completo, es DRLCAP, un marco de *frequency capping* de GPU en tiempo de ejecución basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema para detectar cambios de fase y ajustar el límite de frecuencia sin intervención manual, reduciendo en promedio 22 % de la energía de GPU con menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [56]. Este resultado confirma que el control *online* de frecuencia es factible y efectivo, pero también expone una diferencia de diseño relevante para este trabajo: la política se aprende mediante refuerzo profundo, orientada exclusivamente a GPU, en lugar de formularse como una clasificación supervisada ligera de fases seguida de una decisión discreta e interpretable, coordinada sobre CPU y GPU.

Los antecedentes revisados permiten observar una evolución clara del problema. Calore et al. [10] mostraron que diferentes regiones de una aplicación pueden requerir configuraciones DVFS distintas; Guerreiro et al. [24] demostraron que la sensibilidad a la frecuencia puede caracterizarse a partir de métricas de ejecución; Ali et al. [1] utilizaron esta idea para estimar configuraciones energéticamente convenientes sin recorrer todo el espacio DVFS; Antici et al. [3] evidenciaron que la distinción entre cargas *compute-bound* y *memory-bound* puede plantearse como un problema de clasificación; Simsek et al. [51] comprobaron que modificar dinámicamente la frecuencia dentro de una aplicación científica puede producir ahorros energéticos medibles; y Wang et al. [56] demostraron con DRLCAP que un controlador *online* aprendido puede gobernar la frecuencia de GPU en tiempo real con resultados energéticos favorables. En conjunto, estos trabajos cubren las principales etapas necesarias para construir un mecanismo de gestión adaptativa, aunque dichas etapas aparecen generalmente resueltas de manera independiente.

A partir de esta revisión, el problema de investigación no radica en demostrar nuevamente que DVFS puede disminuir el consumo energético ni en establecer que las cargas pueden clasificarse de acuerdo con su comportamiento. La cuestión que permanece abierta para el presente trabajo es cómo integrar ambas capacidades dentro de un mecanismo de control de baja intrusión que pueda operar durante la ejecución. En particular, interesa determinar si la telemetría disponible desde espacio de usuario permite alimentar un clasificador supervisado ligero que identifique el régimen predominante de la carga y utilice esa información como señal para aplicar una política DVFS previamente caracterizada sobre el hardware.

Finalmente, la revisión evidencia una concentración importante de los enfoques recientes de predicción y selección de frecuencia en GPU. Aunque Calore et al. [10] consideran conjuntamente CPU y GPU, los trabajos de Guerreiro et al. [24], Ali et al. [1], Simsek et al. [51] y Wang et al. [56] profundizan principalmente en aceleradores gráficos. Esta diferencia

es relevante para un entorno heterogéneo, ya que CPU y GPU no ofrecen las mismas fuentes de telemetría, poseen mecanismos distintos de gestión de frecuencia y pueden responder de manera diferente ante una reducción del reloj. Por esta razón, una estrategia de control para sistemas CPU–GPU debe reconocer dichas diferencias en lugar de aplicar una única regla de observación y actuación a ambos dispositivos. El propósito del presente trabajo se sitúa precisamente en esta integración: utilizar mecanismos de clasificación de bajo costo como señal de contexto para decisiones DVFS y evaluar empíricamente si dicha estrategia mejora el compromiso entre energía y rendimiento frente a los mecanismos nativos de gestión de frecuencia.

Capítulo 2

Metodología

2.1. Enfoque metodológico

La investigación adopta un enfoque cuantitativo, experimental y aplicado para evaluar el efecto de las decisiones de frecuencia del agente sobre la eficiencia energética de las cargas de trabajo. La variable de respuesta principal es el Producto Energía–Retardo (EDP), que integra la energía consumida y el tiempo de ejecución en una sola medida; no obstante, ambas magnitudes se conservan por separado para interpretar el origen de las variaciones observadas.

El estado de frecuencia constituye la variable independiente y se modifica sobre un conjunto controlado de cargas, mientras sus efectos se observan en el tiempo de ejecución, la energía y el EDP; a su vez, el carácter aplicado de la investigación se concreta en un agente de software cuya utilidad se compara con las políticas de frecuencia disponibles en el sistema operativo.

El desarrollo se estructura en cuatro fases secuenciales e interdependientes, que abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema.

2.2. Diseño metodológico

2.2.1. Población y muestra

La población de estudio comprende las aplicaciones ejecutables en sistemas heterogéneos CPU–GPU; de esta población se selecciona una muestra no probabilística e intencional

de benchmarks y microbenchmarks con regímenes de ejecución contrastantes. La selección proporciona cargas controladas y reproducibles para construir el conjunto de datos de entrenamiento, sin pretender representar exhaustivamente todas las aplicaciones científicas.

Las cargas se admiten en el catálogo cuando verifican la corrección de su resultado, permiten observar directamente su trabajo de punto flotante y producen una magnitud comparable con un techo Roofline expresado en FLOPs. En CPU, el trabajo se mide mediante los contadores de instrucciones de punto flotante retiradas descritos en la sección 2.3.11, mientras que en GPU se obtiene mediante el perfilado presentado en la sección 2.3.5; cuando una carga reporta su propio conteo de operaciones, este se utiliza solo como contraste independiente. De acuerdo con la regla de validez de la sección 2.3.2, se excluyen las cargas dominadas por aritmética entera, ya que su intensidad operacional no es comparable con el techo empleado.

2.2.2. Plataforma experimental

Los experimentos se ejecutan sobre el nodo heterogéneo descrito en la Tabla 2.1, seleccionado porque integra una GPU NVIDIA observable mediante NVML y permite acceder, sin privilegios administrativos, a la medición energética interna del procesador; ambas capacidades son necesarias para obtener telemetría de CPU y GPU bajo las mismas condiciones experimentales.

La microarquitectura determina la codificación de varios eventos crudos de la PMU que no disponen de un mapeo genérico en el kernel del nodo (sección 1.1.3); por esta razón, antes de habilitarlos, el instrumento comprueba que la familia y el modelo del procesador correspondan con los valores de la Tabla 2.1. Si la identificación no coincide, la codificación se rechaza para evitar una interpretación incorrecta de los conteos del hardware; esta protección se aplica, entre otros, a los subeventos de `FP_ARITH_INST_RETIRED` utilizados para medir FLOPs (sección 2.3.11).

Tabla 2.1: Características de la plataforma experimental.

Componente		Descripción
Procesador		2 × Intel Xeon Gold 5315Y
Microarquitectura		Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)
Identificador	CPUID	6 / 106
(cpu family/model)		
Extensiones SIMD	rele-	AVX2, FMA3, AVX-512 (avx512f, avx512bw, avx512cd, avx512dq, avx512vl, avx512_vnni, entre otras) — el ancho de 512 bits es el que domina el conteo de FLOPs medido en las cargas de álgebra lineal densa del catálogo
vantes		
Núcleos		8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)
Nodos NUMA		2 (uno por zócalo)
Jerarquía de caché		L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB por zócalo
Tamaño de línea de caché		64 bytes
Controlador de frecuencia		intel_pstate
Rango de frecuencia		0.8–3.6 GHz
Dominio de frecuencia		Por núcleo lógico
Medición energética		Intel RAPL (dominios de paquete y DRAM por zócalo)
Acelerador		NVIDIA A100 PCIe 40 GB
Gestor de recursos		Slurm

Las condiciones de ejecución se controlan mediante asignación exclusiva del nodo a través del gestor de recursos, de modo que ninguna carga ajena al experimento comparta el procesador durante las mediciones. Los procesadores lógicos empleados se restringen a hilos primarios de un mismo zócalo, evitando la asignación simultánea de hilos hermanos del mismo núcleo físico, de acuerdo con lo expuesto en la sección 1.1.6.

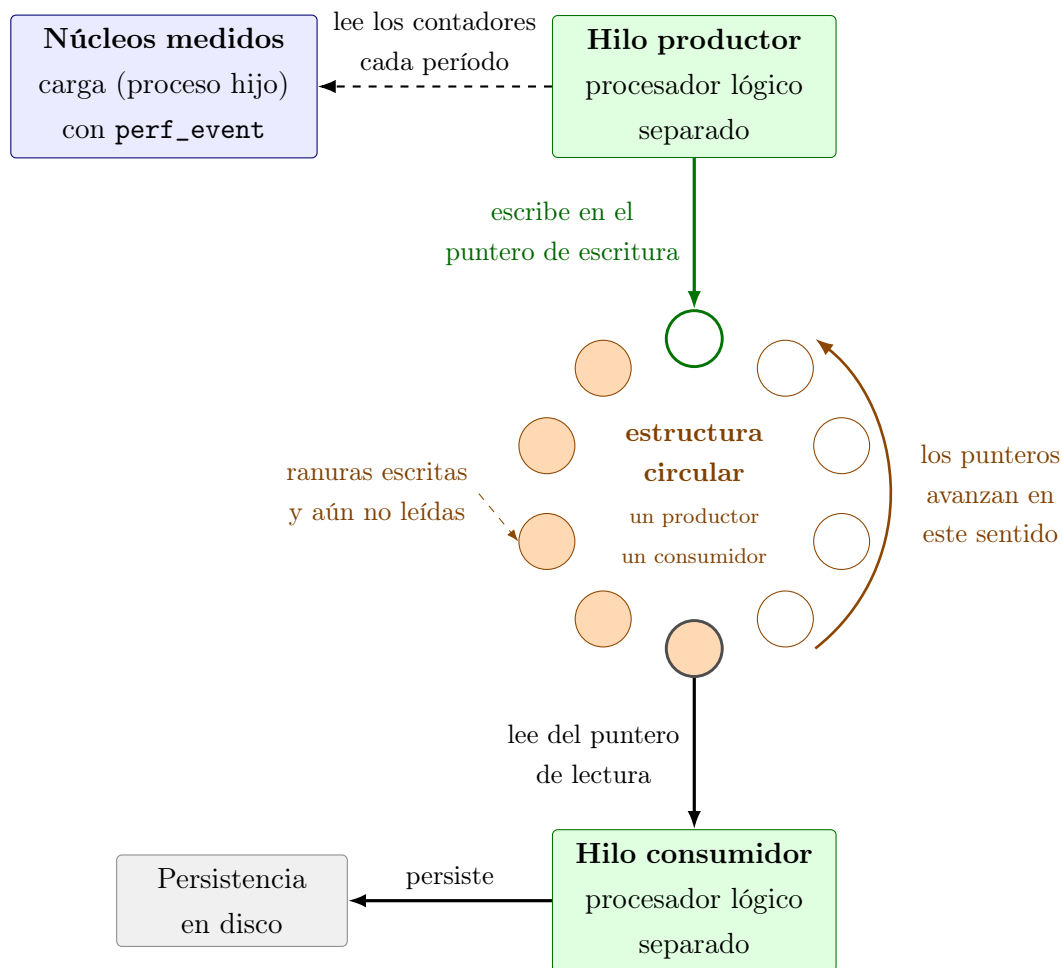
2.2.3. Instrumentos de recolección

La recolección de telemetría se implementa mediante un instrumento propio, desarrollado específicamente para este trabajo, compuesto por dos capas con responsabilidades separadas.

La capa inferior es un ejecutor escrito en C++ que lanza la carga de trabajo como proceso hijo y adjunta los contadores de hardware sobre ella. El diseño de esta capa responde

directamente a los requisitos de atribución expuestos en la sección 1.1.5: el proceso hijo se detiene inmediatamente después de su creación y antes de ejecutar la primera instrucción de la carga, momento en el cual el proceso padre abre los contadores asociados a ese identificador de proceso con herencia habilitada; solo entonces se reanuda la ejecución. Este protocolo garantiza que ninguna instrucción de la carga medida se ejecute fuera del intervalo de observación y que la medida abarque el árbol completo de procesos e hilos que la carga genere, sin incluir actividad ajena.

El muestreo se realiza mediante un hilo productor dedicado, fijado a un procesador lógico distinto de los asignados a la carga, que a intervalos regulares lee los contadores y deposita las muestras en una estructura circular de productor y consumidor únicos. Un hilo consumidor, igualmente fijado a un procesador lógico separado, extrae las muestras y las persiste. Esta separación evita que la actividad de escritura a disco interfiera con la periodicidad del muestreo, y el aislamiento de ambos hilos respecto de los núcleos medidos reduce su interferencia sobre la carga observada. La ausencia de pérdidas en la estructura circular se registra como indicador de integridad de cada corrida. La Figura 2.1 resume esta ruta.



si el puntero de escritura alcanza al de lectura la muestra se pierde;
ese conteo de pérdidas es el indicador de integridad de la corrida

Figura 2.1: Ruta de una muestra de telemetría, desde la lectura de los contadores hasta su persistencia. El hilo productor escribe cada muestra en la ranura que señala el puntero de escritura (borde verde) y el hilo consumidor la retira por el puntero de lectura (borde negro); ambos punteros avanzan en el mismo sentido, de modo que las ranuras naranjas son las escritas que el consumidor aún no ha leído. Los dos hilos se fijan a procesadores lógicos distintos de los que ejecutan la carga, y la estructura circular desacopla la cadencia de muestreo de la escritura a disco. Fuente: elaboración propia.

La capa superior es un orquestador escrito en Python que administra la lógica experimental mediante la interpretación del manifiesto de campaña, la detección de capacidades, la ejecución de verificaciones previas, la calibración y la planificación de cada combinación; además, aplica el estado de frecuencia, valida las corridas y transforma las muestras crudas en el conjunto de datos final. Esta distribución de responsabilidades reserva a la capa C++

la medición sensible a la latencia y asigna a la capa Python la planificación y la trazabilidad del experimento, cuyo recorrido se resume en la Figura 2.2.

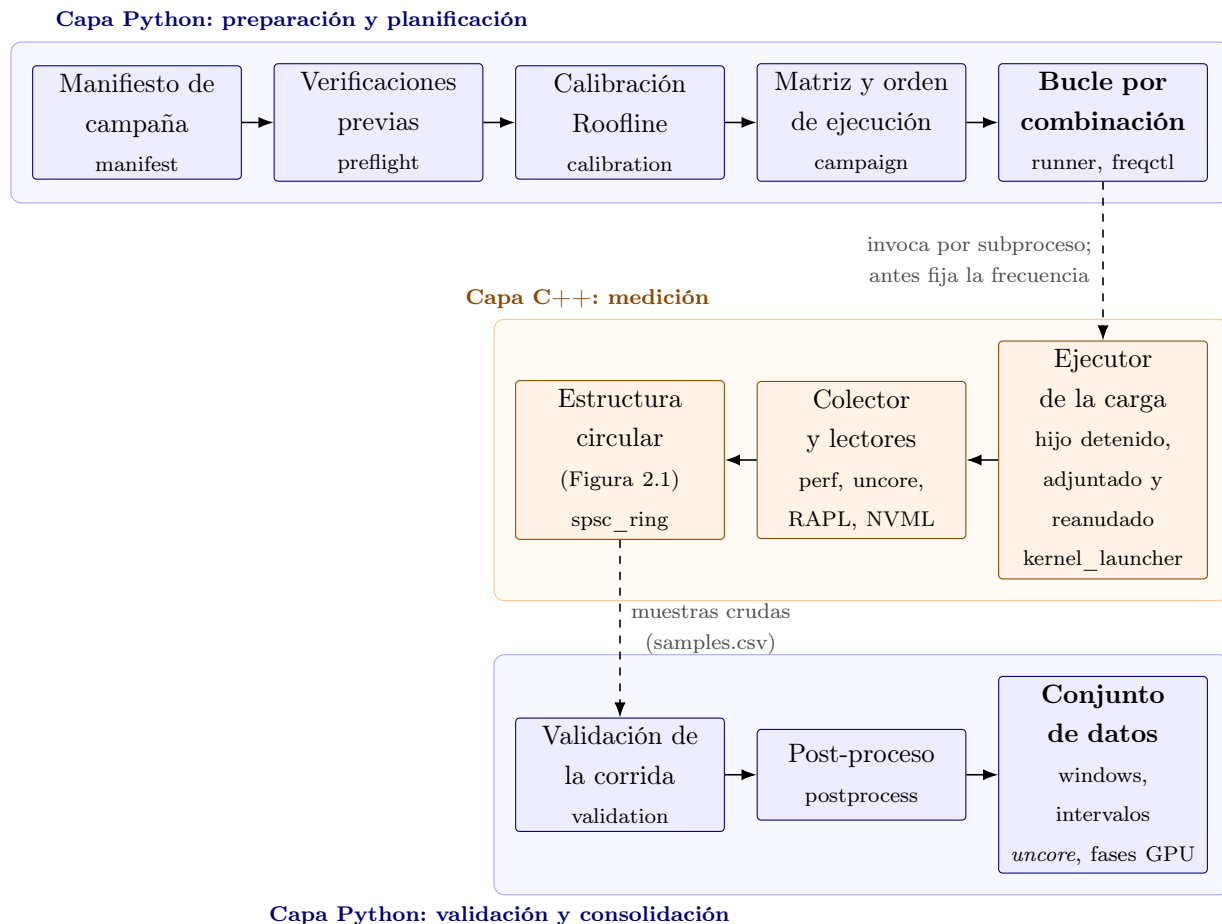


Figura 2.2: Recorrido de una campaña a través de las dos capas del instrumento. La capa Python prepara la campaña (banda superior), invoca por subprocesso el ejecutor de la capa C++ una vez por combinación de carga y frecuencia (banda intermedia), y consolida las muestras crudas que este produce en el conjunto de datos (banda inferior). La capa C++ concentra la medición sensible a la latencia; la capa Python, la lógica experimental, donde priman la claridad y la trazabilidad. Fuente: elaboración propia.

2.2.4. Variables observadas

Antes de describir las señales capturadas, conviene precisar qué se entiende por ventana de muestreo, término empleado de aquí en adelante. El hilo productor descrito en la sección anterior toma una lectura de los contadores cada cierto período configurado en la campaña; cada lectura individual es un valor acumulativo, y por sí sola no describe el comportamiento de un intervalo concreto. Una ventana de muestreo se define como el intervalo delimitado por dos lecturas consecutivas, y su contenido se obtiene restando la lectura anterior de la

lectura actual, conforme a lo argumentado en la sección 1.1.3. Por tanto, una ventana no es un conjunto o agregado de varias muestras, sino el resultado de un único par de lecturas consecutivas; se requieren dos lecturas para producir una ventana, de modo que la primera lectura de cada corrida no genera ventana alguna, al carecer de una lectura previa contra la cual diferenciarse (condición registrada en la Tabla 2.6 como “sin diferencia previa”). La duración real de una ventana corresponde al tiempo efectivamente transcurrido entre ambas lecturas, medido con un reloj monótono, y no al período nominal configurado.

La Tabla 2.2 enumera las señales activas del instrumento de producción y separa los contadores de núcleo, atribuibles a la carga, de los del controlador de memoria, que son de alcance de zócalo. El conjunto de eventos se dimensionó para permanecer dentro del presupuesto de contadores físicos simultáneos de la plataforma, verificado empíricamente sobre la máquina en lugar de asumirse a partir del modelo de procesador, con el fin de evitar el error de extrapolación asociado a la multiplexación descrito en la sección 1.1.3.

Tabla 2.2: Señales de telemetría de CPU activas en el instrumento de producción.

Señal exportada	Evento e interpretación
Contadores de núcleo (<i>atribuibles al proceso medido</i>)	
<code>instructions</code>	PERF_COUNT_HW_INSTRUCTIONS. Base de IPC e instrucciones por segundo.
<code>cycles</code>	PERF_COUNT_HW_CPU_CYCLES. Denominador de IPC y de las fracciones de <i>stall</i> .
<code>cache_references,</code> <code>cache_misses</code>	Eventos genéricos de <code>perf</code> que producen <code>cache_miss_rate</code> y MPKI, los cuales indican presión de memoria, no bytes movidos a DRAM.
<code>stalled_cycles_</code> <code>mem_any</code>	Evento crudo STALLS_MEM_ANY (0xA3/0x14, cmask=0x14). Ciclos de ejecución bloqueados mientras hay una carga de memoria pendiente; se normaliza como <code>stall_mem_ratio</code> . Señal predictiva, no la etiqueta Roofline.
<code>fp_*_double</code>	Subeventos de doble precisión de FP_ARITH_INST_RETIRED (0xC7). Numerador de FLOPs, ponderados por 1, 2, 4 u 8 elementos.
<code>time_enabled_ns,</code> <code>time_running_ns</code>	Metadatos de <code>perf</code> . Diagnostican la multiplexación y escalan los conteos; las ventanas degradadas se excluyen del entrenamiento.
Ámbito de zócalo o de sistema (<i>no atribuibles a un proceso</i>)	
Comandos CAS del IMC: <code>read/write_interval</code>	Número de comandos <i>Column Address Strobe</i> (CAS) de lectura y de escritura emitidos por el controlador de memoria integrado (IMC), contabilizados por su PMU de <i>uncore</i> . Cada comando transfiere una línea de 64 bytes hacia o desde la DRAM, de modo que la suma de ambos es el denominador DRAM del modelo Roofline. Su intervalo práctico (≥ 10 ms) es más grueso que la ventana de CPU, y su etiqueta se difunde a las ventanas que cubre.
RAPL: <code>pkg_uj,</code> <code>dram_uj</code>	Energía acumulada por zócalo. Sus diferencias, con corrección de desbordamiento, producen energía y potencia; no intervienen en la etiqueta <i>compute/memory-bound</i> .
<code>scaling_cur_freq</code> por CPU delegado	Lectura de <i>sysfs</i> por núcleo lógico. Verifica la frecuencia efectiva bajo carga y detecta divergencias entre los núcleos delegados (sección 1.1.6).

A partir de estas señales se derivan, para cada ventana, IPC, tasa de fallos de caché, MPKI, `stall_mem_ratio`, intensidad operacional y magnitudes energéticas. Las tasas de núcleo se calculan sobre el intervalo real entre lecturas consecutivas; la intensidad y la etiqueta Roofline conservan la granularidad del intervalo *uncore*, conforme a lo argumentado en la sección 1.1.3.

2.3. Fase 1: Recolección de información y caracterización de cargas

La primera fase tuvo como objetivo construir el conjunto de datos etiquetado empleado para entrenar el clasificador, junto con el instrumento y el protocolo necesarios para asegurar la validez de las mediciones. A continuación se presenta el procedimiento aplicado.

2.3.1. Verificación previa de la plataforma

Antes de cada campaña se ejecutaron verificaciones automáticas para confirmar que el nodo cumpliera las condiciones del diseño experimental. Se comprobaron las capacidades de control y medición de frecuencia, la afinidad NUMA y la configuración de multihilo, la disponibilidad de contadores y dominios energéticos, la integridad y los permisos de los binarios, la memoria y el almacenamiento disponibles y, cuando correspondía, los permisos de escritura sobre los dominios de frecuencia asignados.

Las comprobaciones se clasificaron en obligatorias e informativas; el incumplimiento de una condición obligatoria detuvo la campaña antes de modificar el sistema o producir datos. Este control formó parte del mismo comando de ejecución, por lo que ninguna medición pudo iniciarse sin completar previamente la verificación del entorno.

2.3.2. Catálogo de cargas de trabajo

El catálogo se organizó en cargas de calibración y cargas de caracterización. Para estimar los techos del modelo Roofline se utilizaron un microbenchmark de ancho de banda, cuyo conjunto de trabajo excedió la caché de último nivel, y otro de alta densidad aritmética; con ellos se obtuvieron BW_{pico} y P_{pico} , respectivamente. Las cargas de caracterización procedieron de suites de uso extendido y se emplearon sin modificar su código fuente. Cada binario se compiló en la plataforma experimental y se identificó mediante una suma de verificación, comprobada antes de cada corrida para evitar que una recompilación alterara las condiciones de la campaña. La Tabla 2.3 presenta las cargas y su procedencia.

Tabla 2.3: Catálogo de cargas de trabajo.

Carga	Suite u origen	Papel y régimen esperado
<i>Calibración de los techos Roofline</i>		
STREAM	McCalpin [44]	BW_{pico} de CPU.
ERT	Lo et al. [43]	P_{pico} de CPU.
Microbenchmarks propios (BW, FP32, FP64)	Elaboración propia	Techos de ancho de banda y de cómputo del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia independiente del techo de cómputo del acelerador.
<i>Conjunto de datos — CPU</i>		
BT, MG, CG, SP, FT, LU	NAS Parallel Benchmarks [4]	Métodos implícitos, multigrid, gradiente conjugado y FFT; mayoría <i>memory-bound</i> .
DGEMM	Biblioteca BLAS optimizada [22]	Multiplicación densa de matrices; ancla <i>compute-bound</i> .
LavaMD (OpenMP)	Rodinia [12]	Dinámica molecular de N-cuerpos; régimen intermedio.
3MM (OpenMP)	RAJA Performance Suite [6]	Triple multiplicación densa, con verificación numérica propia; régimen intermedio.
<i>Conjunto de datos — acelerador</i>		
gaussian, backprop, heartwall, LUD, myocyte, DWT2D	Rodinia [12]	Eliminación gaussiana, retropropagación, seguimiento de rasgos, factorización LU, electrofisiología y transformada wavelet; mayoría <i>memory-bound</i> .
LavaMD	Rodinia [12]	Principal ancla <i>compute-bound</i> del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia <i>compute-bound</i> del acelerador.
<i>Candidatos de ampliación del catálogo (sección 2.3.3)</i>		
EP	NAS Parallel Benchmarks [4]	Ancla <i>compute-bound</i> pura en CPU.
kmeans, SRAD, NW, particlefilter	Rodinia [12]	Patrones de acceso irregular en CPU: agrupamiento iterativo, difusión anisotrópica, programación dinámica en diagonal y filtro de partículas.
kmeans, SRAD, NW, b+tree, CFD	Rodinia [12]	Los mismos patrones en el acelerador, más recorrido de árbol y volúmenes finitos; kmeans , SRAD y NW forman pares CPU/acelerador.

Las cargas de calibración y caracterización iniciales se utilizaron para construir y validar el instrumento; posteriormente, el catálogo se amplió con nuevas cargas y se definió la familia algorítmica como unidad de diversidad.

Una familia algorítmica reunió las variantes de tamaño que compartían el mismo patrón de cómputo y acceso a memoria. Esta agrupación evitó considerar como ejemplos independientes varias ejecuciones del mismo algoritmo y permitió evaluar la generalización sobre patrones computacionales distintos.

La admisión de familias al conjunto definitivo siguió tres reglas:

1. **Representación por clase y dispositivo.** Cada clase debía incluir varias familias distintas en CPU y GPU; las variantes de tamaño de una misma familia no aumentaron ese conteo.
2. **Separación entre controles y datos de entrenamiento.** Los microbenchmarks propios se reservaron para calibrar y verificar el instrumento, por lo que no se utilizaron para entrenar el clasificador.
3. **Validez de la medida de trabajo.** Solo se admitieron cargas cuyo trabajo pudiera expresarse de manera significativa mediante intensidad operacional en FLOPs por byte; las cargas de aritmética entera o con una semántica incompatible se excluyeron y se registró el motivo.

Las familias adicionales se eligieron para ampliar la diversidad de patrones de acceso a memoria y la representación de ambas clases. Antes de incorporarlas se exigieron un binario reproducible, una suma de verificación, una validación numérica y, para GPU, una intensidad operacional obtenida mediante el procedimiento de la sección 2.3.5.

2.3.3. Selección preliminar y definición del catálogo definitivo

Antes de ejecutar el barrido completo se realizó una evaluación preliminar con tres niveles de frecuencia por dispositivo y el número mínimo de repeticiones. Esta etapa permitió comprobar la calidad de las señales, descartar cargas no utilizables y estimar la cobertura de las clases, sin buscar configuraciones óptimas de energía. En GPU, la intensidad operacional se caracterizó previamente mediante el procedimiento de la sección 2.3.5.

Para cada familia se analizaron su posición respecto al punto de inflexión, la representación de las clases y las causas de rechazo por calidad. En GPU, donde la etiqueta se obtuvo fuera de línea y permaneció constante durante cada corrida, también se examinó si las señales NVML permitían diferenciar los regímenes. Con estos criterios se definió y versionó la

lista de familias admitidas en la campaña definitiva; cuando la cobertura fue insuficiente, se incorporaron nuevas familias sin reducir el criterio mínimo establecido.

2.3.4. Calibración y criterio de etiquetado

La calibración se ejecutó al inicio de cada campaña con la misma asignación de núcleos y la misma afinidad utilizadas en las corridas posteriores. A partir de los valores medidos se calculó el punto de inflexión de la ecuación (1.3) para cada dispositivo, precisión aritmética y nivel de frecuencia, de acuerdo con la sección 1.1.2. Los resultados se compararon con las referencias empíricas declaradas en el manifiesto; una diferencia superior a la tolerancia establecida detuvo la campaña para evitar el uso de una calibración obtenida bajo condiciones anómalas.

La etiqueta de cada unidad de observación se obtuvo al comparar su intensidad operacional con el punto de inflexión calibrado, sin asignación manual. La unidad de observación utilizada para este cálculo se define en la sección 2.3.10. El catálogo también conservó una expectativa cualitativa basada en la literatura, pero esta se empleó solo para detectar posibles inconsistencias del instrumento y nunca para sustituir o corregir la etiqueta medida.

La estabilidad de cada campaña se comprobó mediante repeticiones de una carga de referencia. Una dispersión relativa superior al umbral establecido se registró como advertencia, pues indicaba que las condiciones de medición no permitían sostener comparaciones precisas entre corridas.

2.3.5. Caracterización de intensidad operacional y calibración Roofline en GPU

NVML proporcionó durante cada corrida la potencia, la utilización, el reloj, la temperatura y la energía de la GPU, pero no los FLOPs ni los bytes requeridos para calcular la intensidad operacional. Por ello, cada carga se caracterizó previamente y fuera de línea con NVIDIA Nsight Compute [49], que permitió medir por lanzamiento el tráfico hacia la memoria del dispositivo y las operaciones de punto flotante ejecutadas. Para una precisión aritmética determinada, la intensidad operacional se calculó como

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_{\ell}^{\text{add}} + N_{\ell}^{\text{mul}} + 2 N_{\ell}^{\text{fma}})}{\sum_{\ell=1}^L B_{\ell}}, \quad (2.1)$$

donde ℓ recorre L lanzamientos consecutivos del kernel; N_ℓ^{add} , N_ℓ^{mul} y N_ℓ^{fma} son los conteos de operaciones de suma, multiplicación y multiplicación–suma fusionada de punto flotante que el perfilador reporta para el lanzamiento ℓ , y B_ℓ es el total de bytes transferidos hacia o desde la memoria del dispositivo en ese lanzamiento. Cada multiplicación–suma fusionada contribuye con dos operaciones, según la misma convención adoptada para CPU en la ecuación (2.2).

La caracterización se repitió con cantidades crecientes de trabajo y la intensidad solo se aceptó cuando el cambio relativo entre los dos puntos de mayor tamaño fue inferior al 1 % y el número de lanzamientos observado coincidió con el solicitado. Las cargas que no cumplieron este criterio se registraron como no convergentes. Asimismo, se midieron simultáneamente las operaciones de precisión simple y doble; las cargas con mezcla no trivial de precisiones o sin trabajo de punto flotante significativo se excluyeron, porque no podían compararse de forma válida con un único techo Roofline. Para asegurar la trazabilidad se conservaron las salidas del perfilador, los comandos y las versiones del controlador y de las herramientas.

La intensidad aceptada se registró como propiedad de la carga y se reutilizó durante el procesamiento de la campaña NVML. Esta medición se realizó una sola vez por configuración del problema, ya que el perfilado detallado altera el tiempo de ejecución y no era adecuado para la adquisición en línea. Cuando Nsight Compute requirió más contadores de los disponibles en una pasada, repitió el lanzamiento completo por grupos de métricas, por lo que no se aplicó el escalado temporal utilizado por la multiplexación de CPU.

Los techos de GPU se calibraron por separado para precisión simple y doble mediante microbenchmarks de ancho de banda y de multiplicación–suma fusionada sobre datos residentes en registros. Esta separación evitó comparar una carga con un techo de distinta precisión o con rutas especializadas que no representaran la aritmética general del catálogo.

2.3.6. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Los parámetros de las cargas GPU se ajustaron para obtener una duración suficiente sin alterar innecesariamente la intensidad operacional medida. Cuando el parámetro solo repetía el mismo patrón de cómputo, se aumentó el número de repeticiones sin volver a caracterizar la carga; cuando modificaba el tamaño del problema, se repitió el procedimiento de la sección 2.3.5, pues el nuevo conjunto de datos podía cambiar la relación entre cómputo y tráfico de memoria.

Si la implementación no permitía repetir el mismo trabajo y el tamaño máximo estaba

limitado por el hardware, se conservó la mayor configuración válida y se registró la limitación de duración. No se forzaron configuraciones que invalidaran la caracterización previa.

2.3.7. Observabilidad efectiva y latencia de conmutación de reloj en GPU

Antes de la campaña GPU se midieron la cadencia efectiva de actualización de las señales NVML y la latencia de los cambios de reloj bajo carga. La cadencia se seleccionó comparando varios períodos de consulta y eligiendo el más amplio que conservara la cobertura de las cargas cortas y permitiera localizar los cambios de señal. Las lecturas consecutivas iguales no se trataron como observaciones independientes, porque la frecuencia de consulta puede ser mayor que la frecuencia de actualización del sensor.

La latencia de conmutación se midió para cada par de niveles que podía solicitar la política, con al menos tres repeticiones y la GPU bajo carga. El nivel de destino se consideró estable cuando varias lecturas consecutivas permanecieron dentro de la tolerancia definida y no se observaron condiciones de limitación del dispositivo. Como valor conservador se utilizó la mayor latencia observada; esta cota determinó el intervalo mínimo entre decisiones y permitió excluir del análisis energético las fases demasiado breves para completar una transición.

2.3.8. Matriz experimental y control de sesgos

La matriz experimental se construyó como el producto entre las cargas seleccionadas, los niveles de frecuencia y las repeticiones independientes. La evaluación preliminar de la sección 2.3.3 utilizó tres niveles por dispositivo para seleccionar las familias, mientras que la campaña definitiva empleó la rejilla de la Tabla 2.4. El nivel REF mantuvo la configuración nativa del sistema y se utilizó como referencia.

Tabla 2.4: Rejilla fina de frecuencia de la campaña definitiva.

Dispositivo	Niveles de reloj (MHz; REF es el punto nativo)
CPU	REF, 3200, 3100, 3000, 2900, 2800, 2600, 2400, 2200, 2000, 1400, 800
GPU	REF, 1410, 1350, 1290, 1230, 1170, 1110, 810, 510, 210

La separación entre niveles no fue uniforme: se concentraron más puntos en la parte alta del rango para observar si reducciones pequeñas de frecuencia disminuían el consumo

sin superar el presupuesto de degradación del rendimiento. En la parte baja se utilizaron intervalos mayores, suficientes para caracterizar la tendencia general.

Los niveles se declararon en MHz y se verificaron contra los relojes admitidos por el nodo. Cuando un valor no estaba disponible se utilizó el nivel soportado más cercano o se excluyó, dejando registro de la decisión. La calibración Roofline se repitió para cada dispositivo, precisión y nivel de frecuencia.

Para reducir el sesgo experimental, cada combinación se ejecutó en varias repeticiones independientes y el orden se aleatorizó mediante una semilla registrada. La afinidad, el número de hilos y la asignación de núcleos permanecieron constantes; además, el sistema restauró su configuración de frecuencia al finalizar cada campaña o ante una terminación anómala.

Para fijar las frecuencias de CPU se deshabilitó temporalmente *Turbo Boost*, ya que un límite escrito por núcleo no garantiza el reloj físico mientras la aceleración dinámica permanece activa. El estado se verificó por relectura y se restauró al finalizar; por esta razón, 3200 MHz fue el nivel fijo más alto y 3600 MHz quedó reservado al nivel nativo REF. Durante cada corrida se leyó la frecuencia de todos los núcleos asignados: la ausencia o inconsistencia estructural de las lecturas rechazó la corrida, mientras que las desviaciones puntuales respecto al objetivo se anotaron por ventana según la Tabla 2.5.

Tabla 2.5: Taxonomía de clasificación de frecuencia por ventana de muestreo.

Anotación	Condición
Válida	Todos los núcleos delegados leídos en la ventana caen dentro de la tolerancia declarada respecto al reloj efectivamente aplicado.
No confiable	Al menos un núcleo delegado se aparta de la tolerancia, fuera del margen de gracia posterior a una transición de nivel.
No verificable (margen de gracia)	La ventana cae dentro del margen de exclusión inmediatamente posterior a una transición de nivel, donde el transitorio físico de conmutación aún no se ha resuelto; la desviación, si la hay, no se atribuye a una falla de actuación.
No aplicable (gobernador nativo)	La corrida se ejecuta bajo el gobernador nativo de la plataforma (nivel de referencia), sin un objetivo de frecuencia fija contra el cual evaluar la tolerancia.

Esta anotación se conserva como columna independiente de la calidad general de la ventana descrita en la Tabla 2.6: una ventana puede ser válida en calidad general y, simultáneamente, no confiable en frecuencia, o viceversa. Solo las ventanas anotadas como válidas o no aplicables cuentan hacia el mínimo de ventanas exigido para aceptar la corrida; las anotadas como no confiables o no verificables se conservan íntegras en el conjunto de datos, con su anotación, pero no contribuyen a ese mínimo. Adicionalmente, cada corrida produce un resumen de cobertura de frecuencia — fracción de ventanas válidas y longitud de la racha más larga de ventanas no confiables consecutivas — que no condiciona por sí solo la aceptación, pero permite auditar después, corrida por corrida, cuánta de su duración total quedó efectivamente verificada en frecuencia.

2.3.9. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transformaron en una traza auditable mediante la diferenciación entre lecturas consecutivas, conforme a la sección 1.1.3. Cada fila representó una ventana de muestreo e incluyó las métricas derivadas y los campos necesarios para reconstruir su procedencia, entre ellos el identificador de corrida, la carga, el nodo, la frecuencia solicitada y observada, las referencias de calibración y la suma de verificación del binario.

Ninguna observación se eliminó durante este procedimiento. Cada ventana se conservó con una anotación de calidad según la Tabla 2.6; de este modo, el filtrado se realizó durante el análisis y quedó documentada la proporción de datos en cada categoría.

El intervalo inicial de cada corrida se trató como un transitorio de arranque y se excluyó del análisis para caracterizar únicamente la fase estable del kernel. Su duración se calculó durante el post-procesamiento, por lo que la adquisición conservó la traza completa y permitió ajustar el criterio sin repetir la ejecución.

La detección se aplicó sobre instrucciones por ciclo en CPU y sobre la utilización del dispositivo en GPU, mediante ventanas móviles de tamaño fijo. Primero se buscó el instante a partir del cual el coeficiente de variación permaneció por debajo del umbral durante dos ventanas consecutivas y la media superó un nivel mínimo de actividad; esta última condición evitó confundir el reposo inicial de la GPU con un estado estable. Cuando este criterio no produjo un punto válido, se utilizó una detección de cambios basada en la reducción del error cuadrático entre segmentos y se seleccionó el primero cuya media alcanzó una fracción declarada de la meseta de mayor actividad [19, 5]. En las cargas de GPU, el límite propuesto también se contrastó con la transición observada en la traza de potencia; cuando la cadencia del dispositivo no permitió identificarla, la limitación quedó registrada en el catálogo y no se

asignó un valor sin respaldo.

La calibración se integró en la propia campaña. En una primera pasada se procesó la traza completa sin excluir ventanas iniciales; después se calcularon los intervalos de calentamiento, se actualizó el catálogo con control de versión y suma de verificación, y se reprocesaron los datos crudos sin volver a ejecutar los kernels. Finalmente, se reevaluó la aceptación de cada corrida con los intervalos calibrados y se registró cualquier cambio de veredicto, sin eliminar los artefactos originales.

Tabla 2.6: Taxonomía de anotaciones de calidad por ventana de muestreo.

Anotación	Condición
Válida	La ventana satisface todos los criterios de calidad.
Sin diferencia previa	Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.
Excluida por calentamiento	La ventana pertenece al intervalo inicial declarado como transitorio de arranque.
Contadores degradados	Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.
Intensidad indefinida	No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable.
Energía inválida	La lectura energética correspondiente no es válida en una campaña que la requiere.
Sin lectura de frecuencia	No se dispone de la frecuencia efectivamente observada durante la ventana.

Además, cada corrida se validó de forma integral mediante la estructura de muestreo, la suma de verificación del binario, el criterio de éxito de la carga y la unicidad de su identificador. Las corridas que no cumplieron estas condiciones se registraron como rechazadas y sus artefactos se conservaron para inspección.

2.3.10. Unidad de observación del conjunto de entrenamiento

Las ventanas se conservaron como registro auditable, pero no se trataron siempre como observaciones independientes para el entrenamiento. La unidad de análisis se definió por dispositivo, de acuerdo con la resolución de la medición que sustentó la etiqueta; así se evitó sobrestimar el tamaño efectivo del conjunto de datos y el desempeño del modelo.

CPU: intervalo del controlador de memoria. Cada fila correspondió a un intervalo de lectura del controlador de memoria, cuya resolución fue menor que la de los contadores de núcleo (sección 1.1.3). Los FLOPs y las demás métricas se recalcularon a partir de los deltas crudos agregados dentro de ese intervalo, en lugar de promediar cocientes calculados por ventana. Se rechazaron los intervalos sin lecturas válidas de memoria, con cobertura insuficiente, que cruzaron el límite de calentamiento o que contuvieron contadores o frecuencias degradados; el motivo quedó registrado y no se imputaron valores sustitutos.

GPU: corrida completa. Cada fila correspondió a una combinación de carga, frecuencia y repetición, porque las señales en línea se actualizaron a una cadencia menor y la intensidad operacional se midió una sola vez para todo el kernel (sección 2.3.7). Las características se obtuvieron mediante estadísticos robustos de las muestras válidas posteriores al calentamiento. La intensidad operacional, el punto de inflexión y la etiqueta se conservaron solo para trazabilidad y se excluyeron de las entradas del modelo; asimismo, los microbenchmarks con fases programadas se mantuvieron como controles diagnósticos y no se utilizaron para el entrenamiento.

2.3.11. Medición directa de FLOPs por ventana y su validación empírica

En CPU, los FLOPs de cada ventana se midieron directamente mediante los cuatro subeventos de doble precisión de la familia `FP_ARITH_INST_RETIRED`. Esta decisión evitó estimarlos a partir del total reportado por la carga y permitió alinear la medición con las demás señales de telemetría.

Codificación y ponderación. En el procesador Intel Xeon Gold 5315Y, la familia utilizó el *EventSelect* `0xC7` y las máscaras `0x01`, `0x04`, `0x10` y `0x40` para las operaciones escalares y vectoriales de 128, 256 y 512 bits. Como el sistema no expuso alias simbólicos para estos eventos, se programaron como eventos crudos mediante `perf_event_open`. Cada conteo se ponderó por el número de elementos FP64 de la instrucción; las operaciones FMA ya fueron contabilizadas por el hardware como dos operaciones por elemento y no requirieron una corrección adicional [31]:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta N_{\text{escalar},i} + 2 \cdot \Delta N_{128,i} + 4 \cdot \Delta N_{256,i} + 8 \cdot \Delta N_{512,i} \quad (2.2)$$

La codificación se obtuvo de la definición oficial de Intel para Ice Lake-SP [31] y se

Tabla 2.7: FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.

Ancho vectorial	Elementos FP64	FLOPs (ADD/MUL)	FLOPs (FMA)
Escalar	1	1	2
128 bits	2	2	4
256 bits	4	4	8
512 bits	8	8	16

contrastó con el grupo FLOPS_DP de LIKWID [54].

Verificación. La medición se contrastó con una multiplicación de matrices de volumen analíticamente conocido y con un kernel multihilo de NAS Parallel Benchmarks, ambos ejecutados con la afinidad utilizada en la campaña. También se verificó la razón entre el tiempo de conteo y el tiempo habilitado para detectar multiplexación. La configuración de producción conservó los cuatro subeventos de punto flotante dentro del presupuesto de nueve contadores descrito en la sección 1.1.3; cuando alguno no estuvo disponible, la ventana quedó con intensidad operacional indefinida. Los resultados y el alcance de esta validación para Ice Lake-SP se presentan en la sección 3.1.1.

2.3.12. Reproducibilidad

Cada campaña se definió mediante un manifiesto bajo control de versiones, con las cargas, frecuencias, repeticiones, período de muestreo, asignación de núcleos, semilla y referencias de calibración. Junto con los datos se conservaron el perfil de capacidades del nodo, los resultados de calibración y los metadatos de cada corrida.

Para reanudar una campaña, el orquestador comparó la huella del protocolo almacenada con la del manifiesto vigente; cualquier diferencia detuvo la ejecución para evitar mezclar corridas obtenidas con configuraciones distintas. Cuando la huella coincidió, se reutilizó la calibración previamente validada. Además, después de modificar una carga se repitió una combinación completa y se comprobó la consistencia de su intensidad operacional, su punto de inflexión y su etiqueta.

2.3.13. Selección y registro de características

Las características se seleccionaron por separado para CPU y GPU. Primero se calcularon las correlaciones de Pearson y Spearman entre las columnas candidatas; cuando un par superó

el umbral definido, se conservó la medición física más directa. Después se utilizó el factor de inflación de la varianza para identificar relaciones entre más de dos variables, y se excluyeron las columnas ausentes, constantes o con valores no finitos.

Las variables empleadas para construir la etiqueta —intensidad operacional, punto de inflexión, FLOPs, bytes y expectativa cualitativa del catálogo— se prohibieron como entradas del clasificador, con el fin de evitar fuga de información (sección 1.1.11). La lista final se revisó y se registró en una especificación versionada antes del entrenamiento; también se documentaron las columnas descartadas y el motivo de cada decisión.

2.4. Fase 2: Desarrollo del modelo de aprendizaje automático

En esta fase se desarrollan las técnicas de análisis orientadas a la construcción de un modelo predictivo capaz de identificar el comportamiento de la carga de trabajo en tiempo de ejecución. El proceso de preprocesamiento incluye la limpieza, la normalización y la selección de características relevantes, partiendo del conjunto de datos producido en la Fase 1 y aplicando sobre él los criterios de filtrado a partir de las anotaciones de calidad descritas en la Tabla 2.6. Para GPU, este proceso ya se ejecutó (sección 3.9); para CPU permanece pendiente.

El problema se formula como una tarea de clasificación supervisada binaria, donde el modelo recibe como entrada un vector de telemetría — correspondiente a una ventana de ejecución en CPU, o a una corrida completa en GPU (sección 3.9) — y produce como salida la etiqueta de régimen. Dado que el sistema debe operar en tiempo de ejecución, se prioriza el uso de modelos de baja complejidad computacional, entre ellos árboles de decisión, bosques aleatorios, regresión logística y ensambles potenciados, comparados mediante métricas de clasificación y latencia de inferencia medida.

Un criterio metodológico relevante en esta fase es la estrategia de partición del conjunto de datos. Dado que las observaciones provenientes de una misma corrida no son independientes entre sí, una partición aleatoria a nivel de observación produciría una estimación optimista de la capacidad de generalización. Por ello, la partición se realiza a nivel de familia algorítmica (*leave-one-family-out* para GPU), de modo que la evaluación mida la capacidad del modelo para generalizar hacia algoritmos no observados. A partir de este análisis se selecciona el modelo que represente el mejor compromiso entre desempeño predictivo y costo de inferencia, y se serializa para su integración en el sistema de control.

2.5. Fase 3: Implementación del agente de control DVFS

2.5.1. Arquitectura de dos bucles desacoplados

El agente de control se implementó como un proceso en espacio de usuario formado por dos bucles desacoplados y una señal de coordinación. El bucle de CPU operó sobre ventanas de aproximadamente 1 ms, coherentes con la telemetría de la Fase 1, y ejecutó la inferencia y la decisión de frecuencia a baja latencia. El bucle de GPU operó por fases de ejecución completas, dado que la conmutación de reloj en GPU implicó un coste que no se amortizaba a la cadencia del orden del milisegundo. Cuando el bucle de GPU detectó actividad sostenida por encima de un umbral, forzó un piso de frecuencia en el dominio CPU para evitar interpretar esperas bloqueantes como fases *compute-bound*.

2.5.2. Detección de fase en GPU

La detección de fases en GPU se implementó mediante sondeo periódico de la utilización reportada por la interfaz de gestión del dispositivo, sin instrumentar el binario objetivo. Se configuró la cadencia de sondeo y un umbral de ruido para emitir eventos de inicio de fase al cruzar el umbral en sentido ascendente; las transiciones descendentes se registraron para trazabilidad. Esta solución se adoptó por su bajo nivel de intrusión y su eficacia práctica.

2.5.3. Diseño del bucle de CPU e integración con el clasificador

El bucle de CPU incorporó una máquina de decisión independiente de la biblioteca de inferencia, de modo que su lógica se verificó aisladamente. En cada ciclo se construyó el vector de características a partir del snapshot de telemetría, se invocó el modelo exportado desde la Fase 2 y se consultó la tabla de política. Solo se aplicó un nuevo nivel de frecuencia cuando la clase inferida cambió respecto al ciclo anterior, para reducir oscilaciones. La integración se realizó en el mismo lenguaje del colector de telemetría para evitar cruces costosos entre runtimes.

2.5.4. Derivación *offline* de la tabla de política

La tabla de política se derivó *offline* a partir del barrido de frecuencia de la Fase 1. Para cada combinación de dispositivo, clase de régimen y nivel de frecuencia se calculó la mediana del Producto Energía–Retardo por kernel y se comparó con la referencia nativa mediante pruebas de significancia pareadas (Wilcoxon o *t-test*, según corresponda). Cuando ningún nivel mejoró el EDP de forma significativa, la política se fijó en *no actuar*. Para GPU

se exigió además una latencia de conmutación verificada bajo carga real antes de habilitar cualquier actuación. La tabla resultante se serializó y el agente la cargó al arrancar.

2.5.5. Actuación y validación en modo simulado

La aplicación de niveles de frecuencia se centralizó en un componente único que registró el estado previo y garantizó su restauración incluso ante terminaciones anómalas. Antes de habilitar escrituras reales, el agente se ejecutó en modo de simulación, donde se registraron las decisiones sin modificar hardware, y se validó el comportamiento frente a corridas históricas. Solo tras estas comprobaciones se habilitó la actuación real bajo control administrativo y de reserva de recursos.

2.6. Fase 4: Validación experimental y análisis de resultados

2.6.1. Escenarios de comparación

La evaluación comparó el agente propuesto frente a tres escenarios de referencia: los gobernadores reactivos nativos (`ondemand`, `schedutil`) y un modo de frecuencia fija (`performance`). En cada escenario se ejecutó el mismo catálogo de cargas bajo las condiciones de aislamiento de la Fase 1 y se midieron tiempo de ejecución, energía, potencia media y Producto Energía–Retardo.

2.6.2. Conmutación de gobernadores y protocolo de medición

La conmutación de gobernadores se implementó verificando primero su disponibilidad en el nodo y aplicándola dentro de un contexto acotado que restauró el gobernador original al finalizar. Las condiciones de afinidad, número de hilos y asignación de núcleos se mantuvieron constantes entre escenarios para garantizar comparabilidad.

2.6.3. Cálculo del EDP y prueba de significancia

El EDP se calculó a partir de las trazas y se contrastó con las líneas base mediante las mismas pruebas pareadas empleadas en la generación de la política, reportándose los resultados desagregados por combinación de dispositivo y clase de régimen.

2.6.4. Generación del reporte comparativo

El reporte final se generó a partir de los archivos de telemetría identificados por campaña. El procedimiento manejó escenarios incompletos omitiendo comparaciones faltantes con aviso explícito, lo que permitió generar reportes parciales sin introducir supuestos.

2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda dicha asignación, de acuerdo con lo expuesto en la sección 1.1.6. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos. Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

Capítulo 3

Resultados

(Cambiar este párrafo, esta mal para la version final) Este capítulo reporta los resultados de la Fase 1: la construcción y validación del instrumento de medición, la campaña CPU solicitada en múltiples niveles — junto con la verificación posterior que demostró que estos no constituyeron frecuencias físicas distintas —, la caracterización de precisión aritmética del catálogo de GPU, y la ampliación del catálogo con kernels de patrón de acceso irregular. Incluye también un primer entrenamiento y análisis del clasificador GPU (Fase 2, secciones 3.9 en adelante), posible porque el conjunto de entrenamiento de GPU ya quedó definido dentro de esta misma fase; el resto de la Fase 2 (clasificador CPU), la Fase 3 (implementación del agente) y la Fase 4 (evaluación de EDP frente a los gobernadores nativos) están fuera del alcance de estos resultados y se reportarán al cerrarse el desarrollo completo del proyecto.

3.1. Descripción del conjunto de datos experimental

El catálogo experimental final está compuesto por 9 cargas de CPU (7 de conjunto de datos y 2 de calibración) y 12 cargas de GPU (8 de conjunto de datos y 4 de calibración). Todas las cargas de CPU operan en doble precisión, verificado empíricamente para las 9 sobre el catálogo completo, no solo sobre las empleadas en la validación principal (sección 2.3.11). El conjunto de datos GPU combina siete kernels de la suite Rodinia y una multiplicación densa de matrices sobre cuBLAS. Las cuatro cargas de calibración corresponden a tres microbenchmarks propios que sostienen los techos Roofline — ancho de banda, pico FP32 y pico FP64 — y una DGEMM de cuBLAS conservada como referencia informativa independiente.

La auditoría de precisión aritmética de la sección 3.3 excluyó del catálogo GPU un kernel de simulación térmica cuya intensidad operacional real, con precisión mixta, cae demasiado

cerca del punto de inflexión para clasificarlo con confianza; se reemplazó por una eliminación gaussiana de la misma suite, confirmada empíricamente pura en la precisión declarada.

3.1.1. Validación de la medición directa de FLOPs

La medición directa descrita en la sección 2.3.11 reprodujo el volumen analítico de operaciones de la multiplicación de matrices con errores relativos de 0.29 % sin afinidad fijada y de 0.30 % con la afinidad de la campaña. En el kernel de NAS Parallel Benchmarks, el error respecto del total reportado por el programa fue de 7.48 %; la diferencia se atribuyó a que el contador cubrió el proceso completo, incluida la verificación final, mientras el tiempo interno del programa delimitó solo su ciclo principal. En ambas pruebas, la razón entre tiempo de conteo y tiempo habilitado fue igual a uno, sin evidencia de multiplexación.

La verificación posterior comprendió 66 corridas y cerca de 1.29 millones de ventanas con diferencias válidas; todas obtuvieron sus FLOPs mediante los contadores de hardware y ninguna presentó degradación en estos eventos. Estos resultados respaldan el uso del método en la plataforma Ice Lake-SP evaluada, pero no constituyen una cota de error ni se extrapolan a otras microarquitecturas sin repetir la validación.

3.2. Validación de la actuación DVFS en CPU

Metodología de actuación verificada

La actuación de frecuencia por núcleo se verifica en tres niveles, no solo por el código de retorno de la escritura. Primero, la escritura restringe siempre, junto con cada procesador lógico delegado, a sus hermanos de *SMT* según la topología real de la plataforma — de lo contrario los hermanos sin restringir permiten que el planificador entregue ciclos adicionales al núcleo físico compartido, diluyendo el límite solicitado sobre el procesador delegado. Segundo, la confirmación de que el nuevo límite se hizo efectivo se realiza generando actividad de cómputo real sobre los mismos procesadores mientras se lee el reloj reportado por el sistema operativo — esa lectura deriva de la proporción entre ciclos reales y de referencia acumulados, una cantidad que solo es representativa bajo carga activa. Tercero, la verificación por ventana excluye un margen de ~ 10 ms inmediatamente posterior a cada transición de nivel, correspondiente al tiempo físico de asentamiento del reloj entregado.

Verificado experimentalmente: restringiendo únicamente los procesadores lógicos delegados, la razón de rendimiento entre el nivel más alto y el más bajo del barrido se mantiene en 0.995 (sin escalar); restringiendo además sus hermanos de *SMT*, la razón sube a 3.78, cercana

a la razón de cuatro esperada entre los niveles comparados. El ancho de banda calibrado, limitado por memoria y no por reloj de núcleo, no depende de esta verificación: 58.0 GB/s bajo el nivel más alto del barrido frente a 57.45 GB/s en estado nativo sin restricción, dentro del margen de ruido de medición.

Resultado de la campaña

Sobre la plataforma descrita en la Tabla 2.1: 7 kernels de conjunto de datos, 6 niveles solicitados (REF y F0–F4, Tabla 2.4), 3 repeticiones por combinación — 126 combinaciones, 126 aceptadas, 4.67 horas-núcleo, 1,355,352 ventanas (95.4 % con estado de calidad ok). Para cada nivel fijo, `scaling_max_freq/scaling_min_freq` conservan exactamente el valor solicitado (3.6, 2.9, 2.2, 1.5 y 0.8 GHz para F0–F4) y la restauración final devuelve el sistema a su estado anterior.

Esta aceptación confirma la integridad de la adquisición y el etiquetado, pero los seis niveles solicitados no corresponden a seis frecuencias físicas distintas: la plataforma mantiene el turbo activo bajo `intel_pstate/HWP`, y los controles globales capaces de limitarlo (`no_turbo`, `max_perf_pct`) no son escribibles con los permisos actuales — una carga solicitada a 2.2 GHz sostiene ~ 3.5 GHz reales bajo carga, confirmado con 40 lecturas periódicas de sysfs independientes del instrumento. La campaña no se presenta, en consecuencia, como un barrido DVFS válido: sus intensidades y etiquetas describen las cargas en el único punto de operación realmente observado, pero las repeticiones F0–F4 no son estados DVFS independientes hasta resolver el control de turbo.

3.3. Caracterización de precisión aritmética en el catálogo de GPU

Como se argumentó en la sección 2.3.5, la intensidad operacional de cada carga de GPU se determina perfilando con Nsight Compute **ambas** precisiones aritméticas simultáneamente, nunca solo la declarada por el catálogo — el único método capaz de detectar que un kernel ejecuta una fracción real de su trabajo en una precisión distinta de la nominal. Se reperfiló bajo este criterio el catálogo completo de siete kernels de Rodinia.

Tres kernels resultaron empíricamente puros en la precisión declarada: una simulación de N-cuerpos (100 % doble precisión, intensidad operacional dentro de 0.1 % del valor declarado), una descomposición LU (100 % simple precisión, convergencia verificada hasta 200 lanzamientos de kernel) y una transformada de *wavelets* (100 % simple precisión, dentro de

0.5 %).

Los otros cuatro mostraron mezcla de precisión no trivial. Un kernel de simulación térmica ejecuta 30.4 % de sus operaciones en doble precisión pese a declararse de precisión simple, con intensidad operacional real de 7.21 FLOP/byte — justo en el borde del punto de inflexión de simple precisión (7.28–7.98 FLOP/byte) —, sin base metodológica sólida para clasificarlo contra un único techo; se retiró del catálogo. Dos kernels adicionales (retropropagación y un solucionador de ecuaciones diferenciales ordinarias) resultan mayoritariamente de doble precisión (82.3 % y 71.3 %) pese a declararse de precisión simple; la clasificación *memory-bound* no cambia en ninguno de los dos — la intensidad operacional real permanece órdenes de magnitud por debajo de cualquier techo posible en esta plataforma —, y la precisión declarada en el catálogo se corrigió para reflejar lo medido.

El cuarto caso, la multiplicación densa de matrices sobre cuBLAS, ejecuta sobre unidades Tensor Core. Su intensidad operacional declarada (68.0 FLOP/byte) proviene del conteo analítico del propio algoritmo, contrastado independientemente contra la tasa que el binario reporta al finalizar (discrepancia menor a 0.2 %) — las instrucciones Tensor Core no incrementan los contadores clásicos de multiplicación–suma fusionada, así que el conteo por instrucciones de los demás kernels de Rodinia no es aplicable a este caso.

3.4. Estado del control de frecuencia en GPU

La actuación de frecuencia de GPU se implementa mediante el bloqueo de reloj de *streaming multiprocessor* (`nvidia-smi -lgc`), delegado mediante privilegios restringidos de `sudo`, y se verificó bajo carga real y no solo por el código de retorno del comando: dentro de una asignación exclusiva, con un microbenchmark FP64 propio como carga y restauración automática del estado al finalizar, un objetivo de 600 MHz produjo 305 lecturas con utilización activa, todas exactamente a 600 MHz; un objetivo de 1200 MHz produjo 174 lecturas activas, todas exactamente a 1200 MHz. El actuador retiene el reloj solicitado bajo carga real en los dos niveles probados (NVIDIA 610.57.04/CUDA 13.3).

El resto del *plumbing* que una campaña DVFS de GPU exige — verificación de actividad ajena antes de cada combinación, piso mínimo de utilización de 5 % para superar el ruido del sensor, y desacoplamiento completo de los ejes de frecuencia CPU y GPU mediante su producto cartesiano — cuenta con pruebas unitarias, pero el producto cartesiano y el *pipeline* completo multi-frecuencia aún no se han ejercitado como una campaña GPU de producción; la recolección y aceptación del conjunto multi-frecuencia permanecen como el siguiente paso experimental.

3.5. Medición directa de bytes movidos en memoria (*uncore*)

La lectura de los contadores de *uncore* del controlador de memoria (sección 1.1.3) usa la capacidad CAP_PERFMON asignada al binario `perf`, sin reducir la restricción de `perf_event Paranoid` para todo el sistema. El instrumento invoca a `perf` como subproceso, fijado a un núcleo reservado para telemetría — nunca a los núcleos donde corre la carga medida, para no contaminar los contadores por proceso que allí se miden —, y procesa en vivo su salida periódica en vez de abrir los contadores directamente. El presupuesto simultáneo de eventos de PMU es de nueve, no diez: la vigilancia por interrupción no enmascarable del sistema operativo (ajena a este proyecto, presente en cualquier despliegue de servidor típico) reserva de forma permanente un contador de hardware por núcleo; el evento descartado es el *proxy* de tráfico de caché de segundo nivel, sin papel en la clasificación desde que *uncore* lo reemplaza como fuente admitida.

La medición directa de *uncore* reemplaza por completo al *proxy* de fallos de caché en la clasificación por ventana — no como respaldo, sino como única fuente admitida — aunque no a la granularidad fina de una ventana individual: `perf stat` reporta en intervalos de al menos ~ 10 ms, mientras que una ventana de CPU dura ~ 1 ms. El instrumento agrega los FLOPs de todas las ventanas cubiertas por cada intervalo de *uncore* y calcula la intensidad operacional real a esa granularidad más gruesa, difundida a cada una de esas ventanas como `operational_intensity/phase_label_train`. Una ventana que ningún intervalo real llega a cubrir queda deliberadamente indefinida, nunca recurre al *proxy* como alternativa. `bytes_moved_window` (el *proxy*) se sigue calculando y reportando en su propia columna, únicamente como referencia de comparación — nunca alimenta la clasificación.

Verificado de extremo a extremo con una carga de tráfico de memoria real: de 332 ventanas de una corrida de referencia, 322 quedaron cubiertas por al menos un intervalo real de *uncore*, con conteos de lectura/escritura variables y coherentes entre intervalos consecutivos — nunca en cero, la señal esperada de una medición genuina bajo carga activa. De esas 322, 301 quedaron efectivamente clasificadas con el valor real; las 21 restantes tenían cobertura de bytes pero al menos una ventana del grupo sin FLOPs medidos válidos, y quedaron correctamente indefinidas en vez de calcular una intensidad con un numerador incompleto.

3.5.1. Validación de esfuerzo y aislamiento de la campaña

Antes de comprometer una campaña real de varias horas, se ejecutó una prueba de esfuerzo dirigida a las condiciones que una corrida aislada no expone: cinco repeticiones

consecutivas del patrón de arranque/cierre del subproceso de medición, una carga de duración extrema por debajo del piso práctico de intervalo, y una carga más prolongada para observar la cadencia de intervalos de principio a fin. Las cinco repeticiones consecutivas completaron con datos reales y sin procesos remanentes; el espaciado mínimo entre intervalos consecutivos se sostiene en los ~ 10 ms esperados de manera consistente a lo largo de toda la corrida, incluidos sus primeros instantes.

Los contadores de *uncore*, de alcance de sistema, exigen reserva exclusiva del nodo — verificada como precondition bloqueante antes de iniciar cualquier campaña que los solicite, para que ningún proceso ajeno pueda contaminar una lectura que, por diseño, no puede atribuirse a un proceso concreto. Antes de comprometer una campaña real, una prueba de humo a escala reducida ejercita el mismo camino de código de principio a fin — verificación previa, calibración, escritura de frecuencia, *uncore*, post-procesamiento y validación de ventanas — sobre una única carga, antes de repetir el mismo procedimiento contra la campaña completa.

Verificado con dos campañas de humo completas: la razón mínima de tiempo contando sobre tiempo habilitado se sostiene en 1.0 a lo largo de toda una corrida de prueba, sin ningún delta negativo en los cuatro sub-eventos de punto flotante; una campaña de humo con barrido completo de frecuencia (REF y F0–F4, dieciocho combinaciones) acepta la totalidad; y una segunda, sobre los siete kernels reales del catálogo a frecuencia nativa (veintiuna combinaciones), acepta también la totalidad, con clasificaciones físicamente coherentes: la multiplicación densa de matrices resulta *compute-bound* en el 100 % de sus ventanas válidas, con intensidad operacional entre 15.0 y 29.6 FLOP/byte, muy por encima del punto de inflexión corregido (sección 3.6); los seis kernels de la suite NAS Parallel Benchmarks resultan predominantemente *memory-bound*, con la excepción de una fracción minoritaria de ventanas *compute-bound* en el kernel de resolución de sistemas por métodos implícitos, consistente con el desplazamiento del punto de inflexión documentado en esa misma sección.

3.6. Validación cruzada de los techos Roofline con una herramienta independiente

Los techos del modelo Roofline se determinan, como se explicó en la sección 1.1.2, mediante calibración empírica sobre la propia plataforma. Verificar el resultado de esa calibración exige una vía independiente del mecanismo de calibración: se empleó Intel Advisor, que construye su propio análisis Roofline mediante instrumentación binaria directa del bucle más costoso de un programa — contabilizando FLOPs y bytes ejecutados realmente, sin pasar por contadores de PMU —, un mecanismo de medición genuinamente distinto y por tanto

una validación cruzada real [30].

Los microbenchmarks de calibración solicitan explícitamente el ancho vectorial máximo de la plataforma (AVX-512, ocho elementos de doble precisión por instrucción) en el microbenchmark de densidad aritmética — no en el de ancho de banda, limitado por memoria y no por ejecución vectorial, donde el ancho vectorial máximo produce una regresión de rendimiento medible. El mismo criterio se aplica a la cadena de construcción de los seis kernels de conjunto de datos de la suite NAS Parallel Benchmarks, sin la bandera de ancho vectorial máximo por combinar regímenes *memory-bound* y *compute-bound* bajo una configuración de compilación compartida.

Contrastado sobre una multiplicación de matrices densas de referencia, Advisor midió 577.7 GFLOP/s con una intensidad operacional de 1.091 FLOP/byte sobre los mismos seis núcleos de la calibración de producción — coherente con el punto de inflexión propio del catálogo. El ancho de banda calibrado valida contra Advisor dentro de 12–15 % (58.4–58.8 GB/s frente a 64.0–66.8 GB/s), y el techo de cómputo calibrado por este instrumento (417.8–555 GFLOP/s) es consistente con la referencia independiente de Advisor. El punto de inflexión final queda en el rango **7.0–9.3 FLOP/byte**, la referencia empleada en el resto de este capítulo para clasificar el catálogo de conjunto de datos (sección 3.5.1).

3.7. Evolución del catálogo GPU: del DGEMM de referencia a un reemplazo verificable

Los envoltorios de cuBLAS para multiplicación densa de matrices (`cutlassDgemm`) resultan, en esta combinación de GPU/controlador, inutilizables como ancla *compute-bound* del catálogo: el binario enruta internamente a un kernel de unidades Tensor Core (`cutlass_80_tensorop_d884gen` confirmado por el nombre de kernel en el CSV crudo de Nsight Compute) sin que se le solicite explícitamente, y las instrucciones Tensor Core no incrementan los contadores clásicos de multiplicación–suma fusionada que este instrumento usa para calcular la intensidad operacional — 0 instrucciones DFMA contadas para una carga que, por su escalamiento $O(N^3)$ de cómputo frente a $O(N^2)$ de memoria, debería ser de las más *compute-bound* del catálogo. El *pipeline* excluye esta ruta mediante un guardarraíl explícito (`fase1_telemetria/ncu_convergence.py`, condición `tensor_instructions_observed > 0`) sobre el formato de salida de Nsight Compute usado en producción; auditados los 37 kernels del catálogo con este criterio, 17 muestran actividad Tensor Core no nula (umbral de fracción calibrado en 10 %, sobre un hueco real observado entre 3.3 % de ruido y 20.8 % de actividad genuina).

El reemplazo verificado es CUTLASS con la clase de operación `OpClassSimt` — que fuerza explícitamente unidades de cómputo escalares, nunca Tensor Core —: un programa *standalone* instanciando `cutlass::gemm::device::Gemm<double, ..., OpClassSimt, Sm50, ...>` mide una intensidad operacional de **26.73 FLOP/byte**, confirmada con conteo exacto de instrucciones DFMA igual a $N^3 = 4096^3$, muy por encima del ridge de doble precisión en el nivel de referencia (6,898 FLOP/byte, sección 3.8) y sin ninguna actividad Tensor Core detectada. Este kernel (*gpu_cutlass_simt_dgemm_n4096*) es la ancla *compute-bound* de multiplicación densa del catálogo final, que queda en 18 kernels y 540 corridas planificadas; *gpu_rajaperf_gemm* (*Polybench_GEMM* de RAJAPerf, 2.81 FLOP/byte, cerca del ridge) se conserva como segunda referencia de suite de terceros, más cercana al punto de inflexión.

3.8. Barrido de memoria real y calibración de warmup: composición final del dataset

3.8.1. Techos Roofline de referencia para GPU

Siguiendo el mismo principio de calibración empírica descrito para CPU (sección 3.6), el ridge de GPU se determina por precisión aritmética y nivel de frecuencia, mediante un microbenchmark propio de densidad aritmética pura (bucle de multiplicación-suma fusionada en registros, sin cuBLAS) y un microbenchmark compartido de ancho de banda equivalente a STREAM Triad. En el nivel de referencia (REF, reloj nativo sin restricción), el ridge resultante es de **14.965852 FLOP/byte en simple precisión** y **6.898377 FLOP/byte en doble precisión** — ambos calculados como $P_{\text{pico, precisión}}/BW_{\text{pico}}$ a partir del rendimiento y ancho de banda pico medidos en ese mismo nivel. Estos valores, y no una aproximación previa citada informalmente durante el desarrollo de este trabajo (del orden de 3.4–3.7 y 7.3–8.0 FLOP/byte, correspondientes a un nivel de frecuencia distinto de REF), son los empleados en la Tabla 3.3 y en la Figura 3.3 para posicionar cada kernel del catálogo final frente al techo de su propia precisión.

3.8.2. Barrido de memoria a escala real de VRAM

El catálogo incluye, para las cinco familias *dual_** de GPU, tamaños que ocupan una fracción significativa de la memoria física del acelerador (A100, 40 GB de VRAM) — el mismo principio ya aplicado en CPU mediante un barrido de tamaños sobre *dual_fft_cpu* — además de sus variantes de escala pequeña ya validadas. Los tamaños se verifican con ejecución real antes de comprometer tiempo de perfilado, contra la memoria medida en vivo

con `nvidia-smi` (no contra una estimación por fórmula). La Tabla 3.1 y la Figura 3.1 resumen los cinco tamaños verificados y perfilados, apuntando a $\sim 50\%$ de VRAM.

Tabla 3.1: Resultado del barrido de memoria a escala real de VRAM para las cinco familias *dual_** de GPU.

Familia	N final	Memoria real	% VRAM	Resultado / decisión
<i>dual_fft_gpu</i>	25600	20.43 GB	49.9 %	Excluido: precisión mixta detectada
<i>dual_cholesky_gpu</i>	32768	8.64 GB	21.1 %	Excluido: no converge (cache-residencia)
<i>dual_axpy_gpu</i>	1 280 000 000	19.50 GB	48.8 %	En campaña final
<i>dual_spmv_gpu</i>	200 000 000	19.80 GB	49.5 %	En campaña final
<i>dual_stencil_gpu</i>	36864	20.66 GB	51.7 %	En campaña final

De las cinco familias, tres (*dual_axpy_gpu*, *dual_spmv_gpu*, *dual_stencil_gpu*) entran a la campaña final a escala de VRAM real. *dual_cholesky_gpu_N32768* y *dual_fft_gpu_N25600* quedan excluidos: el primero no converge dentro del criterio de estabilidad del 1 % a esta escala (efecto de residencia en caché que se diluye al crecer N , intensidad operacional de 0.217 FLOP/byte); el segundo mezcla precisión simple y doble a partir de este tamaño (Nsight Compute), a diferencia de *dual_fft_gpu_N4096* (en campaña, precisión pura confirmada). A la escala de memoria que efectivamente usa un acelerador grande, las cinco familias *dual_** se confirman o profundizan hacia el régimen *memory-bound*.

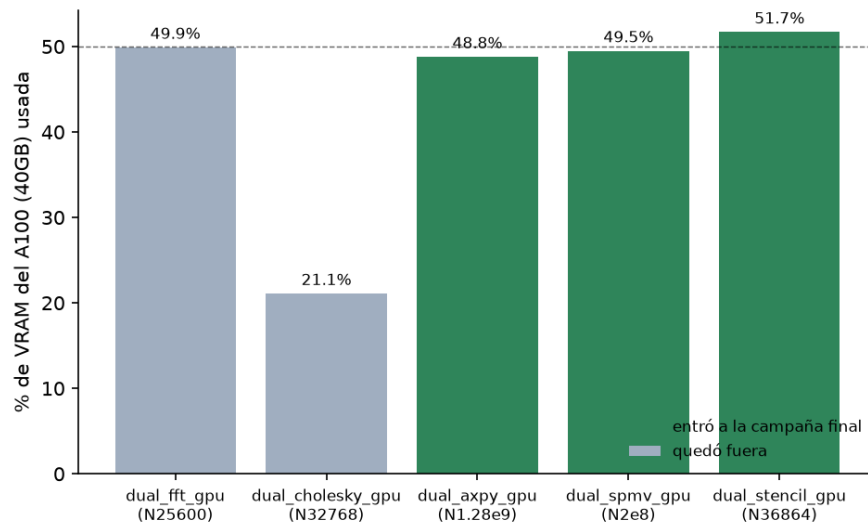


Figura 3.1: Porcentaje de VRAM del A100 (40 GB) alcanzado por cada tamaño perfilado en el barrido de memoria. Los tres tamaños que entraron a la campaña final se distinguen de los dos que quedaron documentados pero excluidos. Fuente: elaboración propia.

3.8.3. Calibración de warmup y su efecto sobre el conjunto de entrenamiento

El transitorio de arranque de cada kernel se calibra con evidencia real, no se declara a mano: el detector de estado estable (método `cv_threshold`) exige dos ventanas móviles consecutivas con coeficiente de variación menor o igual a 5 % sobre $\text{IPC}/\text{gpu_util_pct}$, y el warmup adoptado es $1,2\times$ el instante detectado, tomando el máximo observado entre repeticiones y niveles de frecuencia por kernel. Los 18 kernels de `gpu_final.yaml` dan `status=measured` sobre 28–30 corridas analizadas cada uno (Tabla 3.2).

Tabla 3.2: Warmup calibrado con evidencia real para los 18 kernels de la campaña final GPU.

Kernel	Warmup calibrado (s)	Corridas
<i>rodinia_lavamd</i>	6.51	30
<i>rodinia_dwt2d</i>	6.41	30
<i>gpu_cutlass_simt_dgemm_n4096</i>	6.64	30
<i>gpu_rajaperf_gemm</i>	6.81	30
<i>gpu_rajaperf_heat_3d</i>	5.41	30
<i>dual_axpy_gpu_N10000000</i>	5.69	30
<i>dual_axpy_gpu_N1280000000</i>	5.44	30
<i>dual_stencil_gpu_N36864</i>	4.47	30
<i>dual_cholesky_gpu_N2048</i>	4.35	30
<i>dual_stencil_gpu_N1024</i>	4.33	30
<i>dual_spmv_gpu_N1000000</i>	4.34	30
<i>dual_spmv_gpu_N200000000</i>	4.32	30
<i>dual_fft_gpu_N4096</i>	4.11	30
<i>gpu_rajaperf_jacobi_2d</i>	4.09	30
<i>rodinia_heartwall</i>	1.91	30
<i>rodinia_lud</i>	1.67	28
<i>rodinia_gaussian</i>	1.66	30

El conjunto de entrenamiento final, tras excluir el transitorio de arranque de cada corrida, queda en **334 corridas elegibles**, **16 kernels** y **11 familias algorítmicas**: *rodinia_lavamd* y *rodinia_dwt2d* quedan sin ninguna corrida con al menos 5 ventanas de estado estable, el objetivo por repetición — su warmup calibrado (6.4–6.5 s) es largo en relación con su duración total de corrida. La Figura 3.2 muestra el balance de clases resultante.

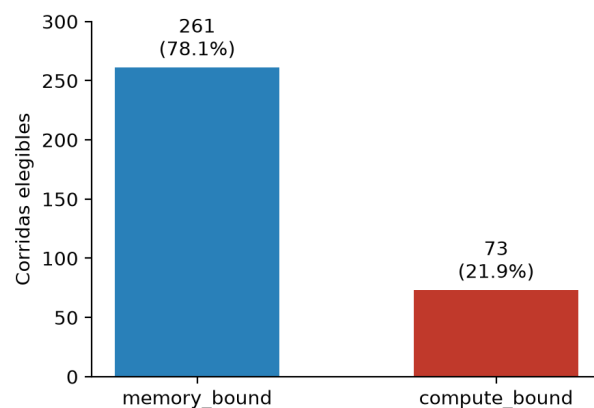


Figura 3.2: Distribución de las 334 corridas elegibles por clase. Fuente: elaboración propia.

La Tabla 3.3 y la Figura 3.3 presentan la posición Roofline final de los 18 kernels de la campaña, usando el ridge exacto de referencia de cada precisión (sección anterior) y marcando cuáles quedan efectivamente sin datos de entrenamiento utilizables tras la calibración de warmup.

Tabla 3.3: Los 18 kernels de la campaña final GPU: precisión, intensidad operacional (OI) declarada, ridge de referencia de su propia precisión, $\log_2(\text{OI}/\text{ridge})$ y zona Roofline resultante.

Kernel	Precisión	OI	Ridge REF	$\log_2$	Zona
<i>rodinia_lavamd</i>	fp64	708.140	6.898	+6.68	<i>compute-bound</i> (sin datos)
<i>gpu_cutlass_simt_dgemm_n4096</i>	fp64	26.730	6.898	+1.95	<i>compute-bound</i>
<i>rodinia_heartwall</i>	fp32	35.300	14.966	+1.24	<i>compute-bound</i>
<i>dual_cholesky_gpu_N2048</i>	fp64	8.215	6.898	+0.25	cerca del ridge
<i>rodinia_lud</i>	fp32	7.820	14.966	-0.94	cerca del ridge
<i>gpu_rajaperf_gemm</i>	fp64	2.810	6.898	-1.30	<i>memory-bound</i>
<i>dual_fft_gpu_N4096</i>	fp64	2.499	6.898	-1.46	<i>memory-bound</i>
<i>rodinia_dwt2d</i>	fp32	2.170	14.966	-2.79	<i>memory-bound</i> (sin datos)
<i>gpu_rajaperf_heat_3d</i>	fp64	0.746	6.898	-3.21	<i>memory-bound</i>
<i>dual_stencil_gpu_N1024</i>	fp64	0.495	6.898	-3.80	<i>memory-bound</i>
<i>gpu_rajaperf_jacobi_2d</i>	fp64	0.295	6.898	-4.55	<i>memory-bound</i>
<i>dual_spmv_gpu_N1000000</i>	fp64	0.274	6.898	-4.65	<i>memory-bound</i>
<i>dual_spmv_gpu_N200000000</i>	fp64	0.266	6.898	-4.70	<i>memory-bound</i>
<i>dual_stencil_gpu_N36864</i>	fp64	0.258	6.898	-4.74	<i>memory-bound</i>
<i>rodinia_gaussian</i>	fp32	0.281	14.966	-5.73	<i>memory-bound</i>
<i>dual_axpy_gpu_N10000000</i>	fp64	0.125	6.898	-5.79	<i>memory-bound</i>
<i>dual_axpy_gpu_N1280000000</i>	fp64	0.086	6.898	-6.33	<i>memory-bound</i>
<i>gpu_rajaperf_stream_triad</i>	fp64	0.083	6.898	-6.38	<i>memory-bound</i>

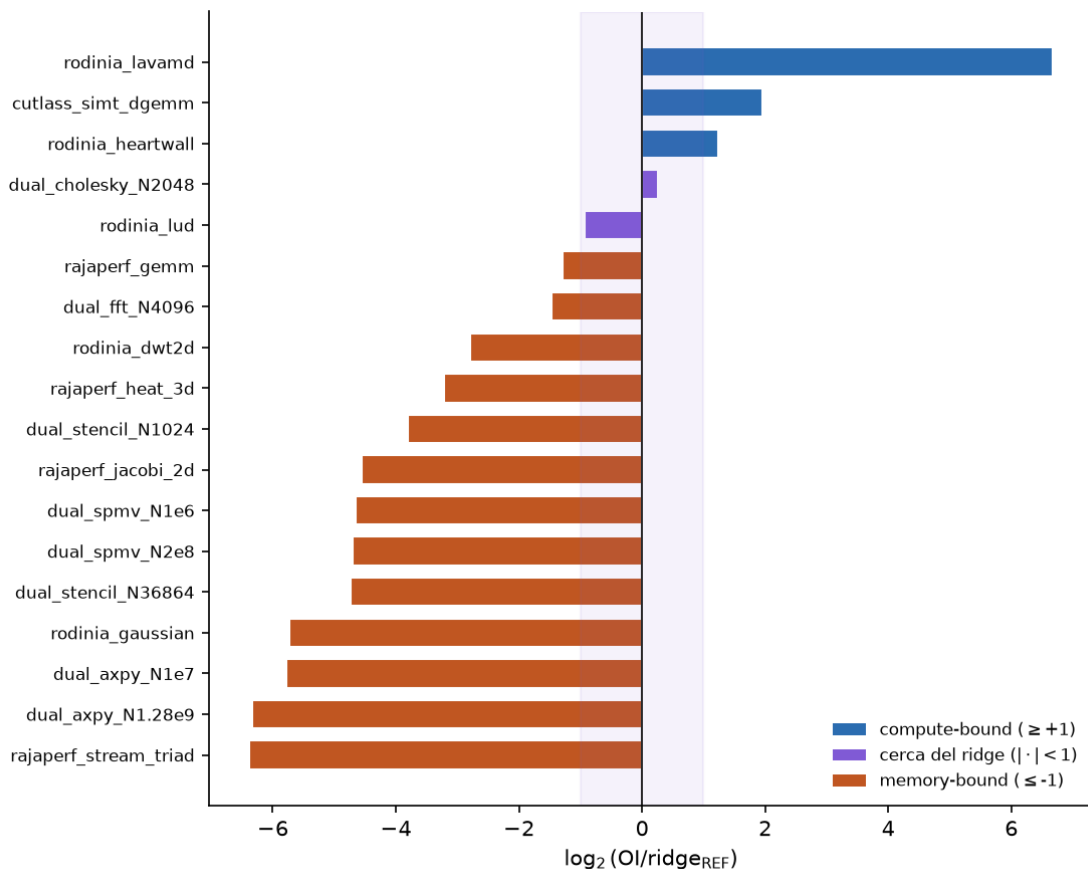


Figura 3.3: $\log_2(\text{OI}/\text{ridge}_{\text{REF}})$ de cada uno de los 18 kernels de la campaña final GPU, coloreado por zona Roofline respecto al ridge de su propia precisión aritmética. Fuente: elaboración propia.

Finalmente, la Figura 3.4 descompone, para cada uno de los 16 kernels con corridas utilizables tras la calibración de warmup, el porcentaje de sus corridas clasificadas *compute_bound* frente a *memory_bound*. Dado que el ridge se recalibra por nivel de frecuencia (sección 1.1.2), el veredicto de un kernel individual puede en principio oscilar entre niveles; en la práctica, 13 de los 16 kernels resultan completamente homogéneos (100 % de una sola clase en todas sus corridas elegibles), y solo *gpu_rajaperf_gemm* (50.0 %/50.0 %) y, en menor medida, *rodinia_lud* (80.0 % *compute_bound*) y *dual_fft_gpu_N4096* (16.7 % *compute_bound*) muestran una composición mixta — consistente con su posición cercana al ridge en la Tabla 3.3, donde un desplazamiento pequeño del techo entre niveles de frecuencia puede efectivamente cruzar el punto de inflexión.

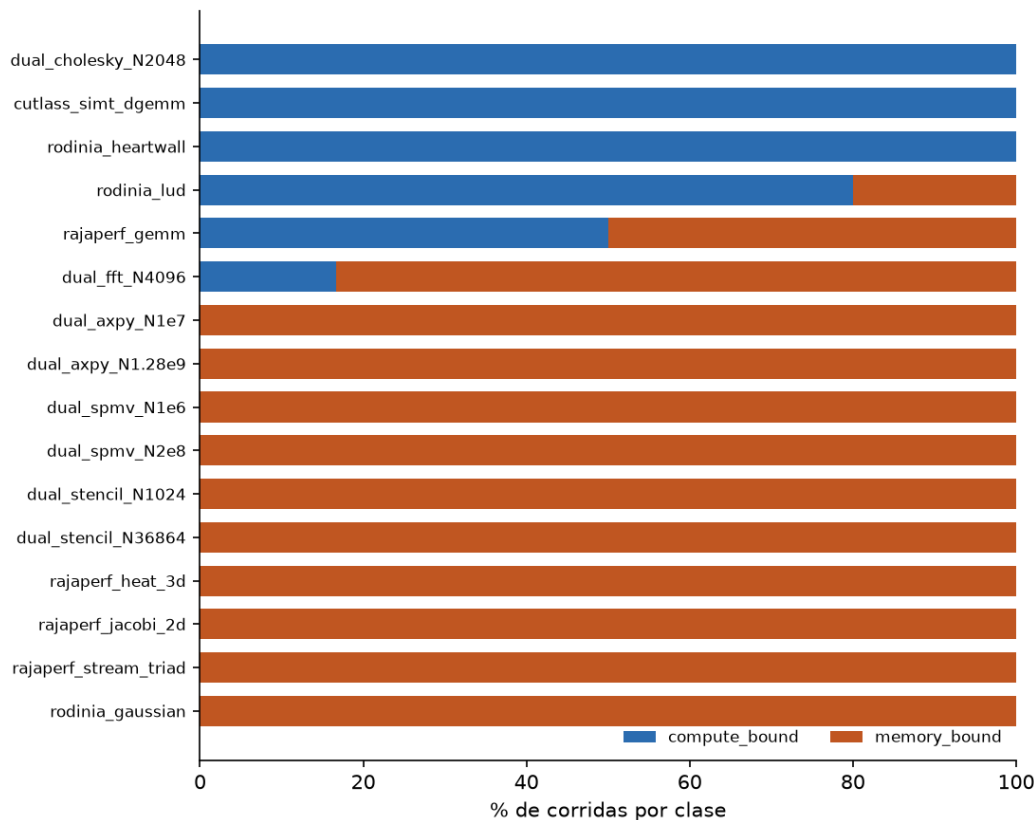


Figura 3.4: Porcentaje de corridas elegibles clasificadas *compute_bound* frente a *memory_bound*, por kernel, sobre las 334 corridas elegibles tras la calibración de warmup. Fuente: elaboración propia.

3.9. Análisis exploratorio y clasificador diagnóstico del dataset GPU

Aunque el entrenamiento del clasificador de fase corresponde formalmente a la Fase 2 de este proyecto, los resultados obtenidos hasta este punto de la Fase 1 — el conjunto de entrenamiento GPU ya definido y depurado (sección 3.8) — permiten un primer análisis diagnóstico real sobre datos reales, que se reporta aquí íntegramente por su valor directo para entender la calidad del propio conjunto de datos de Fase 1, independientemente de cualquier desarrollo posterior del agente de decisión.

3.9.1. Contrato de *features* y granularidad de entrenamiento

Cada corrida real de la campaña GPU (kernel $\times$ nivel de frecuencia $\times$ repetición) se resume en una única fila de `training_gpu_phases.csv`, agregando sus muestras NVML

mediante estadísticos robustos (mediana, media recortada, desviación estándar, rango intercuartílico, mínimo, máximo y fracción válida) sobre cinco señales: utilización de *streaming multiprocessors*, utilización de memoria, potencia, reloj de *SM* y temperatura. El entrenamiento no opera sobre muestras NVML crudas sino sobre estas corridas ya resumidas, por dos razones directamente vinculadas al diseño del instrumento: primero, el propio pipeline de post-procesamiento etiqueta y agrega a esta granularidad como parte de su contrato de salida, de modo que una fila de este archivo es la unidad experimental que efectivamente atraviesa las verificaciones de calidad y warmup descritas en la sección anterior; segundo, entrenar sobre muestras NVML individuales trataría observaciones altamente correlacionadas dentro de una misma corrida (decenas de muestras consecutivas del mismo proceso físico) como si fueran independientes, inflando artificialmente el tamaño efectivo de la muestra y sesgando cualquier validación cruzada que no las agrupe explícitamente por corrida.

El contrato de *features* se determinó con evidencia real, no por diseño asumido: un análisis de correlación de Pearson y Spearman, más el factor de inflación de varianza (VIF), sobre las 540 filas de la campaña completa. La temperatura de GPU (`gpu_temperature_c`) se descartó de las cuatro variantes evaluadas de agregación estadística porque las ocho variantes agregadas de esta señal correlacionan con `gpu_power_mw` por encima de 0.85 en todos los pares evaluados — consistente con que la temperatura observada es, en este régimen de operación, principalmente una consecuencia térmica de la potencia disipada y no una señal adicional independiente. El contrato final retiene cuatro *features*: la mediana de utilización de *SM*, la mediana de utilización de memoria, la mediana de potencia y la mediana de reloj de *SM*, con VIF entre 2.5 y 6.5 — sin colinealidad severa entre ellas. Ninguna variable derivada del propio proceso de etiquetado (intensidad operacional, ridge aplicado, o cualquier variante de la etiqueta de fase) se admite como *feature*, verificado mediante un guardarraíl explícito en el código de entrenamiento que aborta la ejecución si alguna de ellas aparece en el conjunto de columnas de entrada — evitando por construcción una fuga de información desde la propia variable objetivo.

La Figura 3.5 muestra la matriz de correlación de Pearson entre la variante mediana de las cinco señales NVML, recalculada sobre las 522 corridas con muestras NVML válidas de la campaña final ya reprocesada con warmup corregido (sección 3.8) — un conjunto distinto al de 540 filas sobre el que se ejecutó originalmente el análisis de contrato de *features*, pero que reproduce el mismo patrón que motivó la decisión: la correlación entre `gpu_temperature_c_median` y `gpu_power_mw_median` es 0.91, la más alta de toda la matriz y muy por encima del resto de pares entre las señales retenidas (0.50 a 0.53 entre utilización de *SM* y las demás, 0.82 entre utilización de memoria y potencia). El reloj de *SM* es la señal

más independiente del grupo, con correlación negativa y cercana a cero frente a la utilización de *SM* (-0.09) — consistente con que el reloj depende del nivel de frecuencia solicitado por la campaña y no del comportamiento del kernel en ese nivel, mientras que la utilización sí refleja directamente ese comportamiento.

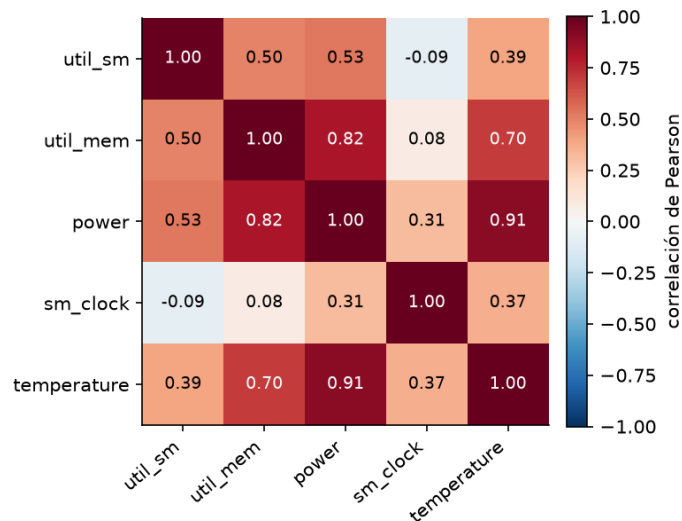


Figura 3.5: Correlación de Pearson entre la variante mediana de las cinco señales NVML disponibles, sobre las 522 corridas con NVML válido de la campaña final reprocesada. Fuente: elaboración propia.

3.9.2. Balanceo de clases y validación

El conjunto de entrenamiento final (334 corridas, 16 kernels, 11 familias algorítmicas) está desbalanceado: 73 corridas *compute_bound* frente a 261 *memory_bound* (21.9%/78.1%). Se aplicó `class_weight="balanced"` de *scikit-learn* a todos los modelos salvo el clasificador de referencia (mayoritaria, sin balanceo por diseño) y XGBoost, que en su lugar recibe `scale_pos_weight` calculado una sola vez sobre la razón global negativa/positiva del conjunto completo (0.2797) — simplificación documentada: el valor correcto se recalcularía por pliegue, no una sola vez.

La validación es *leave-one-familia-out*: cada pliegue retiene como prueba todas las corridas de una familia algorítmica completa, nunca solo una repetición o nivel de frecuencia suyo. La Tabla 3.4 resume el resultado del clasificador final sobre el conjunto de entrenamiento (334 corridas, 11 familias).

Tabla 3.4: Resultado del clasificador diagnóstico GPU final (*leave-one-familia-out*, 334 corridas, 11 familias).

Métrica	Valor
F1 macro	0.617
F1 <i>compute_bound</i>	0.197
F1 <i>memory_bound</i>	0.583

3.9.3. Selección de modelo y búsqueda de hiperparámetros

Resultados

Se compararon siete modelos bajo el mismo protocolo *leave-one-familia-out*: dos árboles fijos (profundidad 1 y 6), el clasificador de referencia (mayoritaria), y cuatro modelos con hiperparámetros ajustables (regresión logística, *random forest*, *extra trees*, XGBoost) afinados con Optuna (muestreador TPE, 30 intentos por modelo) anidado dentro de cada pliegue externo — la búsqueda de un pliegue nunca ve la familia retenida para prueba, y el modelo final se reoptimiza una última vez sobre las 11 familias completas antes de servirlo. La Figura 3.6 compara los siete por F1 macro.

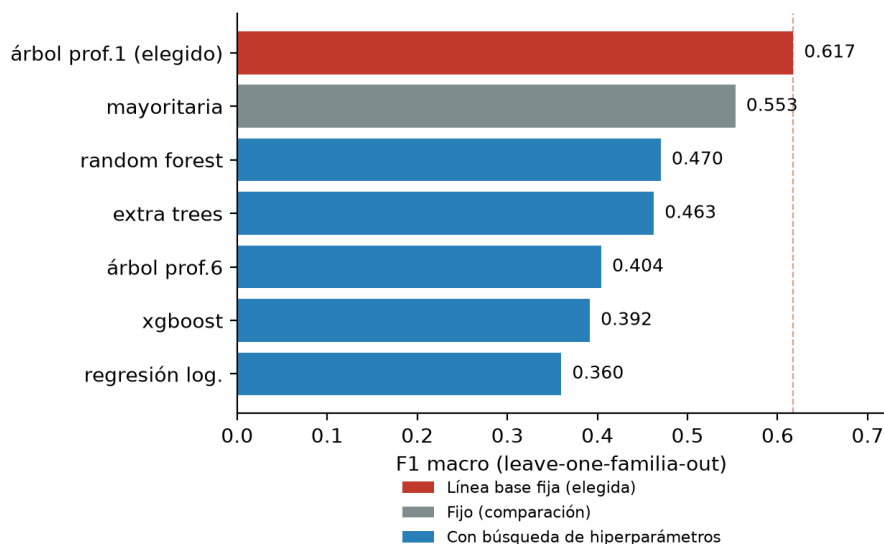


Figura 3.6: F1 macro (*leave-one-familia-out*) de los siete modelos comparados para el clasificador diagnóstico GPU. Fuente: elaboración propia.

El modelo serializado es el árbol de profundidad 1 (`arbol_prof1`): mejor F1 macro del conjunto (0.617) y menor latencia de inferencia ($p_{99} = 118 \mu s$), seleccionado mediante una

puntuación compuesta $\text{Score} = (1 - \overline{F1}_{\text{macro}}) + 0,2 \times (p_{99} / \text{máx}(p_{99}))$ que penaliza la latencia relativa al modelo más lento del conjunto comparado.

Discusión

Ningún modelo con hiperparámetros ajustables superó a la línea base fija, pese a una búsqueda real de 30 intentos por modelo. *Random forest* y *extra trees* obtienen el mejor F1 macro del grupo optimizado (0.470 y 0.463) a un costo de inferencia p99 de ~ 29 ms y ~ 88 ms — 250–750× mayor que el árbol elegido — consistente con evaluar cientos de árboles internos por predicción frente a una sola comparación de umbral. La Figura 3.7 sitúa a los siete modelos en el plano calidad–latencia.

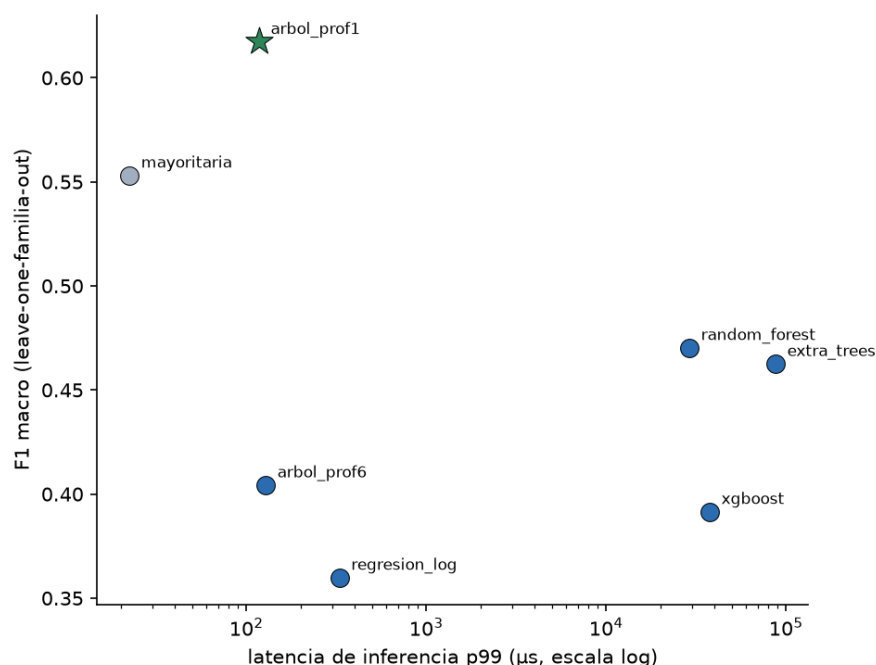


Figura 3.7: F1 macro frente a latencia de inferencia p99 por fila (escala logarítmica), para los siete modelos comparados. El modelo serializado se marca con una estrella. Fuente: elaboración propia.

La lectura más consistente con el tamaño del conjunto de datos: con 11 familias y una búsqueda anidada que subdivide aún más el entrenamiento de cada pliegue externo, cada evaluación interna de un candidato ve muy pocas familias — el régimen exacto en el que un modelo de alta varianza (bosques, *gradient boosting*) sobreajusta la partición en vez de generalizar, mientras que la simplicidad forzada de un árbol de profundidad 1 actúa como una regularización más fuerte que cualquier hiperparámetro que la búsqueda pueda encontrarle a un modelo más flexible.

3.9.4. Matriz de confusión y limitaciones del resultado

Resultados

La Figura 3.8 muestra la matriz de confusión acumulada sobre los 11 pliegues *leave-one-familia-out* del modelo final. De 73 corridas reales *compute_bound*, el modelo predice correctamente 33 (45.2%); de 261 *memory_bound*, predice correctamente 234 (89.7%).

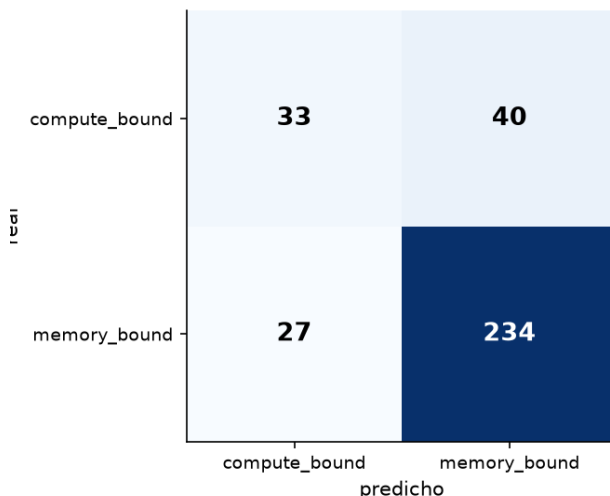


Figura 3.8: Matriz de confusión acumulada sobre los 11 pliegues de validación *leave-one-familia-out* (334 corridas). Fuente: elaboración propia.

La Tabla 3.5 reporta las métricas de clasificación estándar derivadas directamente de esa matriz acumulada.

Tabla 3.5: Métricas de clasificación estándar del modelo final, calculadas sobre la matriz de confusión acumulada (334 corridas).

Métrica	Valor
Exactitud (<i>accuracy</i>)	0.799
Exactitud balanceada (<i>balanced accuracy</i>)	0.674
Coefficiente de correlación de Matthews (MCC)	0.375
Precisión <i>compute_bound</i>	0.550
Sensibilidad (<i>recall</i>) <i>compute_bound</i>	0.452
Precisión <i>memory_bound</i>	0.854
Sensibilidad (<i>recall</i>) <i>memory_bound</i>	0.897

La exactitud simple (0.799) es engañosamente alta dado el desbalance de clase (261

memory_bound frente a 73 *compute_bound*): un clasificador que siempre predijera *memory_bound* ya alcanzaría 0.781. La exactitud balanceada (0.674) y el MCC (0.375, sobre una escala de -1 a $+1$) son las métricas apropiadas bajo este desbalance y confirman una capacidad de discriminación real pero moderada.

El F1 por clase reportado en la Tabla 3.4 (*compute_bound*= 0,197) es el promedio simple sobre los 11 pliegues, distinto del F1 derivado de la matriz acumulada de la Tabla 3.5 (precision= 0,550, recall= 0,452, $F1 \approx 0,496$). No es un error de cómputo: solo 5 de los 16 kernels utilizables son *compute_bound*, así que en la mayoría de los pliegues la familia excluida no contiene ninguna observación de esa clase, y el F1 de *compute_bound* en esos pliegues no aporta evidencia sobre la capacidad predictiva del modelo — bajo la convención de agregación empleada recibe valor cero, por lo que el promedio simple entre los 11 pliegues resulta sensiblemente inferior al F1 obtenido sobre las predicciones acumuladas. El 0.197 depende, en consecuencia, tanto del método de agregación como del desempeño real del modelo; se reportan ambas cifras para no sugerir, con una sola, una precisión distinta de la que el modelo realmente exhibe.

La Tabla 3.6 desglosa los 67 errores (20.1 % de las 334 corridas) por familia algorítmica, en el mismo protocolo *leave-one-family-out*.

Tabla 3.6: Errores del clasificador final por familia algorítmica (*leave-one-family-out*, 334 corridas). Familias con 0 errores omitidas.

Familia	Clase real predom.	n	Errores	comp.→mem.	mem.→comp.
<i>dual_cholesky</i>	<i>compute_bound</i>	16	16	16	0
<i>rodinia_gaussian</i>	<i>memory_bound</i>	18	15	0	15
<i>rajaperf_cuda</i>	<i>memory_bound</i>	66	12	0	12
<i>rodinia_lud</i>	<i>compute_bound</i>	15	12	12	0
<i>rodinia_heartwall</i>	<i>compute_bound</i>	15	9	9	0
<i>dual_fft</i>	<i>memory_bound</i>	18	3	3	0

Los errores no se reparten uniformemente: se concentran en 6 de las 11 familias, y las 5 restantes (*dual_axpy*, *dual_spmv*, *dual_stencil*, *gpu_cutlass_simt_dgemm_n4096*, *rajaperf_stream*) se clasifican sin ningún error. De las cuatro familias *compute_bound* del conjunto de entrenamiento, tres (*dual_cholesky*, *rodinia_lud*, *rodinia_heartwall*) fallan predominante o totalmente hacia *memory_bound*, y solo *gpu_cutlass_simt_dgemm_n4096* — la ancla *compute_bound* más profunda entre las que sí tienen datos de entrenamiento (Tabla

3.3) — se clasifica sin ningún error. *dual_cholesky* y *rodinia_lud* están, en efecto, cerca del ridge (Tabla 3.3), consistente con el patrón esperado; pero *rodinia_gaussian* es la familia *memory_bound* más alejada del ridge del catálogo ($\log_2(\text{OI}/\text{ridge}) = -5,73$) y aun así falla en 15 de sus 18 corridas, en dirección contraria (predicho *compute_bound*) — una falla que la cercanía al ridge no explica.

Esta falla sí es diagnosticable, dado que el árbol de profundidad 1 se reduce a una única regla de umbral. En el pliegue que excluye a *rodinia_gaussian* para prueba, el árbol entrenado sobre las 10 familias restantes queda como: `gpu_util_pct_median <= 77.5 ->memory_bound; >77.5 ->compute_bound`. Las 18 corridas de *rodinia_gaussian* tienen `gpu_util_pct_median` entre 76 y 93 — las 3 corridas con 76 caen del lado correcto, las 15 restantes (79–93) caen del lado *compute_bound* pese a ser, por Roofline, profundamente *memory_bound*. La causa es física, no un defecto del árbol: la utilización de *SM* que reporta NVML mide ocupación/actividad del *scheduler*, no densidad de FLOPs por byte — un kernel *memory_bound* que mantiene los *streaming multiprocessors* ocupados emitiendo y esperando accesos a memoria (sin quedar nunca ocioso) produce la misma señal alta de utilización que un kernel genuinamente saturado de cómputo. La comparación con dos familias correctamente clasificadas en sus propios pliegues lo confirma: *gpu_cutlass_simt_dgemm_n4096* (*compute_bound* real) satura en 100 % de utilización, muy por encima de su propio umbral (93.5); *dual_axpy* (*memory_bound* real) queda entre 25 % y 67 %, muy por debajo. El umbral mismo tampoco es estable: 77.5 en el pliegue de *rodinia_gaussian* frente a 93.5 en el de *gpu_cutlass_simt_dgemm_n4096* — un desplazamiento de 16 puntos porcentuales según qué familias componen el entrenamiento, evidencia adicional de que la regla de un solo umbral no generaliza como una frontera Roofline universal, sino que se reajusta a la composición específica de cada pliegue.

Discusión

El F1 bajo de *compute_bound* tiene, además del desbalance de clase, una causa estructural en la composición del catálogo frente al propio protocolo de validación. La Figura 3.4 ya muestra que 13 de los 16 kernels utilizables resultan homogéneos (100 % de una sola clase en todas sus corridas): la intensidad operacional de un kernel es, en la práctica, una propiedad casi fija del algoritmo, mientras que el ridge solo se desplaza levemente entre niveles de frecuencia (sección 1.1.2), así que solo los kernels cuya posición cae muy cerca del ridge (*rodinia_lud*, *dual_fft*, *gpu_rajaperf_gemm*) llegan a cruzarlo alguna vez. Como *leave-one-familia-out* retiene familias completas para prueba, esa homogeneidad se traduce en que 8 de los 11 pliegues quedan con soporte cero para una de las dos clases: el modelo no puede

acertar ni fallar ahí una clase que simplemente no aparece en ese examen. La convención de *scikit-learn* le asigna $F1=0$ a la clase ausente en cada uno de esos pliegues (sección anterior), así que el promedio de 11 pliegues queda dominado por ceros que no reflejan ningún error real del modelo, sino la estructura del catálogo. Esta característica es del protocolo y del catálogo, no de un modelo en particular: los 11 pliegues son idénticos para los 7 modelos comparados (Tabla 3.4), así que los siete cargan con la misma penalización mecánica sobre su $F1$ de *compute_bound*.

Sobre esa base común, hay un factor adicional que sí distingue a XGBoost del resto: mientras los demás modelos recalculan `class_weight="balanced"` automáticamente en cada ajuste (y por tanto en cada uno de los 11 pliegues, con una proporción de clase que varía entre ellos porque cada pliegue excluye una familia distinta), XGBoost usa un `scale_pos_weight` fijo, calculado una sola vez sobre las 334 filas completas (0.2797). El balance real de entrenamiento por pliegue oscila entre 0.211 y 0.363 según qué familia queda afuera, así que ese peso fijo se aleja hasta un 30 % del valor que le correspondería a cada pliegue. Es consistente con que XGBoost obtenga el $F1$ *compute_bound* más bajo de los cuatro modelos con `class_weight` o equivalente (0.151, frente a 0.229 de *Random Forest* y *Extra Trees*), aunque no explica por sí solo la brecha completa frente a esos dos modelos, que comparten el mismo peso de clase dinámico que *arbol_prof1*.

Ninguna de las dos causas anteriores explica, sin embargo, por qué los modelos de mayor capacidad (*Random Forest*, *Extra Trees*, XGBoost, regresión logística) terminan con $F1$ macro *peor* que el árbol de profundidad 1: eso responde a un mecanismo distinto, discutido a continuación.

La debilidad de *compute_bound* tiene también una explicación directa más allá del desbalance ya corregido: solo 73 corridas de esa clase, repartidas entre 5 kernels (frente a 261 corridas / 11 kernels de *memory_bound*), y un criterio de validación que retiene familias completas fuera del entrenamiento en cada pliegue dejan al modelo con muy pocos ejemplos independientes sobre los cuales generalizar. Perder la ancla *compute_bound* más profunda del catálogo (*rodinia_lavamd*, sin corridas utilizables tras la calibración de warmup) agrava directamente esa escasez; *rodinia_dwt2d* se pierde en la misma corrección pero es *memory-bound* (Tabla 3.3), así que su ausencia no afecta a la clase minoritaria.

Un $F1$ macro de 0.617 supera al clasificador de referencia (0.553) pero permanece débil en términos absolutos. No se investigó todavía si alargar la duración de corrida de *rodinia_lavamd/rodinia_dwt2d* — mismo criterio ya aplicado a otros kernels (sección 3.7) — recuperaría ventanas de estado estable suficientes tras su warmup real (6.4–6.5 s); sería el

paso natural para recuperar la representación *compute_bound* profunda perdida con *rodinia_lavamd*.

3.10. Ampliación del catálogo con kernels de patrón de acceso irregular

Resultados

El plan de trabajo exige, junto a los kernels de acceso regular (*dual_**) ya documentados, kernels de patrón de acceso irregular de la suite Rodinia. Se compilaron 9 binarios nuevos: 4 CPU (*kmeans*, *srad*, *nw*, *particlefilter*) y 5 GPU (*kmeans*, *srad*, *nw*, *b+tree*, *cfid* — *particlefilter* no tiene variante GPU en este catálogo) con scripts de *build* reproducibles que fijan el commit de origen y el hash de cada archivo fuente, siguiendo el mismo patrón ya empleado para *rodinia_lavamd* (`old/scripts/pacca/build_rodinia_lavamd_omp.sh`). Los cuatro binarios CPU (`gcc`) resultaron además reproducibles a nivel de binario — mismo hash `sha256` en dos compilaciones independientes desde la misma fuente —; los cinco binarios GPU (`nvcc`) no lo son (metadatos de depuración con rutas temporales aleatorias, aunque el código ejecutable es idéntico), así que su reproducibilidad se fija únicamente a nivel de fuente. Los 4 kernels CPU se calibraron al piso de duración de corrida de la campaña (criterio F1-CPU-006, $\sim 25\text{--}30$ s de ventana de estado estable); la Tabla 3.7 resume sus parámetros finales.

Tabla 3.7: Parámetros finales calibrados de los 4 kernels CPU de patrón de acceso irregular.

Kernel	Parámetros finales	Tiempo
<i>kmeans_omp</i>	1 000 000 puntos $\times$ 32 <i>features</i> , 6 hilos	31.4 s
<i>srad_omp</i>	6144 $\times$ 6144, 100 iteraciones, 6 hilos	27.5 s
<i>nw_omp</i>	$N = 122\,000$, <i>penalty</i> =10, 6 hilos	25.2 s
<i>particlefilter_omp</i>	212 000 partículas, 128 $\times$ 128 $\times$ 10, 6 hilos	28.3 s

Discusión

La compilación de *nw* (Needleman–Wunsch) expuso un desbordamiento real de enteros de 32 bits en el código fuente original de Rodinia (2010): `malloc(max_rows*max_cols*sizeof(int))` con dimensiones declaradas `int` desborda para $N \gtrsim 46\,000$, produciendo un tamaño de asignación corrupto y un fallo de segmento en tiempo de ejecución — confirmado que no es un límite físico de memoria del nodo (240 GB libres). Se corrigió retipando las dos variables de

dimensión a `long` (con los `malloc` correspondientes convertidos a `size_t`), sin ningún otro cambio al algoritmo; verificado que el parche no introduce ninguna regresión de rendimiento frente al binario sin parchear ($N = 46\,000$: 3.72 s original frente a 3.68 s parcheado).

Capítulo 4

Discusión

Los resultados de esta fase sostienen dos tipos de contribución distintos: uno metodológico, sobre cómo construir con rigor un instrumento de caracterización DVFS antes de confiar en él; y uno empírico, sobre el comportamiento real de la plataforma de medición frente a ese instrumento y sobre el primer clasificador de GPU entrenado con ese instrumento.

Buena parte de la literatura revisada en el estado del arte — los métodos de selección de frecuencia óptima de Ali et al. [1] y de clasificación consciente de DVFS de Guerreiro et al. [24], o el marco de control por refuerzo de Wang et al. [56] — asume, de forma razonable dado su alcance, que la escritura de frecuencia sobre el hardware subyacente funciona como el fabricante la documenta, y concentra su aporte en la capa de decisión: qué frecuencia elegir, o cómo aprenderla. Los resultados de actuación muestran que esa suposición debe verificarse empíricamente bajo carga real: en GPU, la prueba positiva sostuvo dos objetivos distintos durante cientos de lecturas activas y confirmó el mecanismo; en CPU, en cambio, la relectura correcta de los límites escritos quedó refutada como prueba de actuación suficiente cuando el nuevo muestreo durante la carga mostró que el turbo global mantenía una frecuencia distinta. Para cualquier política construida sobre estos actuadores, la diferencia entre éxito administrativo, estado solicitado y efecto físico solo aparece al contrastar directamente el hardware durante el trabajo real. La medición de bytes de *uncore* (sección 3.5) confirma el mismo principio por una vía independiente: un permiso comunicado como concedido no coincidió inicialmente con el estado verificable del sistema, y solo la comprobación directa contra el binario real dejó en evidencia la brecha y permitió confirmar su corrección posterior.

En la misma línea, la corrección del chequeo de carga externa (sección 3.2) ilustra un riesgo metodológico general en la validación de instrumentos de medición sobre cargas propias: una verificación de seguridad diseñada correctamente para un escenario de prueba corto

puede fallar sistemáticamente a escala real, precisamente porque a esa escala dispone de tiempo suficiente para que la señal que observa converja hacia un valor indistinguible de una condición de alarma legítima. Este tipo de falla — a diferencia de un error de sintaxis o de una condición de contorno mal definida — no se manifiesta en pruebas unitarias herméticas ni en una campaña breve; solo aparece al ejercitar el instrumento en las condiciones reales para las que fue diseñado, lo que refuerza la elección metodológica de este trabajo de tratar la campaña real como parte del proceso de validación del instrumento, y no solo como su producto final.

El hallazgo de la sección 3.5.1 extiende ese mismo principio en una dirección que las secciones anteriores no habían cubierto todavía: no basta con verificar que cada mecanismo del instrumento funciona por separado — la escritura de frecuencia, la lectura de *uncore*, la medición directa de FLOPs —, porque dos mecanismos correctos en aislamiento pueden interferir entre sí de una forma que ninguna prueba unitaria, centrada por construcción en un solo mecanismo a la vez, puede exponer. El presupuesto de contadores simultáneos verificado como sostenible en la sección 2.3.11 no era, en sentido estricto, incorrecto en el momento en que se verificó; dejó de sostenerse cuando cambió una condición del entorno — una reserva de contador realizada por un mecanismo del propio sistema operativo, ajeno tanto al instrumento como al proyecto — que ninguna verificación anterior tenía motivo para volver a comprobar. La lección metodológica no es que el presupuesto físico de contadores deba tratarse como una constante de la plataforma, sino que ninguna capacidad de hardware nominal — por verificada que haya estado en un momento dado — puede asumirse estable en el tiempo sin volver a contrastarla contra el estado real del sistema; es la misma disciplina de verificación de las secciones 1.1.5 y 1.1.3, aplicada aquí no a la existencia de un permiso, sino a su persistencia.

Esa misma sección ilustra, además, el valor de una práctica adoptada explícitamente en la segunda mitad de este trabajo: antes de comprometer una campaña real de varias horas, ejecutar una réplica reducida — de minutos, sobre una fracción del catálogo o de la matriz de frecuencias — que ejerce el mismo camino de código de principio a fin. El defecto que esa práctica expuso no era detectable por inspección de código ni por las pruebas unitarias ya existentes, que verifican el formato de los datos y la degradación controlada del instrumento, pero no su interacción con el estado dinámico del sistema operativo bajo carga real; y de no haberse expuesto en una prueba de minutos, se habría descubierto — en el mejor de los casos — al final de una campaña de horas, con el consiguiente costo de cómputo perdido, o — en el peor — habría contaminado silenciosamente el conjunto de datos resultante, dado que una ventana con contadores degradados se descarta de la clasificación pero no impide

que la corrida se acepte si el resto de sus ventanas son suficientes. Generalizar esta práctica — una prueba de humo a escala reducida como paso obligatorio antes de cualquier campaña de producción, no solo la primera — es, junto con la verificación empírica ya discutida, un segundo aporte metodológico de esta fase.

La validación cruzada de los techos Roofline (sección 3.6) constituye una tercera instancia del mismo principio, aplicada esta vez a la integridad de la propia calibración y no a un permiso o a un mecanismo de actuación. Su hallazgo comparte una estructura común con los dos anteriores: un componente que producía un resultado plausible — un microbenchmark de calibración que compilaba, corría y reportaba una cifra de rendimiento sin ningún error visible — resultó, contrastado contra un mecanismo de medición independiente, sistemáticamente incorrecto por casi un orden de magnitud, sin que ninguna verificación interna del propio instrumento pudiera haberlo detectado: la ausencia de error no es evidencia de corrección cuando el propio mecanismo de verificación comparte la misma fuente de sesgo que el componente que pretende verificar. La elección deliberada de aplicar la corrección resultante — el ancho vectorial explícito de compilación — de forma selectiva y no uniforme entre microbenchmarks, tras encontrar que beneficiaba a uno y perjudicaba a otro, refuerza además un punto ya presente en la discusión de la sección 3.3: una corrección que soluciona un defecto real en un caso concreto no debe generalizarse sin verificar, caso por caso, que el mecanismo físico que la justifica aplica igual en cada uno.

El hallazgo de precisión mixta en el catálogo de GPU (sección 3.3) es relevante más allá de este trabajo concreto. La práctica de derivar la precisión de caracterización de un kernel a partir de su tipo de dato declarado en el código fuente — en lugar de la precisión efectivamente ejercitada por el hardware durante su ejecución — es razonable como primera aproximación, pero este trabajo encontró que puede diferir de lo medido en más del 70 % del trabajo aritmético de un kernel, y que esa diferencia puede situarse exactamente en el margen que decide una clasificación Roofline. Ningún trabajo de los revisados en el estado del arte reporta explícitamente esta verificación para GPU, lo que sugiere que la práctica de asumir la precisión declarada, sin contrastarla, podría no ser inusual en caracterizaciones similares.

Finalmente, el resultado sobre precisión de `gpu_dgemv_n4096` (sección 3.3) matiza — sin contradecir — el hallazgo de Guerreiro et al. sobre la sensibilidad de las cargas GPGPU a la ruta de ejecución elegida por bibliotecas de terceros [24]: una ruta acelerada por hardware especializado invalida efectivamente un método de caracterización diseñado para instrucciones aritméticas convencionales, pero no invalida necesariamente la magnitud medida, siempre que exista una vía de verificación independiente — en este caso, el conteo analítico del propio

algoritmo — contra la cual contrastarla.

Capítulo 5

Conclusiones

Este trabajo completó la construcción y validación empírica del instrumento de caracterización CPU–GPU de la Fase 1, pero el conjunto DVFS final aún requiere cerrar los dos recorridos experimentales pendientes. Respecto al primer objetivo específico — caracterizar cargas representativas bajo distintos estados de frecuencia —, el resultado es parcial: en CPU, la campaña limpia de reemplazo terminó con 126/126 combinaciones aceptadas e incorpora FLOPs por hardware, bytes reales de *uncore* y el ridge corregido, pero el muestreo posterior de frecuencia efectiva demostró que F0–F4 no constituyeron estados físicos distintos mientras el turbo global permaneció activo; por ello esos datos no sustentan todavía comparaciones DVFS de tiempo, energía o EDP. En GPU, el catálogo quedó caracterizado y auditado por precisión, el instrumento multi-frecuencia está implementado y `-lgc` ya retiene el reloj bajo carga real, pero falta ejecutar y aceptar la campaña completa. Los objetivos específicos restantes — entrenamiento del clasificador, implementación del agente y evaluación de EDP — corresponden a las Fases 2–4 y no se presentan como ejecutados.

El aporte metodológico central de esta fase, más allá de los datos concretos recolectados, es haber demostrado — y no solo enunciado — que ningún mecanismo de este instrumento se dio por funcional sin verificación directa contra el estado real del hardware: ni la disponibilidad de un contador de PMU, ni el presupuesto físico de contadores simultáneos, ni su persistencia en el tiempo frente a mecanismos ajenos al instrumento, ni la escritura de frecuencia de CPU, ni la precisión aritmética declarada de un kernel de GPU, ni la calibración de los propios techos del modelo Roofline. En varias ocasiones documentadas en este trabajo, esa verificación expuso una falla real que una confianza no verificada en la documentación del fabricante, en una medición previa ya dada por válida, o en el diseño original del instrumento, habría dejado pasar silenciosamente hacia el conjunto de datos.

5.1. Limitaciones

Este trabajo reconoce las siguientes limitaciones. Primera, aunque la clasificación CPU depende exclusivamente de bytes medidos mediante *uncore*, esa señal tiene un piso práctico de intervalo de aproximadamente 10 ms, más grueso que las ventanas de CPU de aproximadamente 1 ms; la intensidad y la etiqueta son, por tanto, propiedades del intervalo *uncore* difundidas a las ventanas que cubre, no mediciones independientes a 1 ms. Segunda, el presupuesto simultáneo de PMU depende del estado de mecanismos del sistema operativo ajenos al proyecto; se redujo a nueve eventos para convivir con la reserva encontrada, pero una reserva futura adicional exigiría repetir la verificación. Tercera, el control CPU por núcleo no domina el turbo global en la configuración actual: el lector por ventana permite detectar la divergencia, pero resolverla requiere autoridad administrativa sobre `no_turbo` o `max_perf_pct`; la temperatura de paquete tampoco está conectada a un gate bloqueante de sensor. Cuarta, el catálogo y los *encodings* crudos están restringidos a la microarquitectura de la Tabla 2.1; no se reclama portabilidad de los mecanismos de actuación sin repetir sus verificaciones bajo carga en cada estado de plataforma. Quinta, aunque el actuador GPU ya fue validado en dos frecuencias bajo carga, todavía no existe una campaña GPU multi-frecuencia completa aceptada, y la campaña CPU disponible no contiene niveles efectivos diferenciados.

5.2. Trabajo Futuro

En el corto plazo, tres tareas siguen directamente habilitadas. Primera, resolver con la administración el control del turbo global de CPU y repetir la campaña limpia, usando el lector por ventana para aceptar cada nivel por su frecuencia efectiva y no solo por la relectura de sysfs. Segunda, ejecutar la campaña GPU multi-frecuencia ahora que `-lgc` fue verificado bajo carga, comenzando por una prueba de humo que ejercite el mismo producto cartesiano y los mismos gates de la campaña final. Tercera, consolidar ambos conjuntos y definir la partición experimental de la Fase 2 por corrida o carga, evitando tratar como observaciones independientes ventanas que comparten una etiqueta de intervalo *uncore* o una intensidad GPU caracterizada por kernel.

Bibliografía

- [1] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [2] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [4] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga. The NAS parallel benchmarks. *The International Journal of Supercomputer Applications*, 5(3):63–73, 1991. doi: 10.1177/109434209100500306.
- [5] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages*, 1 (OOPSLA):1–27, 2017. doi: 10.1145/3133876.
- [6] David A. Beckingsale, Jason Burmark, Rich Hornung, Holger Jones, William Killian, Adam J. Kunen, Olga Pearce, Peter Robinson, Brian S. Ryujin, and Thomas R. W. Scogland. RAJA: Portable performance for large-scale scientific applications. In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, pages 71–81, 2019. doi: 10.1109/P3HPC49587.2019.00012.
- [7] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

- [8] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- [9] Kay H. Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *2010 20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.
- [10] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccone. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.
- [11] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [12] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54, 2009. doi: 10.1109/IISWC.2009.5306797.
- [13] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [14] Davide Chicco and Giuseppe Jurman. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.
- [15] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [16] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [17] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.

- [18] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.
- [19] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous java performance evaluation. In *Proc. 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*, pages 57–76, Montreal, QC, Canada, 2007. doi: 10.1145/1297027.1297033.
- [20] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [21] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
- [22] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [23] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [24] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [25] R. Hebbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.
- [26] Myles Hollander, Douglas A. Wolfe, and Eric Chicken. *Nonparametric Statistical Methods*. John Wiley & Sons, 3rd edition, 2014. doi: 10.1002/9781119196037.
- [27] IBM. What is random forest? IBM Think Topics, 2026. URL <https://www.ibm.com/think/topics/random-forest>. Accessed: Apr. 3, 2026.
- [28] Intel. Cpu metrics reference. Intel VTune Profiler User Guide, 2026. URL <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/cpu-metrics-reference.html>. Accessed: Aug. 7, 2026.

- [29] Intel Corporation. *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021. URL <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>. Revision 1.0.
- [30] Intel Corporation. Roofline analysis with intel advisor. Intel Advisor User Guide, 2026. URL <https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html>. Accessed: Aug. 12, 2026.
- [31] Intel Corporation. perfmon: Performance monitoring events for intel architecture processors (evento fp_arith_inst_retired, familia ice lake), 2026. URL <https://github.com/intel/perfmon>. Accessed: Aug. 10, 2026.
- [32] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [33] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [34] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [35] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [36] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [37] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.
- [38] Linux Kernel Documentation. intel_pstate cpu performance scaling driver, 2026. URL https://docs.kernel.org/admin-guide/pm/intel_pstate.html. Accessed: Sep. 5, 2026.

- [39] Linux man-pages project. `clock_gettime(3)` — linux manual page, 2026. URL https://man7.org/linux/man-pages/man3/clock_gettime.3.html. Accessed: Aug. 29, 2026.
- [40] Linux man-pages project. `perf_event_open(2)` — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.
- [41] Linux perf project. `perf-stat(1)` — linux manual page, 2026. URL <https://man7.org/linux/man-pages/man1/perf-stat.1.html>. Accessed: Aug. 29, 2026.
- [42] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [43] Yu Jung Lo, Samuel Williams, Brian Van Straalen, Terry J. Ligocki, Matthew J. Cordery, Nicholas J. Wright, Mary W. Hall, and Leonid Oliker. Roofline model toolkit: A practical tool for architectural and program analysis. In *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation (PMBS 2014)*, volume 8966 of *Lecture Notes in Computer Science*, pages 129–148. Springer, 2015. doi: 10.1007/978-3-319-17248-4_7.
- [44] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. Technical report, IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, December 1995.
- [45] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [46] Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 9th edition, 2017. ISBN 9781119299363.
- [47] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.
- [48] NVIDIA. Nvidia management library (nvml). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/index.html>. Accessed: Apr. 3, 2026.

- [49] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [50] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [51] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [52] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.
- [53] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [54] J. Treibig, G. Hager, and G. Wellein. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proc. 39th International Conference on Parallel Processing Workshops (ICPPW)*, pages 207–216, San Diego, CA, USA, 2010. doi: 10.1109/ICPPW.2010.38.
- [55] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [56] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.
- [57] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.

- [58] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [59] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>